



**Faculty of Engineering & Technology – Electrical & Computer
Engineering Department**

first Semester 2025 – 2026

ENCS3320, Computer Networks

Project 1 – Socket Programming

Team Members

Yasmin badwan	1230271	Section: 3
Sewar masri	1231240	Section: 3
Hanaa Rayyan	1230263	Section: 3

Instructor: Dr. IbrahimNemar

Submission Date:21,12, 2024

Abstract

This project presents the design and implementation of a comprehensive networking system that demonstrates fundamental concepts in computer networks using low-level socket programming. The project is divided into three main parts.

The first part focuses on understanding and experimenting with essential networking tools such as ping, traceroute, nslookup, and telnet, in addition to analyzing DNS traffic using Wireshark to gain practical insight into network behavior and latency.

The second part involves implementing TCP and UDP client-server applications. TCP is used for reliable control communication through START and END messages, while UDP is used to transmit a large sequence of packets efficiently. The server analyzes the received data by counting the total number of packets and detecting out-of-order packet delivery, highlighting the differences between reliable and unreliable transport protocols.

The third part demonstrates the development of a simple yet complete web server using raw socket programming without relying on web frameworks. The server supports HTTP requests, serves static content such as HTML, CSS, and images, and implements HTTP status codes and redirection. In addition, the system provides secure user registration and authentication using password hashing and session management through cookies, allowing persistent user access across browser tabs.

Overall, this project offers hands-on practice in core networking concepts, transport protocols, and web server design, improving the understanding of network communication and client-server interaction.

Table of content

Abstract.....	1
Table of content	2
Table of figure	4
Theory	6
Computer Network Diagnostic Tools.....	6
Ping (Packet Internet Groper):	6
Traceroute (Tracert):	6
Nslookup:.....	6
Talnet:	6
Internet Control Message Protocol (ICMP)	7
RTT(Round Trip Time)	7
Echo-Request and Reply Message	7
DNS System and Name Resolution	8
Wireshark	8
Transport Layer Protocols and Socket Programming	9
Socket Programming.....	9
client-server model.....	9
TCP vs UDP Protocols	9
HTTP Protocol and Web Server Design.....	10
HTTP Request/Response:	11
HTTP methods	11
HTTP Status Codes	11
Serving Static Content	12
HTTP Cookies and Session Management	13
Procedure.....	14
Part one:	14
Question 1: (Network Tools Definitions):.....	14
Ping:	14
Tracert:	14
NsLookup:.....	14
Telnet:	14
Running commands & brief Explanation of the outputs:.....	15

1-Ping a Local Device (smartphone) :.....	15
2-Ping 1.1.1.1 :.....	16
3-Tracert 1.1.1.1:.....	17
4-Nslookup 1.1.1.1 :.....	17
Capture DNS Messages with Wireshark :.....	18
Part two:.....	19
Purpose of the Server	19
Server Code:.....	19
Client Side Implementation:	24
Testing Part 2:	28
Part three:.....	31
Objective of this part:.....	31
Web Server (System Architecture):	32
Web Page Design Using HTML and CSS	32
Web Server Implementation (server.py).....	53
Testing and Results	68
Server Startup.....	68
English Main Page Tests	68
Arabic Main Page Test	73
CSS File Test.....	76
Image Files Test (png/jpg).....	77
Page Not Found (404) Test.....	79
307 Redirect test :	80
User Registration Tests.....	83
User Login Tests	85
Cookie Validation Test	89
Logout Test	91
Team work	92
Conclusion.....	93
References:	94

Table of figure

Figure 1:RTT(Round Trip Time) Measurement[6]	7
Figure 2:ICMP Request-Response[8]	8
Figure 3:Socket programming [10].....	9
Figure 4:HTTP connection [13].....	10
Figure 5:HTTP request/response[13].....	11
Figure 6:404 not found.....	12
Figure 7:HTTP Cookies and Session[16]	13
Figure 8:ping command output to local smartphone device	15
Figure 9:ping command output to DNS server	16
Figure 10:Traceroute output showing path to 1.1.1.1	17
Figure 11:Nslookup reverse DNS query for 1.1.1.1	17
Figure 12:Captured DNS query and response messages using Wireshark	18
Figure 13:Captured DNS query and response messages using Wireshark	19
Figure 14 Server Waiting State	28
Figure 15 Client Sending Data.....	29
Figure 16 Server Results and Verification	30
Figure 17:main_en.html page	39
Figure 18:main_en.html page	39
Figure 19:main_en.html page	40
Figure 20:main_en.html page	41
Figure 21:main_ar.html page	42
Figure 22:main_ar.html page	42
Figure 23:main_ar.html page	43
Figure 24:main_ar.html page	44
Figure 25:Registration page interface displaying the form for new user account creation	47
Figure 26:login page	50
Figure 27:protected page.....	52
Figure 28:Server running successfully on port 8099	68
Figure 29:English main page loaded using root path.....	68
Figure 30:Terminal output showing GET of en.page	69
Figure 31:Terminal output showing GET of en.page	70
Figure 32:Terminal output showing GET en.page	71
Figure 33:English page accessed using /en.....	72
Figure 34:English page accessed using main_en.html.	72
Figure 35:Arabic main page.....	73
Figure 36:Terminal output showing GET of Arabic page.....	74
Figure 37:Terminal output showing GET of Arabic page.....	75
Figure 38:Terminal output showing GET of Arabic page.....	76

Figure 39:Webpage styled using CSS.....	76
Figure 40:Terminal output showing GET css file	77
Figure 41;Image file(png) served by the web server.	77
Figure 42:Terminal output showing GET of image png.....	78
Figure 43:Image file(jpg) served by the web server.	78
Figure 44:Terminal output showing GET of image jpg.....	78
Figure 45:Custom 404 error page.	79
Figure 46:Terminal output showing GET of not found page.....	79
Figure 47:chatgpt page.....	80
Figure 48: Terminal output showing GET of chat page.....	80
Figure 49: cloudeflare page	81
Figure 50: Terminal output showing GET of cloudeflare page	81
Figure 51:Ritaj page.....	82
Figure 52: Terminal output showing GET of Ritaj page.....	82
Figure 53:register page	83
Figure 54:Terminal output showing GET of register page	83
Figure 55:register page with name yasmin and pasword 123	84
Figure 56:Terminal output showing post of register page	84
Figure 57:data.txt with hashed password.....	84
Figure 58:Registration rejected.....	85
Figure 59:login page	86
Figure 60:Terminal output showing get of login page.....	86
Figure 61:login page with wrong pasword	87
Figure 62:login fail page.....	87
Figure 63:Terminal output showing post of login page	87
Figure 64:Protected page after login.....	88
Figure 65: Terminal output showing post of login page &&Session cookie created.	88
Figure 66:Protected Page Access (Authorization Check).....	89
Figure 67:Session Cookie Persistence in New Browser Tab	89
Figure 68:Session Validation in Server Terminal.....	90
Figure 69:Logout redirection.	91
Figure 71:Contribution distribution among team members (Yasmeen, Hanaa, Sewar).	92

Theory

Computer Network Diagnostic Tools

Ping (Packet Internet Groper):

Ping is a basic network diagnostic tool used to check the availability of a device on a network and measure the round-trip time (RTT) between two hosts using ICMP messages. It helps determine whether a destination is reachable and how long data takes to travel back and forth. Ping is widely used by network administrators to troubleshoot connectivity, latency, and network congestion issues. It is especially important for applications that require fast and real-time communication, such as online gaming and video calls.[\[1\]](#)

Traceroute (Tracert):

Tracert (Traceroute) is a network diagnostic command used to trace the path that data packets take from a source device to a destination across an IP network. It displays each intermediate router (hop) and measures the round-trip time (RTT) to help identify where delays or failures occur. Network administrators use Tracert to troubleshoot connectivity problems, detect routing issues, and locate network bottlenecks. It is especially useful for diagnosing slow connections and understanding how data flows through the network.[\[2\]](#)

Nslookup:

Nslookup is a command-line tool used to retrieve DNS information about a domain name, such as its IP address, nameservers, and DNS records. It helps troubleshoot DNS-related issues by checking whether a domain is resolving correctly and whether the DNS server is responding. Nslookup also supports reverse DNS lookups by finding the domain name associated with an IP address using PTR records. The tool can be used in interactive or non-interactive mode on most operating systems through the Command Prompt or Terminal.[\[3\]](#)

Telnet:

Telnet is a network protocol and command-line tool used in Windows to test connectivity to remote servers and services, usually on specific ports such as port 23 by default. It allows users to check whether a server is reachable and whether certain ports or services (like HTTP, SMTP, or DNS) are open and responding. In Windows, Telnet must be enabled manually through Windows Features before use. It is commonly used for basic network diagnostics and troubleshooting connectivity and service availability issues.[\[4\]](#)

Internet Control Message Protocol (ICMP)

Internet Control Message Protocol (ICMP) is a network layer protocol widely used by network devices such as routers, gateways and hosts to send error messages and operational information. Since the Internet Protocol (IP) itself does not have an inbuilt error-reporting or correction mechanism, ICMP is a supporting protocol within the IP suite that helps in reporting errors and sending diagnostic messages.[\[5\]](#)

RTT(Round Trip Time)

RTT (Round Trip Time) is the time, measured in milliseconds, for a data request to travel from a source to a destination and back. It is commonly measured using the ping command and is an important indicator of network performance and latency.[\[6\]](#)

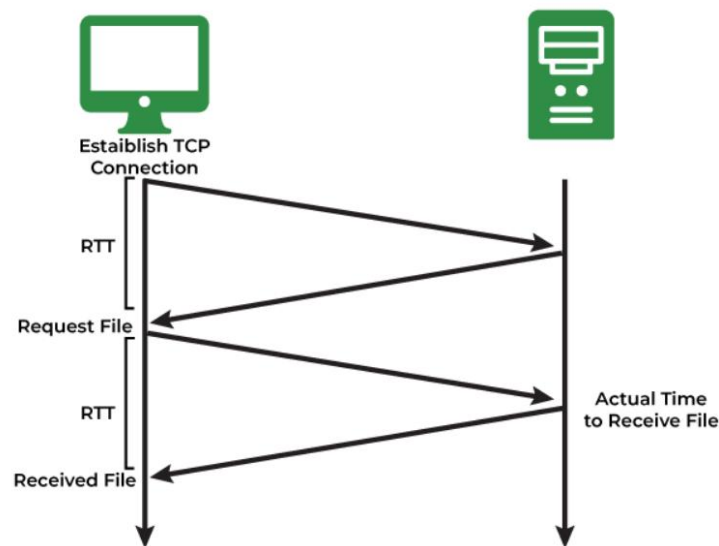


Figure 1:RTT(Round Trip Time) Measurement[\[6\]](#)

Echo-Request and Reply Message

Network managers use it to check the operations of the IP and the host's reachability. The host/router sends the echo request message, while the echo reply message is sent by the host/router that receives an echo request message.

For example, host A wants to check whether it can communicate with another host B so it will send an echo request to B and if there is a link between A and B and B is active then it will send an echo reply to A on receiving the request.[\[7\]](#)

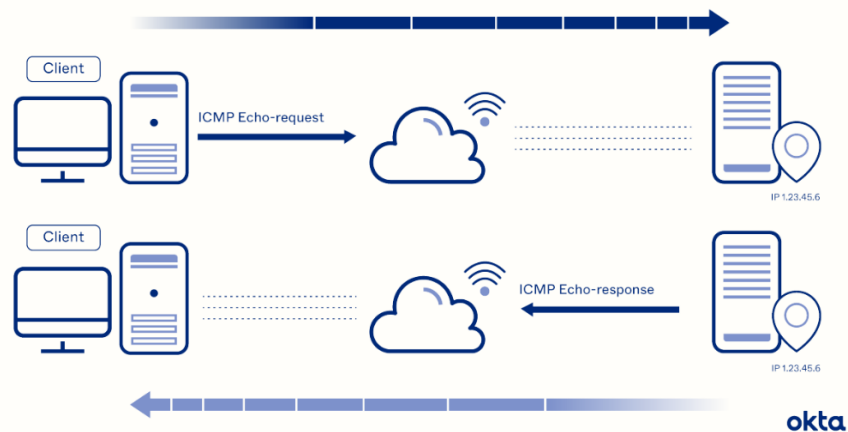


Figure 2:ICMP Request-Response[8]

DNS System and Name Resolution

The Domain Name System (DNS) is a fundamental network service that translates human-readable domain names into IP addresses, allowing users to access websites and network services easily. DNS works through a query and response process, where a client sends a DNS query and receives a response containing the requested IP address or DNS record. DNS queries can be recursive, where the DNS server is responsible for finding the final answer on behalf of the client, or iterative, where the client receives referrals to other DNS servers and continues the query itself. The DNS system is organized in a hierarchical structure consisting of root servers, top-level domain (TLD) servers, and authoritative DNS servers, ensuring efficient and scalable name resolution across the internet.

Wireshark

Wireshark is a free, open-source network protocol analyzer used to capture and analyze network traffic in real time at the packet level. It allows users to capture traffic from multiple interfaces and provides detailed visibility into protocols and packet structures. Wireshark supports hundreds of protocols and offers powerful capture and display filters to precisely inspect specific traffic. Its interface includes packet lists, detailed protocol breakdowns, and raw data views for in-depth analysis. The tool also provides advanced statistics, graphs, and analysis features that help troubleshoot network issues, analyze performance, and support security investigations.[9]

Transport Layer Protocols and Socket Programming

Socket Programming

A socket is an endpoint that enables two-way communication between programs over a network, providing inter-process communication (IPC) using an IP address and port number. In client-server applications, the server creates and binds a socket to a port and waits for connections, while the client creates a socket and connects to the server. Once connected, data can be transmitted between both ends. There are two main types of sockets: datagram sockets, which are connectionless and send packets independently, and stream sockets, which are connection-oriented and provide reliable, ordered data transfer similar to a phone call.[\[10\]](#)

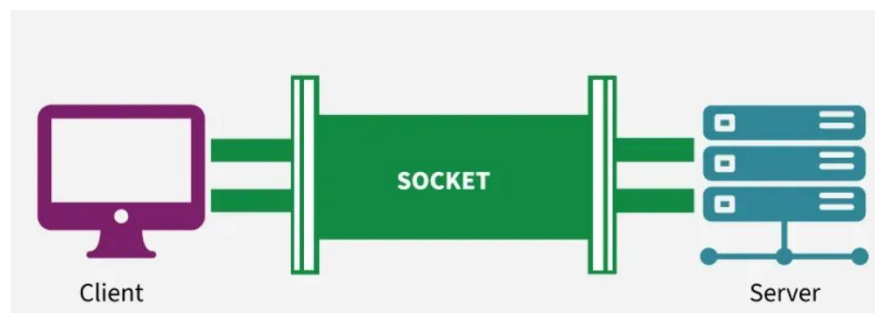


Figure 3:Socket programming [\[10\]](#)

client-server model

The client-server model is a common communication model in networked systems where the client initiates communication by requesting services from a server. The server is a continuously running program that listens for requests and can serve multiple clients at the same time. Clients must know the server's address in advance, while the server does not need prior knowledge of the clients. Communication between clients and servers is handled through layered network protocols, most commonly the TCP/IP protocol suite.[\[11\]](#)

TCP vs UDP Protocols

Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) are two core transport layer protocols in computer networking.

TCP and UDP are transport layer protocols with different characteristics and use cases. TCP is connection-oriented, reliable, and ensures ordered data delivery using acknowledgements, error checking, and retransmission, but it is slower and has higher overhead. UDP is connectionless, faster, and lightweight, with no guarantees for delivery, order, or retransmission. Therefore, TCP

is used for applications requiring reliability like HTTP, FTP, and email, while UDP is preferred for speed-sensitive applications such as DNS, VoIP, and streaming.[\[12\]](#)

HTTP Protocol and Web Server Design

HTTP (Hypertext Transfer Protocol) is a fundamental protocol of the Internet, enabling the transfer of data between a client and a server. It is the foundation of data communication for the World Wide Web. HTTP provides a standard between a web browser and a web server to establish communication. It is a set of rules for transferring data from one computer to another.

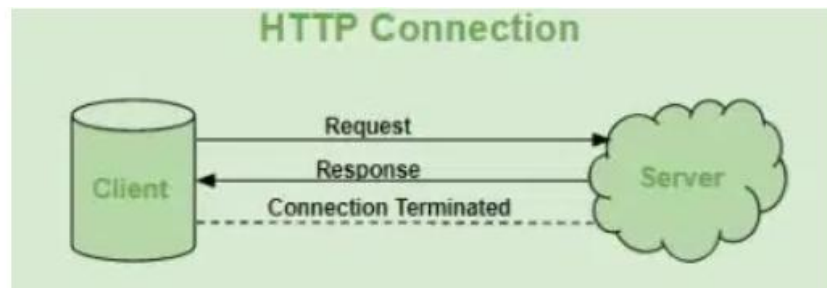


Figure 4:HTTP connection [\[13\]](#)

➤ Features:

- **Stateless:** Each request is independent and the server doesn't retain previous interactions' information.
- **Text-Based:** Messages are in plain text, making them readable and debuggable.
- **Client-Server Model:** Follows a client-server architecture for requesting and serving resources.
- **Request-Response:** Operates on a request-response cycle between clients and servers.
- **Request Methods:** Supports various methods like GET, POST, PUT, DELETE for different actions on resources.

HTTP Request/Response:

HTTP is a request–response protocol where a client sends an HTTP request to a server, including a method, URL, headers, and sometimes a request body. The server processes the request and performs the required operations, such as retrieving data or executing scripts. It then returns an HTTP response containing a status code, headers, and optionally the requested content. Finally, the client receives the response and displays or processes the data based on its type.[13]

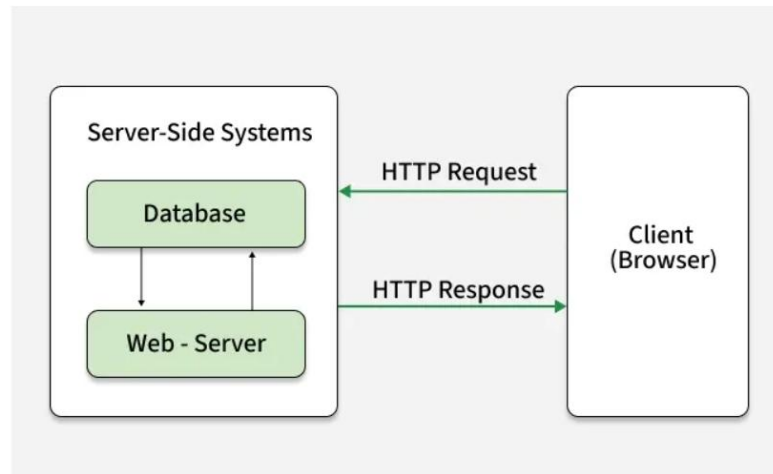


Figure 5:HTTP request/response[13]

HTTP methods

HTTP defines several request methods that specify the action to be performed on a resource. GET is used to retrieve data, while POST sends data to the server for processing or creating resources. PUT and PATCH are used to update resources, with PUT replacing the entire resource and PATCH modifying only specific parts. Other methods like DELETE, HEAD, OPTIONS, TRACE, and CONNECT support resource removal, metadata retrieval, capability discovery, debugging, and secure tunneling.[13]

HTTP Status Codes

200 OK

200 OK is a successful HTTP response status code indicating that the client’s request was processed correctly by the server. For **GET** requests, the requested resource is returned in the response body, while **HEAD** responses include only headers without a body. In **POST** or **PUT** requests, the response body describes the result of the performed action. For **TRACE**, the server returns the received request message in the response body.[14]

307 Temporary Redirect

The server sends this response to direct the client to get the requested resource at another URI with the same method that was used in the prior request. This has the same semantics as the 302 Found response code, with the exception that the user agent must not change the HTTP method used: if a [POST](#) was used in the first request, a POST must be used in the redirected request.[\[14\]](#)

404 Not Found

The server cannot find the requested resource. In the browser, this means the URL is not recognized. In an API, this can also mean that the endpoint is valid but the resource itself does not exist. Servers may also send this response instead of 403 Forbidden to hide the existence of a resource from an unauthorized client. This response code is probably the most well known due to its frequent occurrence on the web.[\[14\]](#)

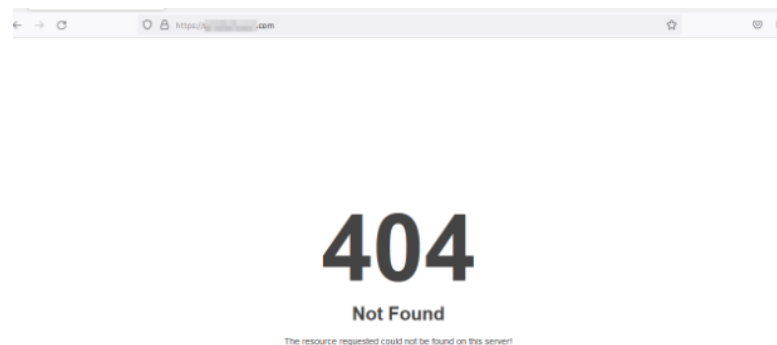


Figure 6:404 not found

Serving Static Content

The Static Content Hosting Pattern is a system design approach focused on delivering unchanging files such as HTML, CSS, JavaScript, images, and media directly to users. It relies on origin servers, caching mechanisms, and Content Delivery Networks (CDNs) to serve content quickly and reduce latency by delivering files from locations closer to users. This pattern improves performance, scalability, reliability, and cost efficiency while simplifying maintenance. It is widely used for websites, blogs, media libraries, documentation, and other applications where content does not change dynamically.[\[15\]](#)

HTTP Cookies and Session Management

Cookies are small pieces of data stored in the user's browser by a website and sent with each HTTP request to help identify the client and store limited information. Cookies differ from sessions in that cookies are stored on the client side, while sessions store user data on the server and use a session ID (usually saved in a cookie) to identify the user. Session IDs are unique identifiers generated by the server and sent to the client, allowing the server to match each request with the correct user session. Sessions are essential for authentication because HTTP is stateless, so sessions enable the server to remember logged-in users, maintain login state, and securely manage user interactions across multiple requests.

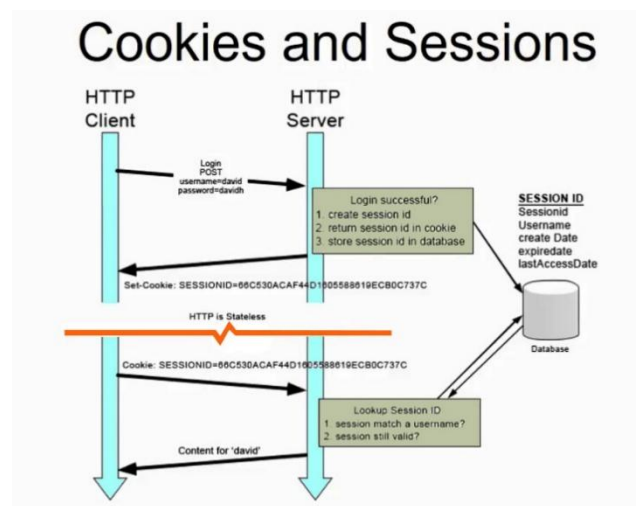


Figure 7:HTTP Cookies and Session[16]

Procedure

Part one:

Question 1: (Network Tools Definitions):

Ping:

- **Definition:** A network diagnostic tool that tests connectivity between two devices by sending ICMP echo request packets and measuring the round-trip time and packet loss.

Tracert:

- **Definition:** A network utility that displays the route and measures transit delays of packets across an IP network by listing each router (hop) between the source and destination.

Nslookup:

- **Definition:** A command-line tool used to query Domain Name System (DNS) servers to obtain domain name or IP address mapping information and troubleshoot DNS-related issues.

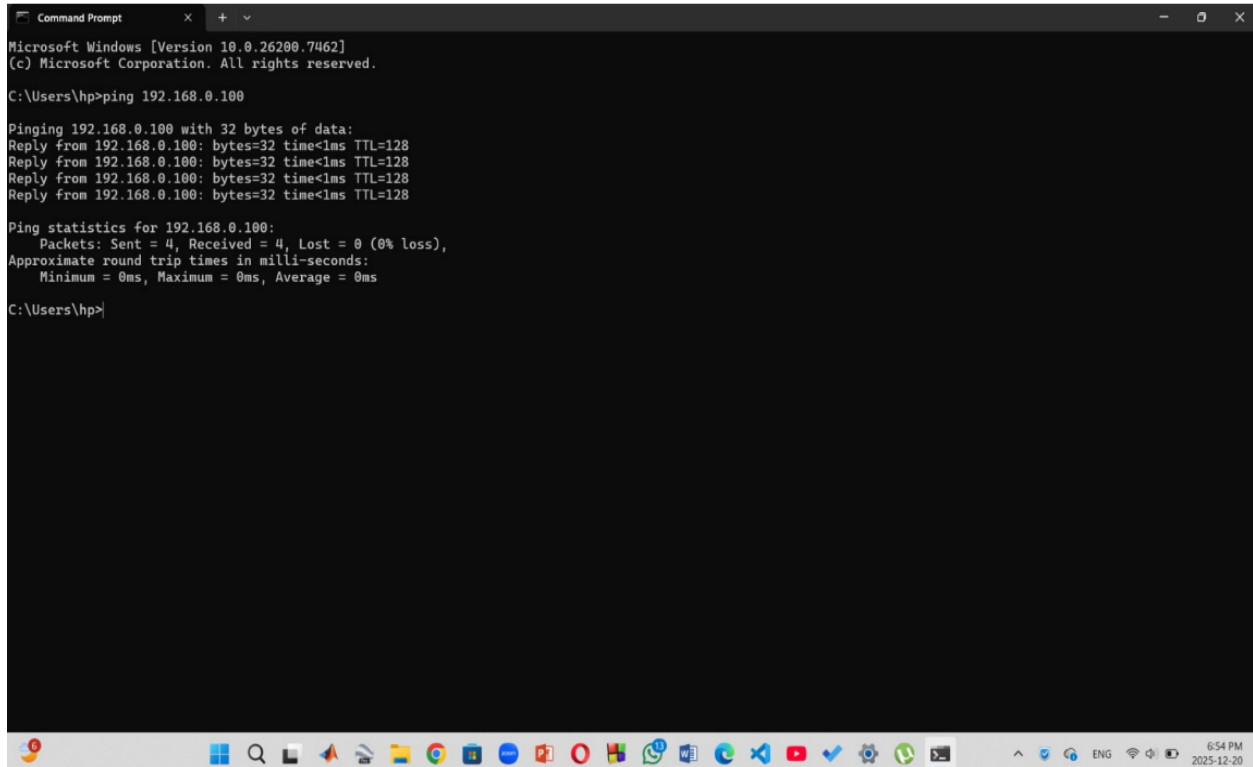
Telnet:

- **Definition:** A network protocol and command-line tool that provides bidirectional interactive text-oriented communication to establish remote terminal connections over TCP/IP networks

Running commands & brief Explanation of the outputs:

1-Ping a Local Device (smartphone) :

- **Commnad used :**
ping 192.168.0.100
- **Screenshot of the output:**



```
Microsoft Windows [Version 10.0.26200.7062]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hp>ping 192.168.0.100

Pinging 192.168.0.100 with 32 bytes of data:
Reply from 192.168.0.100: bytes=32 time<1ms TTL=128
Reply from 192.168.0.100: bytes=32 time<1ms TTL=128
Reply from 192.168.0.100: bytes=32 time<1ms TTL=128
Reply from 192.168.0.100: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.0.100:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Users\hp>
```

Figure 8:ping command output to local smartphone device

- **Output Explanation**

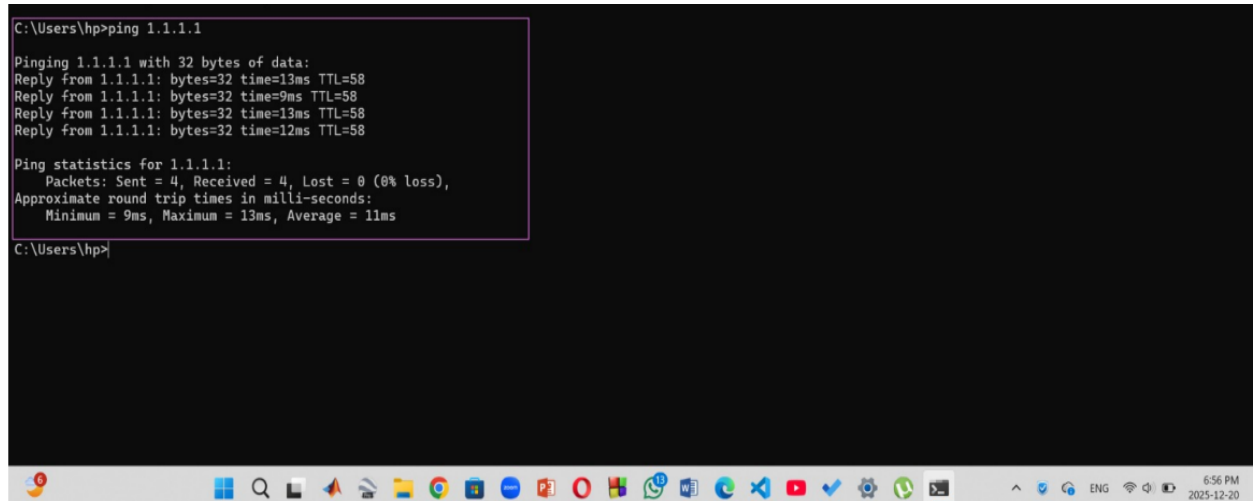
The ping command was executed to test connectivity between the laptop and another device on the same local network with IP address 192.168.0.100. The results show that all four ICMP echo requests were successfully sent and received with 0% packet loss, confirming reliable communication between the two devices. The response time was less than 1 ms, which is expected for devices connected within the same local network. The TTL value of 128 indicates that the destination device is directly reachable without passing through external routers.

- **Result Interpretation**

These results indicate excellent local network performance. The very low latency and zero packet loss confirm that both devices are connected to the same network segment and that the local network infrastructure is functioning correctly.

2-Ping 1.1.1.1 :

- **Command used :**
ping 1.1.1.1
- **Screenshot of the output:**



```
C:\Users\hp>ping 1.1.1.1

Pinging 1.1.1.1 with 32 bytes of data:
Reply from 1.1.1.1: bytes=32 time=13ms TTL=58
Reply from 1.1.1.1: bytes=32 time=9ms TTL=58
Reply from 1.1.1.1: bytes=32 time=13ms TTL=58
Reply from 1.1.1.1: bytes=32 time=12ms TTL=58

Ping statistics for 1.1.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 9ms, Maximum = 13ms, Average = 11ms

C:\Users\hp>
```

Figure 9:ping command output to DNS server

- **Output Explanation**

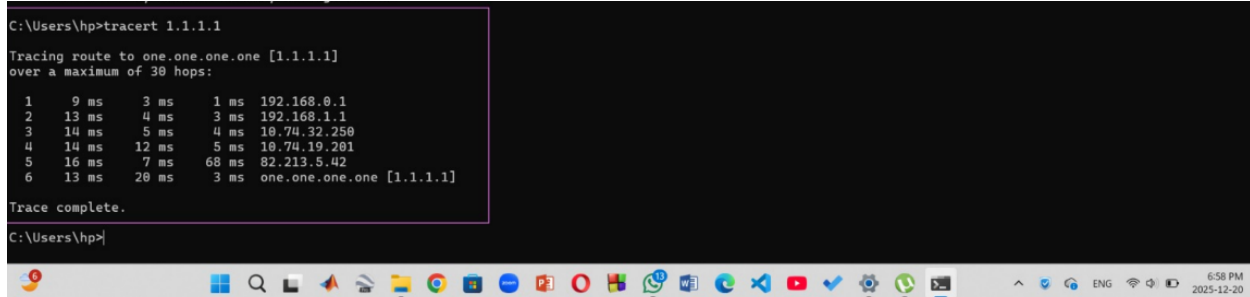
The ping command was used to test connectivity between the laptop and the public DNS server at IP address 1.1.1.1. All four packets were successfully sent and received with 0% packet loss, indicating a stable internet connection. The round-trip times ranged between 9 ms and 13 ms, with an average of approximately 11 ms. The TTL value of 58 suggests that the packets passed through multiple intermediate routers before reaching the destination.

- **Result Interpretation**

Compared to the local network ping, the response time is higher, which is expected when communicating over the internet. The absence of packet loss confirms reliable connectivity to the Cloudflare DNS server.

3-Tracert 1.1.1.1:

- **Command used:**
Tracert 1.1.1.1
- **Screenshot of the output :**



```
C:\Users\hp>tracert 1.1.1.1

Tracing route to one.one.one.one [1.1.1.1]
over a maximum of 30 hops:

  0  9 ms    3 ms    1 ms  192.168.0.1
  1  13 ms    4 ms    3 ms  192.168.1.1
  2  14 ms    5 ms    4 ms  10.74.32.250
  3  14 ms    12 ms   5 ms  10.74.19.201
  4  16 ms    7 ms    68 ms 82.213.5.42
  5  13 ms    20 ms   3 ms  one.one.one.one [1.1.1.1]

Trace complete.

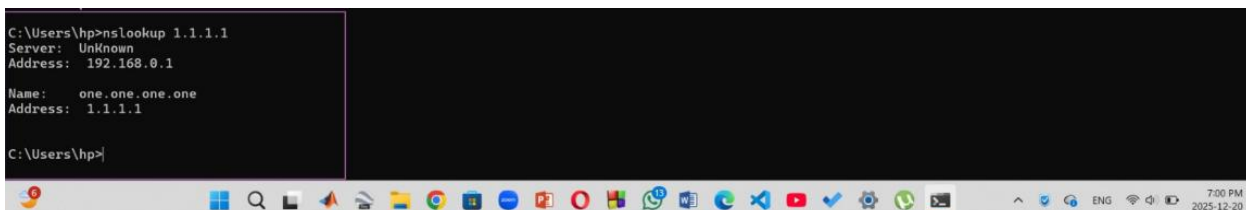
C:\Users\hp>
```

Figure 10:Traceroute output showing path to 1.1.1.1

- **Output Explanation**
The traceroute command displays the path taken by packets from the local device to the destination IP address 1.1.1.1. The output shows several intermediate routers (hops), starting from the local router and ending at the destination. Each hop reports the round-trip time, which reflects the latency at that point in the network.
- **Result Interpretation**
The results show that the destination was reached in a small number of hops, indicating efficient routing. The gradual increase in delay across hops is normal as packets move farther away from the local network.

4-Nslookup 1.1.1.1 :

- **Command used:**
nslookup 1.1.1.1
- **Screenshot of the output :**



```
C:\Users\hp>nslookup 1.1.1.1
Server: UnKnown
Address: 192.168.0.1

Name:   one.one.one.one
Address: 1.1.1.1

C:\Users\hp>
```

Figure 11:Nslookup reverse DNS query for 1.1.1.1

- **Output Explanation**

The nslookup command was used to perform a reverse DNS lookup for the IP address 1.1.1.1. The output shows that this IP address resolves to the domain name *one.one.one.one*. The DNS query was handled by the local DNS server at IP address 192.168.0.1.

- **Result Interpretation**

This result confirms that DNS resolution is functioning correctly and that the Cloudflare DNS service is reachable from the network.

Capture DNS Messages with Wireshark :

Purpose: we used wireshark to capture and analyze DNS messages to understand how name resolution works.

The DNS request originates from the source IP 192.168.0.104 to the destination IP 192.168.0.1 over UDP port 53.

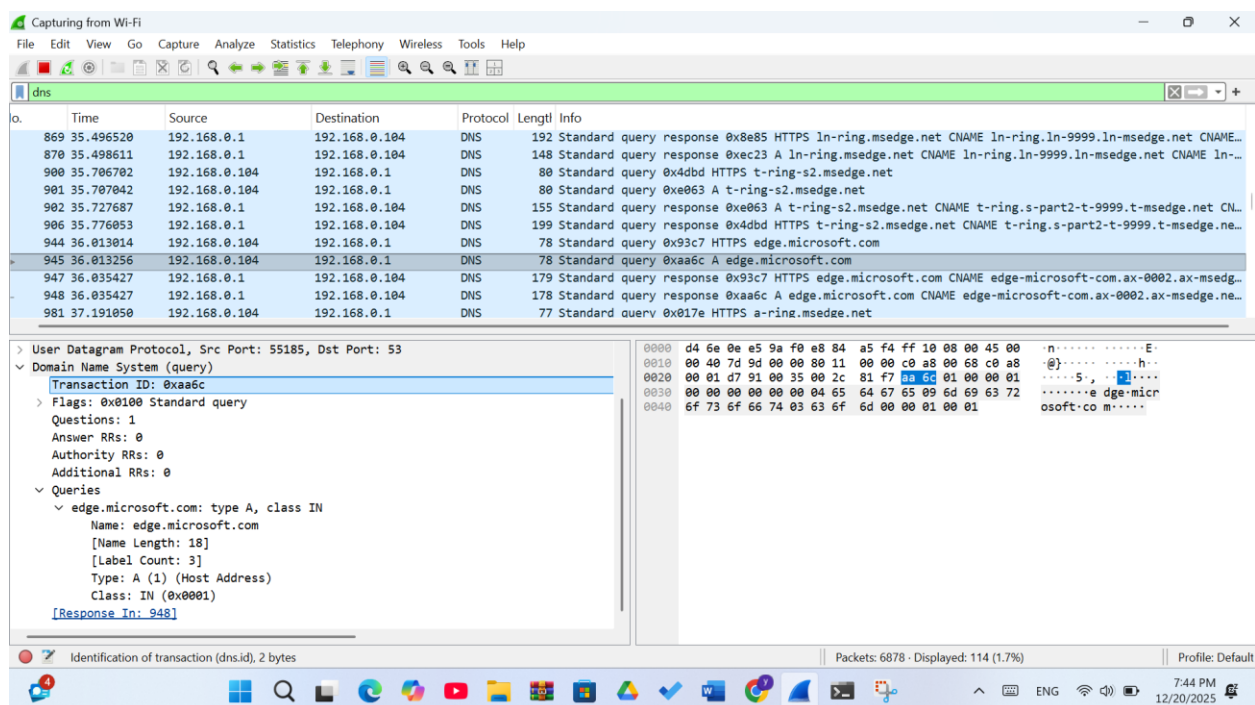


Figure 12: Captured DNS query and response messages using Wireshark

The DNS response is sent from the DNS server 192.168.0.1 back to the client 192.168.1.104 .

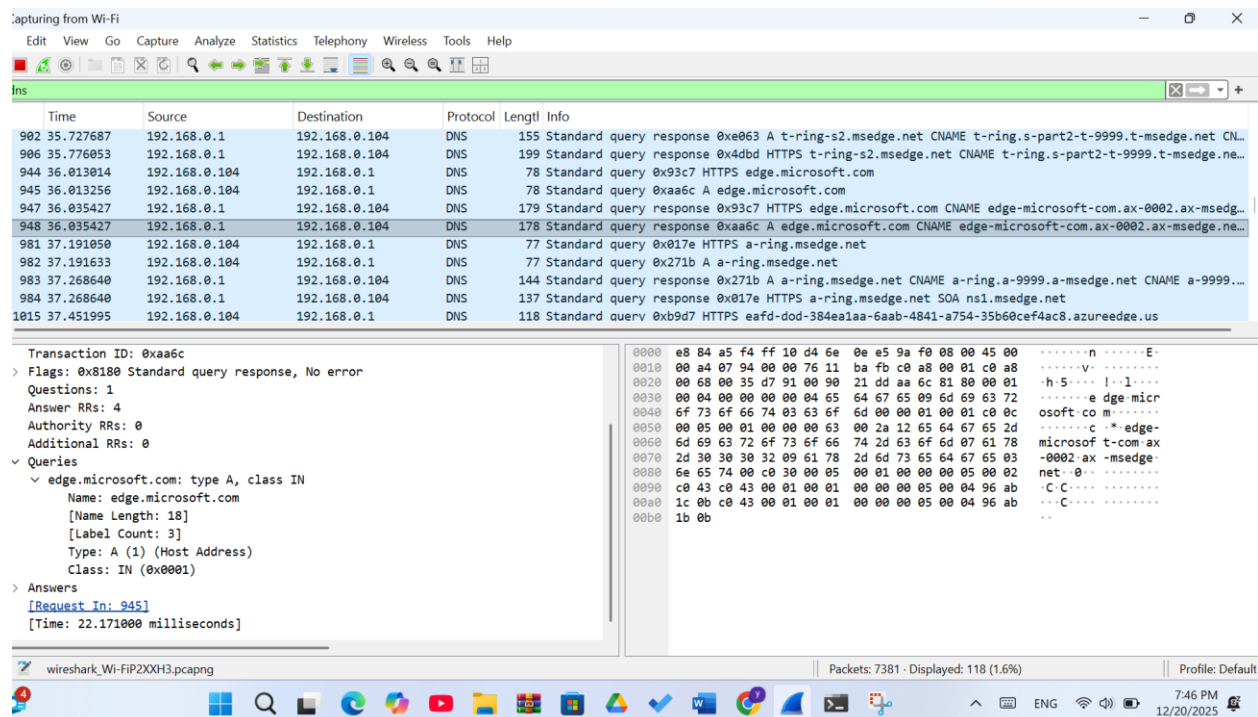


Figure 13: Captured DNS query and response messages using Wireshark

Part two:

Purpose of the Server

The server is responsible for:

- Receiving **control messages** (START and END) from the client using **TCP**.
- Receiving a large sequence of numbers (from 0 to 1,000,000) using **UDP**.
- Counting the total number of received UDP packets.
- Detecting how many packets arrived **out of order**.

TCP is used for control messages because it is reliable, while UDP is used for data transmission because it is faster and has lower overhead.

Server Code:

Importing Libraries and Defining Ports

```
server.py > run_server
1  import socket
2
3  TCP_PORT = 8090
4  UDP_PORT = 8090
5  BUFFER_SIZE = 1024 # maximum size of a UDP packet in bytes
6
```

In this part, the socket library is imported because it provides the basic networking functionality required to implement TCP and UDP communication. Both TCP and UDP are configured to use the same port number, 8090, which simplifies the coordination between control messages and data transmission. The buffer size is defined to specify the maximum amount of data that can be received in a single UDP packet, which is sufficient since each packet contains only one number.

Creating the Main Server Function

```
7
8  def run_server():
```

This function represents the main execution point of the server. All server operations, including socket creation, client handling, data reception, and result processing, are implemented inside this function to keep the code organized and easy to manage.

TCP Socket Setup for Control Messages

```
9      # Create TCP socket for START and END messages
10     tcp_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
11     tcp_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
12     tcp_sock.bind(('', TCP_PORT))
13     tcp_sock.listen(1)
14     print("Server is listening on TCP port", TCP_PORT)
```

Here, a TCP socket is created to handle control messages from the client. TCP is used because it guarantees reliable and ordered communication, which is important for messages such as START and END. The socket is bound to port 8090 and set to listen for incoming connections. The reuse address option is enabled to allow the server to restart without waiting for the port to be released by the operating system.

UDP Socket Setup for Receiving Data

```
15
16     # Create UDP socket for receiving numbers
17     udp_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
18     udp_sock.bind(('', UDP_PORT))
19     print("Server is listening on UDP port", UDP_PORT)
```

In this section, a UDP socket is created to receive the sequence of numbers sent by the client. UDP is chosen for data transmission because it has low overhead and allows faster communication compared to TCP. The socket is bound to the same port as the TCP socket, making it easier for the client to send both TCP and UDP traffic to the server.

Accepting a TCP Client Connection

```
21     while True:
22         print("\nWaiting for a TCP client...")
23         conn, addr = tcp_sock.accept()
24         print("New TCP connection from:", addr)
```

The server enters a loop where it waits for a client to establish a TCP connection. When a connection request arrives, the server accepts it and stores the connection object and the client's address. This allows the server to communicate directly with the connected client through TCP.

Receiving and Validating the START Message

```
26     # Receive START message over TCP
27     start_msg = conn.recv(1024).decode('utf-8').strip()
28     print("Received over TCP:", start_msg)
29
30     if start_msg != "START":
31         # If the first message is not START, close this client
32         print("First message was not 'START'. Closing connection.")
33         conn.close()
34         continue
```

After establishing the TCP connection, the server receives the first message from the client. This

message must be START to indicate the beginning of the data transmission. The received data is decoded from bytes into a string and cleaned from extra spaces. If the message is not START, the server closes the connection and waits for another client, ensuring that the protocol is followed correctly.

Preparing to Receive UDP Data

```
38         # Set timeout for UDP socket (to stop when client finishes)
39         udp_sock.settimeout(2.0)
40
41         count = 0          # number of UDP packets received
42         out_of_order = 0   # how many times the order was wrong
43         last_number = -1   # last number received
44     ..
```

Before receiving UDP packets, a timeout is set on the UDP socket. This timeout allows the server to detect when the client has finished sending data. Several variables are initialized to track the total number of received packets, the number of out-of-order packets, and the last received number, which are needed to analyze the behavior of UDP transmission.

Receiving UDP Packets and Processing Data

```
45         while True:
46             try:
47                 # Receive one UDP packet
48                 data, client_addr = udp_sock.recvfrom(BUFFER_SIZE)
49                 text = data.decode('utf-8').strip()
50
51             if text == "":
52                 # Ignore empty packets
53                 continue
```

In this loop, the server continuously listens for incoming UDP packets. Each received packet is decoded into text and prepared for processing. This loop continues as long as packets are received within the timeout period.

Sequence Checking and Statistics Update

```

55         # Convert text to integer
56         try:
57             n = int(text)
58         except ValueError:
59             # Ignore invalid data
60             print("Received non-integer UDP data:", text)
61             continue
62
63         # Increase packet counter
64         count += 1
65
66         # Check if the sequence is broken
67         if last_number != -1 and n != last_number + 1:
68             out_of_order += 1
69
70         last_number = n

```

Once a valid number is received, the server increments the packet counter and checks whether the number follows the expected sequence. If the received number does not equal the previous number plus one, it is considered an out-of-order packet. This information is recorded to evaluate the reliability of UDP transmission.

Stopping UDP Reception on Timeout

```

72         except socket.timeout:
73             # No UDP packets for a while, stop receiving
74             print("No UDP packets for some time. Stopping UDP receiving.")
75             break
76

```

If no UDP packets are received within the specified timeout period, a timeout exception occurs. This indicates that the client has finished sending data, so the server stops receiving UDP packets and moves to the next phase.

Receiving the END Message

```

77         # Receive END message over TCP
78         end_msg = conn.recv(1024).decode('utf-8').strip()
79         print("Received over TCP:", end_msg)
80

```

After the UDP transmission is completed, the server waits for the END message from the client via TCP. This message confirms that the client has fully completed its transmission process.

Printing Results and Closing the Connection

```
81         # Print results
82         print("\n=== RESULTS ===")
83         print("Total UDP packets received:", count)
84         print("Last number received:", last_number)
85         print("Out-of-order events:", out_of_order)
86         print("=====\n")
87
88         conn.close()
89         print("TCP connection closed.")
```

In the final part, the server prints the collected statistics, including the total number of received UDP packets, the last received number, and the number of out-of-order events. These results help analyze packet loss and reordering in UDP communication. After printing the results, the TCP connection with the client is closed, and the server becomes ready to accept a new connection.

Client Side Implementation:

Purpose of the Client

The client is responsible for:

- Establishing a TCP connection with the server to control the communication flow.
- Sending the START message to notify the server that data transmission is about to begin.
- Sending a large sequence of numbers (from 0 to 1,000,000) to the server using UDP, where each number is sent in a separate packet.
- Sending the END message to indicate that all data has been transmitted.

TCP is used by the client to send the START and END messages because it guarantees reliable and ordered delivery, ensuring that the server correctly detects the beginning and completion of the transmission. UDP is used to send the sequence of numbers because it provides faster transmission with lower overhead, which is suitable for sending a large amount of data, even though it does not guarantee packet delivery or order.

Client Code:

Importing Libraries and Setting Configuration

```
client.py > ...
1  import socket
2  import time
3
4  SERVER_IP = "127.0.0.1"
5  TCP_PORT = 8090
6  UDP_PORT = 8090
7  MAX_NUMBER = 1000000
```

In this section, the client imports the socket library because it will create both TCP and UDP sockets to communicate with the server. The server IP is set to 127.0.0.1, which means the client is intended to run on the same machine as the server (localhost). The ports are set to 8090 for both TCP and UDP to match the server configuration. The variable MAX_NUMBER defines the last value in the sequence, meaning the client will send numbers starting from 0 up to 1,000,000, which matches the project requirement.

Creating the Main Client Function

```
9
10 ∨ def run_client():
```

This function is the main entry point of the client program. It organizes the client logic into one place, including establishing the TCP connection, sending the control messages, sending UDP data packets, and closing the sockets properly at the end.

TCP Connection Setup (Control Channel)

```
12      # Create TCP socket
13      tcp_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
14      tcp_sock.connect((SERVER_IP, TCP_PORT))
15      print("Connected to server.")
16
```

Here, the client creates a TCP socket and connects it to the server on port 8090. TCP is used as a control channel because it guarantees reliable delivery, so the server will always correctly

receive the start and end signals. After connecting successfully, the client prints a message to confirm that the TCP connection has been established.

Sending the START Message over TCP

```
17 # Send START
18 tcp_sock.sendall(b"START\n")
19 print("Sent START.")
```

In this part, the client sends the message START using TCP to notify the server that the UDP transmission is about to begin. This approach was chosen because UDP does not guarantee delivery, so sending control messages over TCP ensures the server receives them correctly and starts listening for UDP packets at the correct time.

UDP Socket Creation (Data Channel)

```
21 # Create UDP socket
22 udp_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

After sending START, the client creates a UDP socket that will be used to transmit the actual data (the sequence of numbers). UDP is used here because it is lightweight and faster than TCP, which is important when sending a very large number of packets. However, this also means some packets may be lost or arrive out of order, which is exactly what the server is designed to measure.

Sending Numbers 0 → MAX_NUMBER over UDP

```
24 # Send numbers over UDP
25 for i in range(MAX_NUMBER + 1):
26     msg = (str(i) + "\n").encode() # convert number to bytes
27     udp_sock.sendto(msg, (SERVER_IP, UDP_PORT))
28
29     if i % 100 == 0:
30         print("Sent number:", i)
31
32 print("Finished sending all numbers.")
```

In this loop, the client sends the sequence of numbers starting from 0 up to MAX_NUMBER. The loop uses MAX_NUMBER + 1 so that the last value is included in the transmission. Each number is

converted into a string and then encoded into bytes before being sent in a single UDP packet using `sendto`. Printing progress every 100,000 numbers helps monitor execution without printing every packet, which would slow down the program significantly.

Closing UDP and Sending END over TCP

```
--
34     # Send END
35     tcp_sock.sendall(b"END\n")
36     print("Sent END.")
37
```

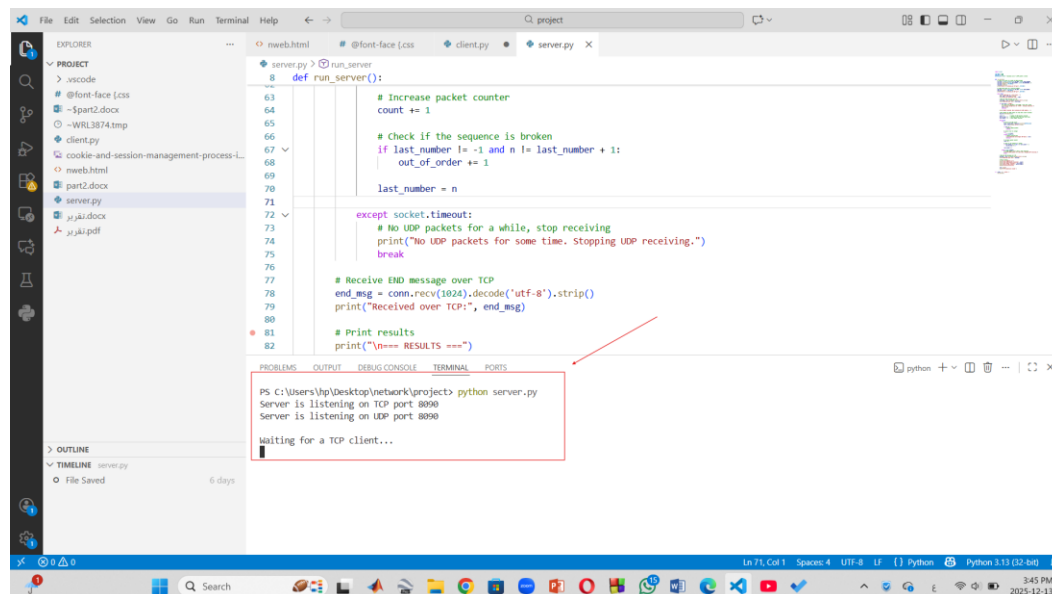
Once the client finishes sending all UDP packets, it closes the UDP socket because it is no longer needed. After that, it sends the END message over TCP to inform the server that the transmission is completed. Using TCP for the END message ensures that the server reliably receives the completion signal, even if the UDP stream experienced packet loss or delays

Closing the TCP Connection

```
38     # Close sockets
39     udp_sock.close()
40     tcp_sock.close()
41     print("Client finished.")
42
43 |
44 # Make sure the client runs when we execute this file
45 run_client()
46
```

Finally, the client closes the TCP socket to properly terminate the connection. This ensures that all resources are released cleanly and the program exits without leaving open sockets.

Testing Part 2:



The screenshot shows a VS Code editor with a project named 'project'. The file explorer on the left shows files like 'web.html', 'font-face.css', 'part2.docx', 'client.py', 'server.py', and 'part2.docx'. The main editor window displays the 'server.py' file. The code defines a 'run_server()' function that listens on port 8090 for both TCP and UDP connections. It includes comments for increasing a packet counter, checking sequence integrity, and handling timeouts. The terminal at the bottom shows the command 'python server.py' being executed, resulting in the output: 'Server is listening on TCP port 8090', 'Server is listening on UDP port 8090', and 'Waiting for a TCP client...'. A red box highlights the terminal output, and a red arrow points from the 'run_server()' function in the code to the terminal.

```
def run_server():
    # Increase packet counter
    count += 1

    # Check if the sequence is broken
    if last_number != -1 and n != last_number + 1:
        out_of_order += 1

    last_number = n

except socket.timeout:
    # No UDP packets for a while, stop receiving
    print("No UDP packets for some time. Stopping UDP receiving.")
    break

# Receive END message over TCP
end_msg = conn.recv(1024).decode("utf-8").strip()
print("Received over TCP:", end_msg)

# Print results
print("\n== RESULTS ==")
```

```
PS C:\Users\hp\Desktop\network\project> python server.py
Server is listening on TCP port 8090
Server is listening on UDP port 8090
Waiting for a TCP client...
```

Figure 14 Server Waiting State

This screenshot shows the server running successfully and waiting for a client connection. After executing the command `python server.py`, the server starts listening on port 8090 for both TCP and UDP connections. The messages “Server is listening on TCP port 8090” and “Server is listening on UDP port 8090” confirm that both sockets were created and bound correctly. The line “Waiting for a TCP client...” indicates that the server is in an idle state, ready to accept a TCP connection from the client.

To obtain this output, the server program was executed first before running the client. This step is necessary because the server must be active and listening before the client attempts to connect. No data transmission has started at this stage; the server is only prepared to receive the START message over TCP.

This result confirms that the server code is working correctly in its initial state. The server successfully initialized both TCP and UDP sockets without errors and entered the listening phase as expected. Seeing this output verifies that the server setup is correct and that it is ready for communication with the client.

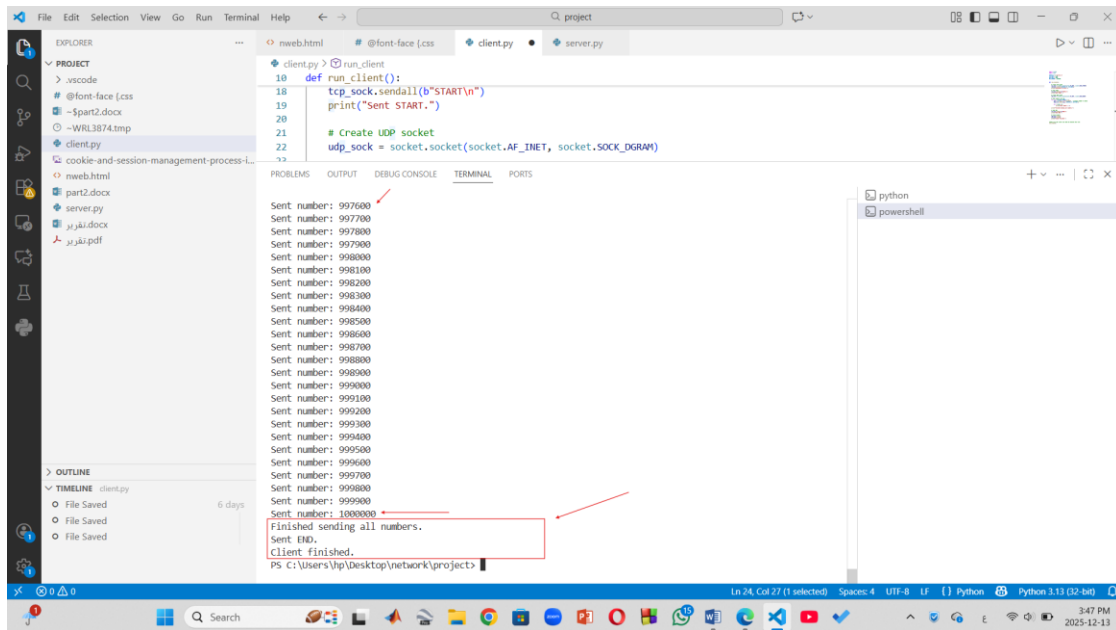


Figure 15 Client Sending Data

This screenshot shows the client program running and actively sending data to the server. After executing the client code, the client successfully establishes a TCP connection with the server and sends the START message. Following that, the client begins transmitting a large sequence of numbers using UDP. The displayed messages such as “Sent number: 997600” up to “Sent number: 1000000” indicate that the client is continuously sending numbers in ascending order, where each number is sent in a separate UDP packet.

To obtain this output, the server was already running and waiting for a connection. The client program was then executed using `python client.py`. Once the TCP connection was established, the client sent the START message to notify the server to begin listening for UDP packets. After that, the client entered a loop that sends numbers from 0 to 1,000,000 using UDP. Progress messages were printed to show that the transmission was ongoing and to confirm that the loop was functioning correctly.

This result confirms that the client is working as expected. The successful sending of numbers up to 1,000,000 shows that the UDP transmission loop completed without errors. The messages “Finished sending all numbers” and “Sent END” confirm that the client properly finished the transmission and notified the server using TCP. This verifies that both control messages and data transmission were correctly implemented on the client side.

```
client.py > run_client
10 def run_client():
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Received over TCP: START
PS C:\Users\hp\Desktop\network\project> python server.py
Server is listening on TCP port 8090
Server is listening on UDP port 8090
Waiting for a TCP client...
PS C:\Users\hp\Desktop\network\project> python server.py
Server is listening on TCP port 8090
Server is listening on UDP port 8090
PS C:\Users\hp\Desktop\network\project> python server.py
Server is listening on TCP port 8090
Server is listening on UDP port 8090
PS C:\Users\hp\Desktop\network\project> python server.py
Server is listening on TCP port 8090
Server is listening on UDP port 8090
Waiting for a TCP client...
New TCP connection from: ('127.0.0.1', 55755)
Received over TCP: START
START received. Now listening for UDP numbers...
No UDP packets for some time. Stopping UDP receiving.
Received over TCP: END
===== RESULTS =====
Total UDP packets received: 1000001
Last number received: 1000000
Out-of-order events: 0
=====
TCP connection closed.
Waiting for a TCP client...
```

Figure 16 Server Results and Verification

This screenshot shows the final output produced by the server after completing the communication with the client. The server first accepts the TCP connection from the client and successfully receives the START message, which indicates the beginning of the data transmission. After that, the server listens for incoming UDP packets and processes the sequence of numbers sent by the client. Once no UDP packets are received for a certain period, the server stops receiving data and waits for the END message over TCP. After receiving the END message, the server prints the final results.

The results displayed include the total number of UDP packets received, the last number received, and the number of out-of-order events. In this test, the server received **1,000,001 UDP packets**, the **last number received was 1,000,000**, and the **number of out-of-order events was 0**.

To obtain this output, the server was started first and left running in listening mode. Then, the client was executed, which connected to the server using TCP, sent the START message, transmitted the numbers from 0 to 1,000,000 using UDP, and finally sent the END message. The server processed all incoming packets and printed the results after the transmission was completed.

The value **“Total UDP packets received: 1,000,001”** represents the total number of UDP packets that the server successfully received from the client. This number is correct because the client sends a sequence of numbers starting from 0 up to 1,000,000 inclusive, which results in exactly 1,000,001 packets. This confirms that no packets were lost during transmission and that the server correctly counted every received UDP packet.

The value **“Last number received: 1,000,000”** indicates the final number processed by the server. This confirms that the entire sequence of numbers was transmitted successfully from the client to the server. If this value were smaller, it would indicate packet loss or early termination of the UDP reception. Since the value matches the expected maximum number, it verifies that the server received the complete data stream.

The value **“Out-of-order events: 0”** shows that all UDP packets arrived at the server in the correct sequence. This means that each received number followed the expected order, where every number was exactly one greater than the previous one. This result is expected in this test because both the client and server were running on the same machine using the localhost interface, which minimizes network delays and packet reordering. In a real network environment, this value could be greater than zero due to the unreliable nature of UDP.

Part three:

Objective of this part:

The objective of Part Three was to build a simple but complete web server using Python, HTML, and CSS. The server uses socket programming to handle HTTP requests and respond correctly to the browser. Our goal was to make the server able to send different types of files (HTML, CSS, images), perform redirects, and support both English and Arabic pages. We also aimed to implement user registration and login using hashed passwords and session cookies. Overall, the purpose was to create a basic, reusable, and flexible web server that can respond to different user inputs.

Web Server (System Architecture):

The web server in this project is implemented using a simple client–server architecture, where the web browser functions as the client and the Python program (server.py) operates as the server. The server listens on port 8099 and waits for incoming HTTP requests. When the user enters a URL such as `http://localhost:8099/en` or `http://localhost:8099/ar`, the browser sends an HTTP request to the server, which then reads the request, identifies the requested path, and determines the appropriate resource to return. For example, the paths `/`, `/index.html`, `/main_en.html`, and `/en` map to the English homepage `main_en.html`, while the `/ar` path returns the Arabic version `main_ar.html`. In addition to serving these HTML pages, the server also delivers static resources—including CSS files and .png or .jpg images—so that the webpage layout and styling are displayed correctly in the browser.

For user-related operations, such as registration and login, the server handles the paths `/register` and `/login` differently. It extracts form data from the request, hashes passwords using SHA256, and stores or validates user information in the `data.txt` file. Once the requested action is identified, the server constructs an appropriate HTTP response consisting of a status line (such as `HTTP/1.1 200 OK` or `HTTP/1.1 404 Not Found`), the required headers (including the correct Content-Type), and a response body. After sending the complete response back to the browser, the server closes the connection and waits for the next incoming request. This cycle repeats for every URL the user accesses, forming a complete and continuous request–response workflow within `server.py`.

Web Page Design Using HTML and CSS

This section describes the user interface of the web application, including the English and Arabic homepage designs, the login and registration forms, and the protected page displayed after successful authentication. All pages are styled using external CSS files to ensure a clean, organized, and visually appealing layout.

1. `main_en.html`

Description:

The English homepage serves as the main entry point to the web server. It welcomes the user with a title banner that includes the sentence: “Welcome to our course Computer Networks, This is a tiny webserver,” with part of the phrase highlighted in blue using CSS. Below the introduction, the page summarizes five caching terms from RFC 2616—public, private, no-cache, no-store, and max-age—each inside a styled box to present the concepts clearly.

The page then displays detailed sections for the group members, including names, IDs, skills, academic background, and an image for each member (one .jpg and one .png as required). The layout uses separate content boxes, shadows, rounded borders, and smooth color gradients for visual structure. The page also includes links to register.html, login.html, and an external Python tutorial, fulfilling the project requirements.

HTML CODE

```
<main_en.html> <html> <body> <div.container> <div.box> <div.text-box> <p>
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>ENC53320-My Tiny Webserver 25/26</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10
11 <!-- Intro Section -->
12 <div class="intro">
13   <h1>Welcome to our course</h1>
14   <h1 class="Blue">Computer Networks,This is a tiny webserver</h1>
15 </div>
16
17 <!-- Summary Box -->
18 <div class="container">
19   <section class="custom-summary">
20     <h2 class="custom-title">Cache Terms from RFC 2616</h2>
21     <p class="custom-subtitle">
22       These terms describe how caching works in HTTP to improve performance and reduce server load.
23     </p>
24     <div class="custom-boxes">
25
26       <!-- Term 1 -->
27       <div class="custom-box">
28         <h3>1</h3>
29         <h4>public</h4>
30         <p>
31           The <b>public</b> directive explicitly allows any type of cache-whether a browser or a shared proxy-to store the response.
32           This is useful for static or openly accessible content where sharing cached copies improves load time and reduces server requests.
33           <br>Example: <code>Cache-control: public, max-age=3600</code> enables caching for one hour.
34         </p>
35       </div>
36
```



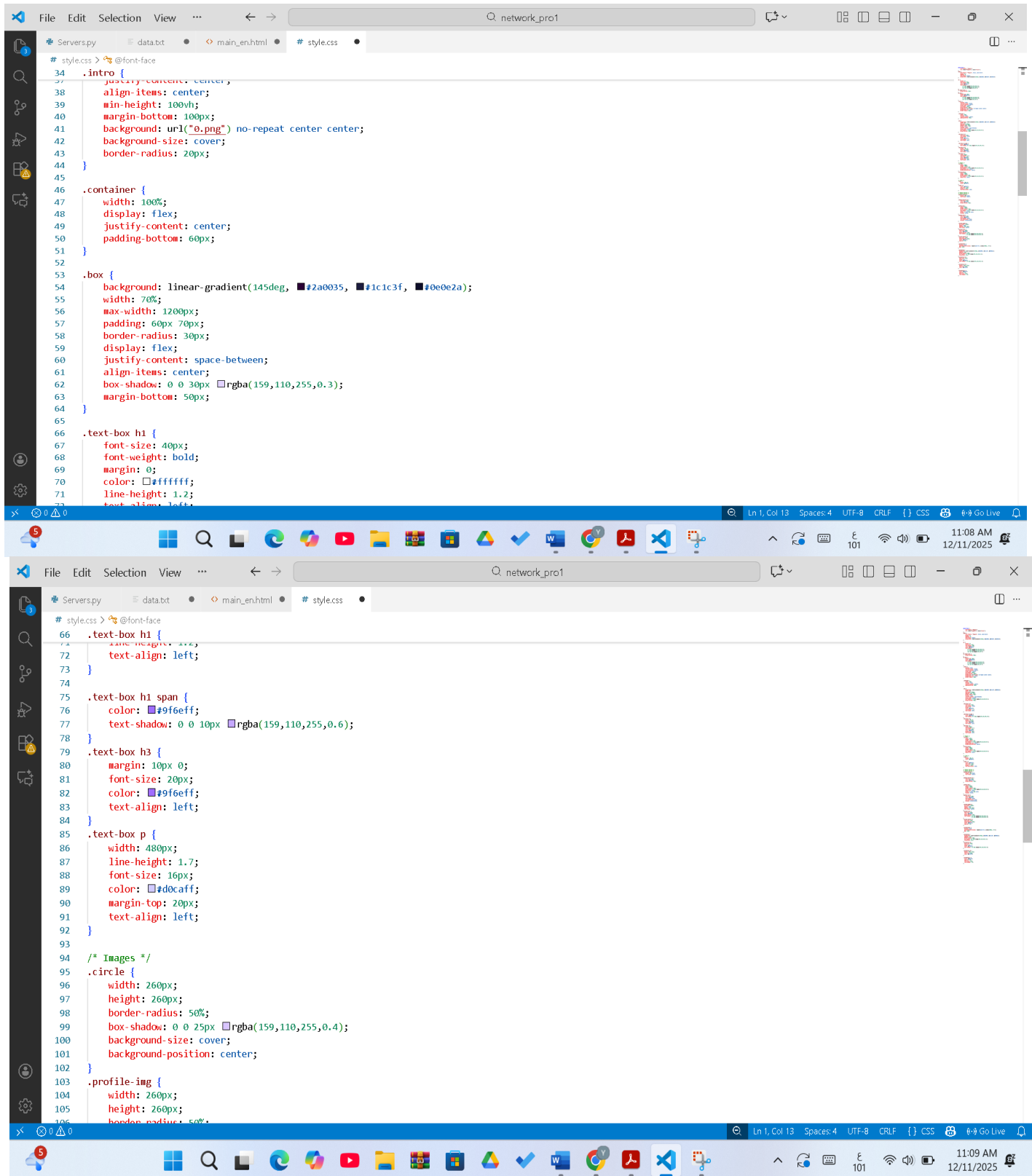
```
File Edit Selection View ... network_pro1
Servers.py data.txt main_en.html
main_en.html > html > body > div.container > section.custom-summary > div.custom-boxes > div.custom-box
2 <html lang="en">
9 <body>
18 <div class="container">
19 <section class="custom-summary">
24 <div class="custom-boxes">
69 </div>
70 <!-- Term 5 -->
71 <div class="custom-box">
72 <h3>5</h3>
73 <h4>max-age</h4>
74 <p>
75 The <b>max-age</b> directive defines the maximum time period (in seconds) during which a cached response is considered fresh.
76 Within this time, the cache can serve the stored response without contacting the server. After it expires, the cache must revalidate or fetch a new version.
77 <br>Example: <code>Cache-Control: max-age=300</code> allows reuse of the cached content for five minutes.
78 </p>
79 </div>
80 </div>
81 </section>
82 </div>
83 <h1 class="team-title">Team Members</h1>
84 <!-- Yasmin -->
85 <div class="container">
86 <div class="box">
87 <div class="text-box">
88 <h1>I'm <span>Yasmin Badwan</span></h1>
89 <h3>Computer Engineering Student</h3>
90 <h3>ID:1230271</h3>
91 <p>
92 Throughout my academic journey, I have gained experience in developing web applications using HTML, CSS, and JavaScript, as well as working with De
93 </p>
94 </div>
95 
96 </div>
97 </div>
98 </div>
99
```

```
File Edit Selection View ... network_pro1
Servers.py data.txt main_en.html
main_en.html > html > body > div.container > div.box > div.text-box > p
2 <html lang="en">
9 <body>
86 <div class="container">
87 </div>
88 </div>
89 <!-- Hanaa -->
90 <div class="container">
91 <div class="box">
92 <div class="text-box">
93 <h1>I'm <span>Hanaa</span></h1>
94 <h3>Computer Science Student</h3>
95 <h3>ID:1230263</h3>
96 <p>
97 I have worked on projects involving web applications, Java OOP, and Python data analysis, and I am skilled in Java, Python, C++, SQL, and Git. I a
98 </p>
99 </div>
100 
101 </div>
102 </div>
103 <!-- Sewar -->
104 <div class="container">
105 <div class="box">
106 <div class="text-box">
107 <h1>I'm <span>Sewar Masric</span></h1>
108 <h3>Computer Engineering Student</h3>
109 <h3>ID:1231240</h3>
110 <p>
111 My coursework and projects have focused on operating systems labs, computer networks, and digital systems design. I have practical knowledge of C
112 </p>
113 </div>
114 
115 </div>
116 </div>
117 </div>
118 </div>
119 </div>
120 </div>
121 </div>
122 </div>
123 </div>
124 </div>
125 </div>
126 </div>
127 </div>
128 </div>
129 </div>
130
```

```
File Edit Selection View ... network_pro1
Servers.py data.txt main_en.html
main_en.html > html > body > div.container > div.box > img.profile-img
2 <html lang="en">
9 <body>
131
132 <!-- Python Section -->
133 <div class="python-section">
134 <h1>To learn Python click the following link</h1>
135 
136 <a href="https://www.w3schools.com/python/python_strings.asp" target="_blank" class="python-link">
137 Python Strings Tutorial
138 </a>
139 </div>
140
141
142 <!-- Links Section -->
143 <div class="links">
144 <h3>Access your account or sign up to start your journey.</h3>
145 <a href="register.html">Register</a> |
146 <a href="login.html">Login</a>
147 </div>
148
149
150
151
152 </body>
153 </html>
154
155
```

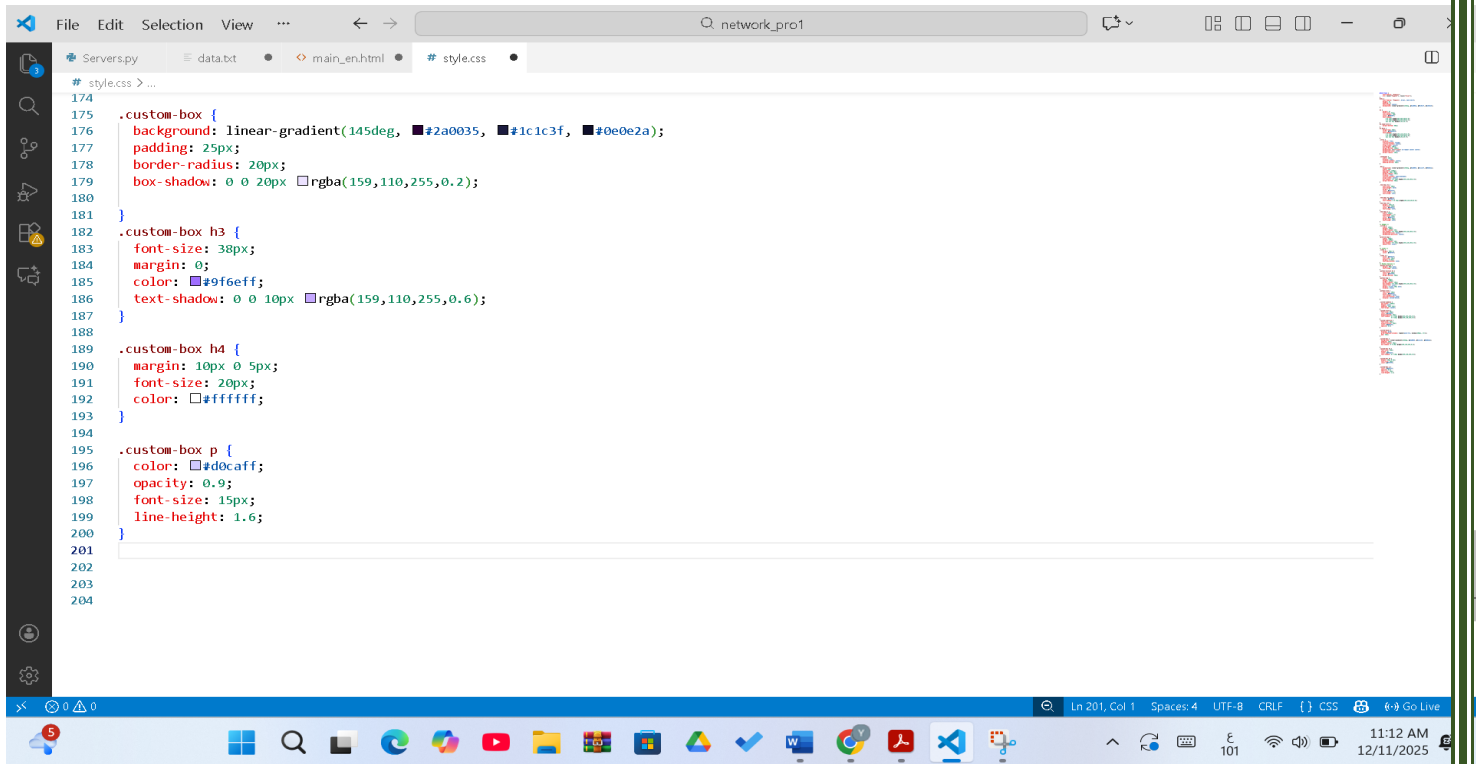
Styling Using CSS:

```
File Edit Selection View ... network_pro1
Servers.py data.txt main_en.html style.css
# style.css > @font-face
1 @font-face {
2 font-family: "Poppins";
3 src: local("Poppins"), local("Arial");
4 }
5 body {
6 font-family: "Poppins", Arial, sans-serif;
7 margin: 0;
8 padding: 0;
9 text-align: center;
10 background: linear-gradient(135deg, #1a002d, #0c0c3f, #130124);
11 }
12
13 h1 {
14 margin: 0;
15 font-size: 42px;
16 font-weight: 700;
17 color: #f0f0ff;
18 text-shadow:
19 0 0 12px rgba(255,255,255,0.5),
20 0 0 25px rgba(255,255,255,0.3),
21 2px 2px 6px rgba(0,0,0,0.7);
22 }
23 h1.team-title {
24 margin-bottom: 80px;
25 }
26 h1.Blue {
27 margin-top: 20px;
28 color: #4bb5e2ce;
29 text-shadow:
30 0 0 12px rgba(159,110,255,0.9),
31 0 0 25px rgba(159,110,255,0.6),
32 2px 2px 6px rgba(0,0,0,0.7);
33 }
34 .intro {
35 display: flex;
36 flex-direction: column;
```



```
File Edit Selection View ... network_pro1
Servers.py data.txt main_en.html style.css
# style.css > python-section h1
103 .profile-img {
106     border-radius: 50%;
107     box-shadow: 0 0 25px rgba(159,110,255,0.4);
108     object-fit: cover;
109 }
110
111 /* Links */
112 .links {
113     margin: 40px 0;
114     color: #f8f5ff;
115 }
116 .links a {
117     color: #9f6eff;
118     font-size: 18px;
119     margin: 0 15px;
120     text-decoration: none;
121 }
122 /* Python Section */
123 .python-section {
124     margin: 80px auto;
125     text-align: center;
126 }
127 .python-section h1 {
128     font-size: 28px;
129     color: #ffffff;
130     margin-bottom: 30px;
131 }
132 .python-img {
133     width: 620px;
134     height: 220px;
135     border-radius: 20px;
136     box-shadow: 0 0 20px rgba(159,110,255,0.4);
137     object-fit: cover;
138     margin: 0 auto 30px auto;
139     display: block;
140 }
```

```
Ln 131, Col 2 Spaces: 4 UTF-8 CRLF CSS Go Live 11:10 AM 12/11/2025
File Edit Selection View ... network_pro1
Servers.py data.txt main_en.html style.css
# style.css > python-img
140
141 .python-link {
142     font-size: 20px;
143     color: #9f6eff;
144     font-weight: bold;
145     text-decoration: none;
146     display: inline-block;
147 }
148
149 .custom-summary {
150     max-width: 1100px;
151     margin: auto;
152     padding: 40px 20px;
153     text-align: center;
154 }
155 .custom-title {
156     font-size: 40px;
157     color: #f0f0ff;
158     margin-bottom: 10px;
159     text-shadow: 0 0 12px rgba(255,255,255,0.5),
160                0 0 25px rgba(159,110,255,0.4);
161 }
162 .custom-subtitle {
163     font-size: 18px;
164     margin-bottom: 40px;
165     color: #d0caff;
166     opacity: 0.9;
167 }
168
169 .custom-boxes {
170     display: grid;
171     grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
172     gap: 25px;
173 }
174
175 .custom-box {
176     background: linear-gradient(145deg, #a3a3ff, #d0caff, #f0f0ff);
177 }
```



Screenshot:

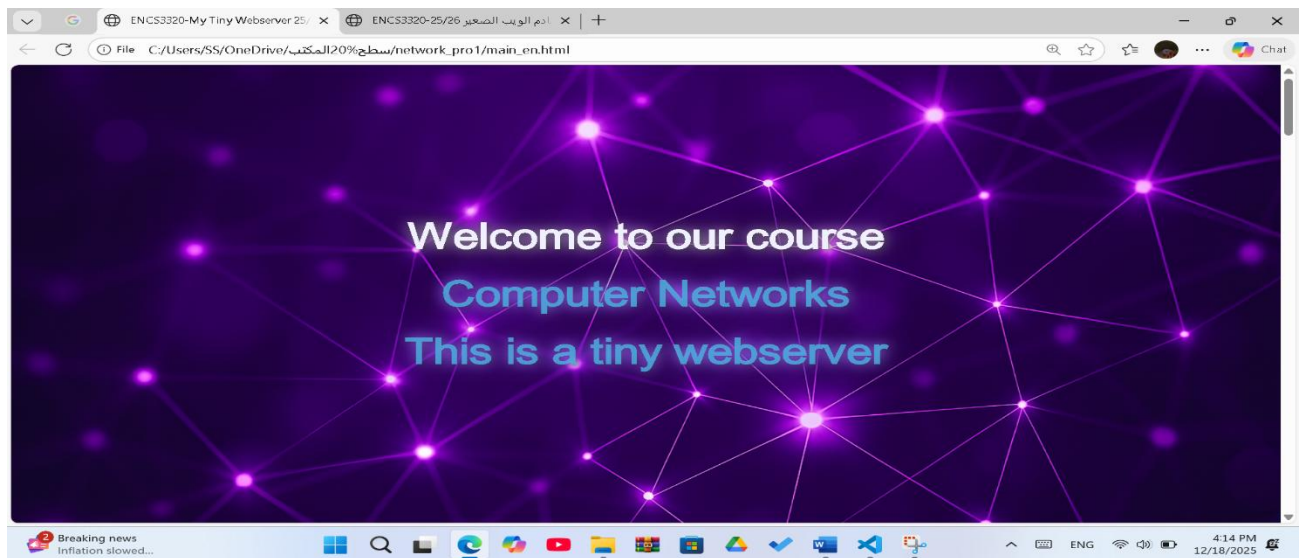
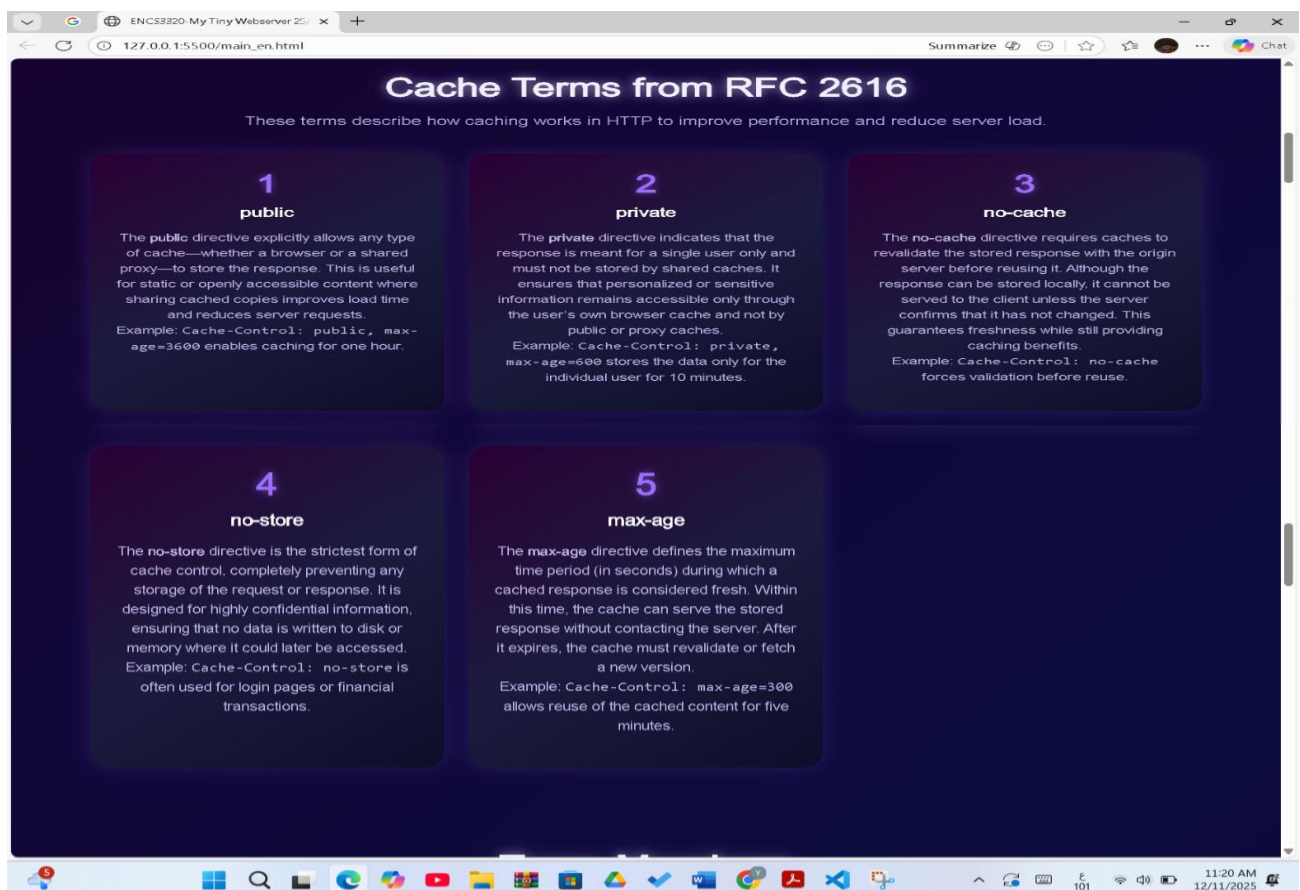


Figure 17:main_en.html page

Figure 18:main_en.html page



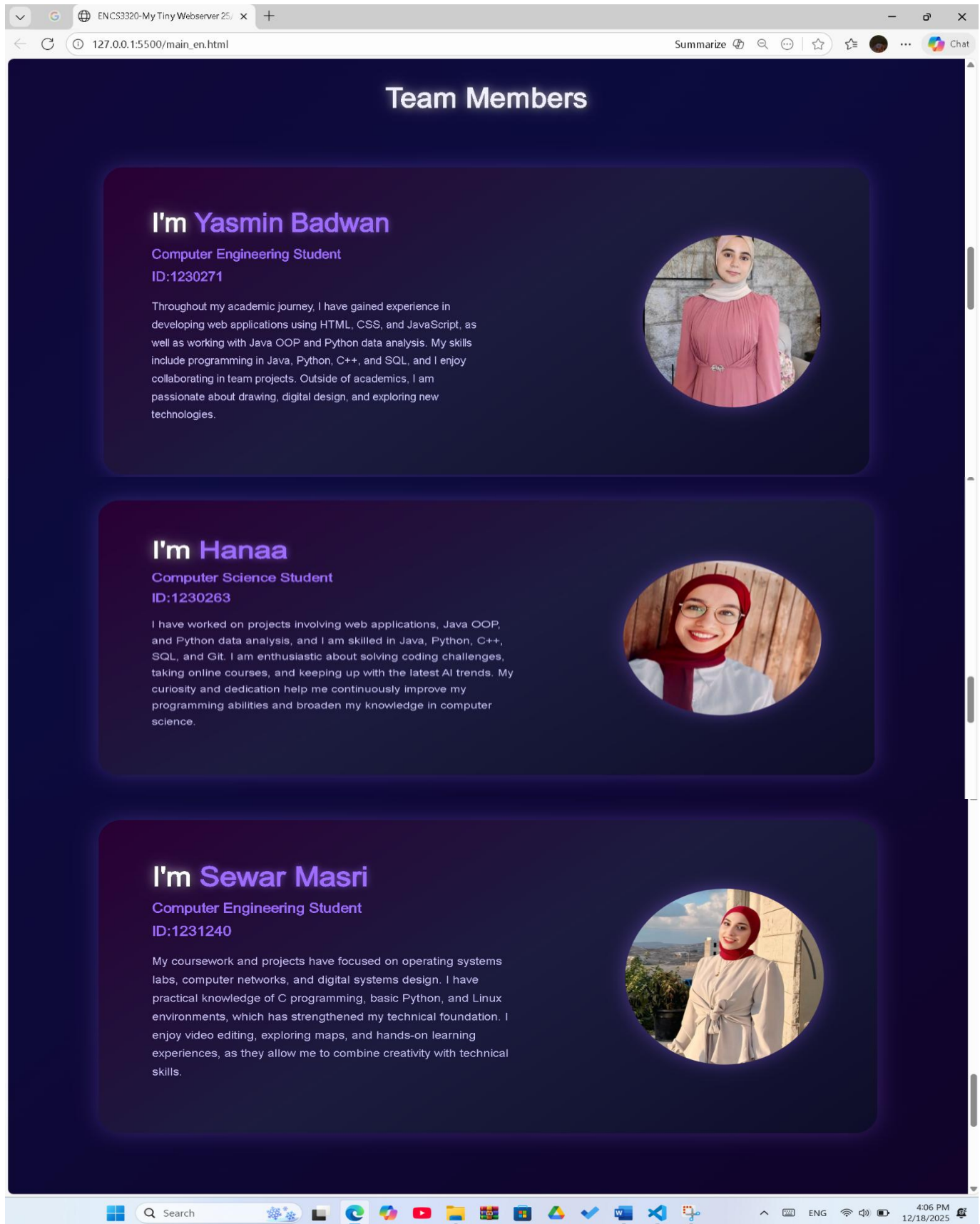


Figure 19:main_en.html page

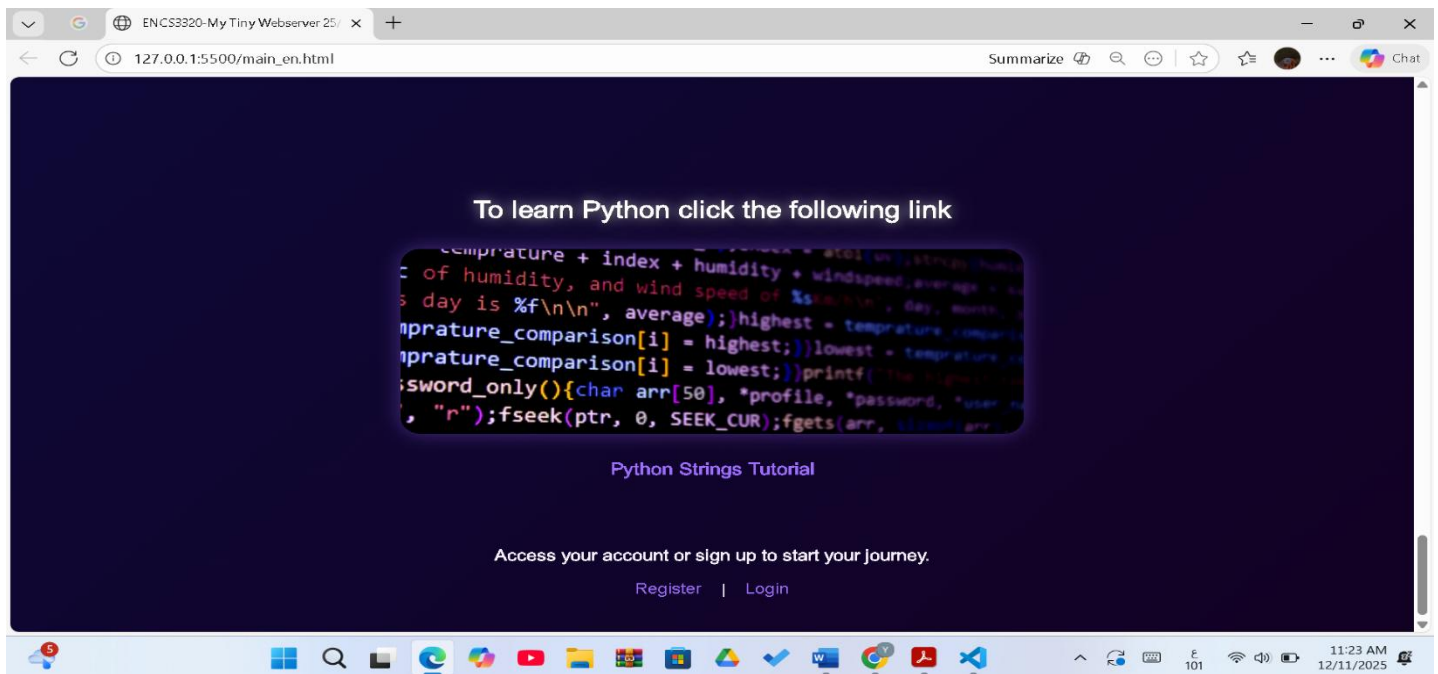


Figure 20:main_en.html page

2. main_ar.html

Description:

The main_ar.html file serves as the Arabic version of the main English webpage and follows the same structure, content organization, and visual layout. It delivers the same user interface experience but with fully translated text and right-to-left formatting to support Arabic readers. All sections—including the welcome message, cache summary, team member information, images, and navigation links—mirror those in the English version to ensure consistency across both languages. The CSS styling remains the same as the original design, requiring no modifications aside from the natural right-to-left direction applied within the HTML page itself.

HTML Code:

The HTML code stayed the same in terms of layout and structure. The only change made was translating all the text into Arabic while keeping the same boxes, sections, images, and links as in the English version.

Styling Using CSS:

The CSS styling was also kept identical to the original design, maintaining the same colors, gradients, shadows, fonts, and layout behavior. The only adjustments made were related to direction and alignment to support Arabic (RTL) formatting. Specifically, properties such as text-align: right and direction: rtl were applied to replace the original left-aligned settings.

Screenshot:

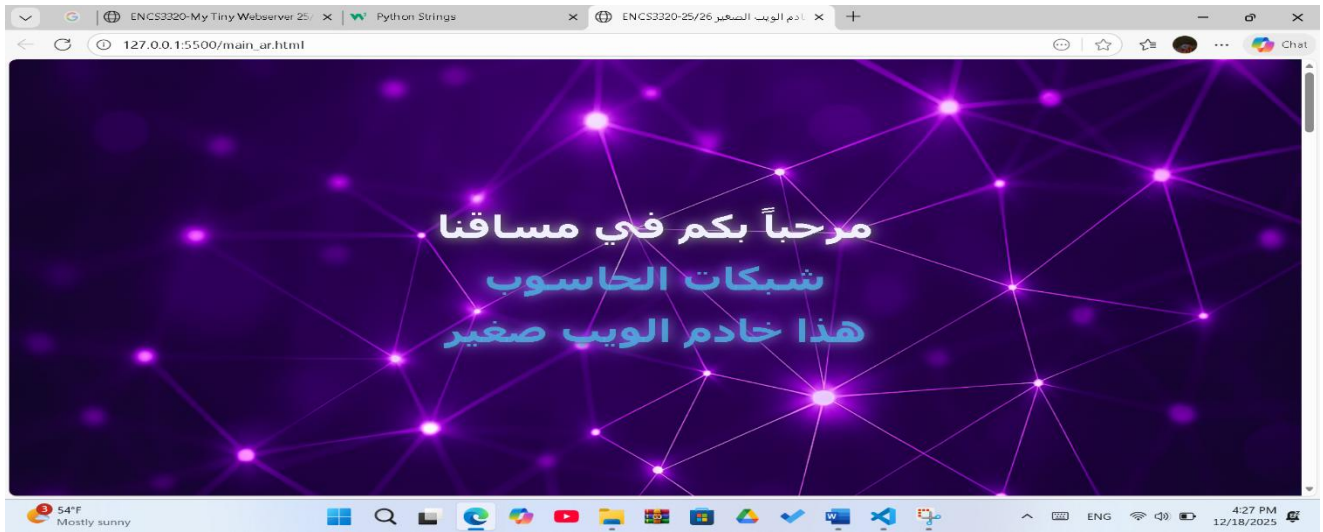


Figure 21:main_ar.html page



Figure 22:main_ar.html page



Figure 23:main_ar.html page

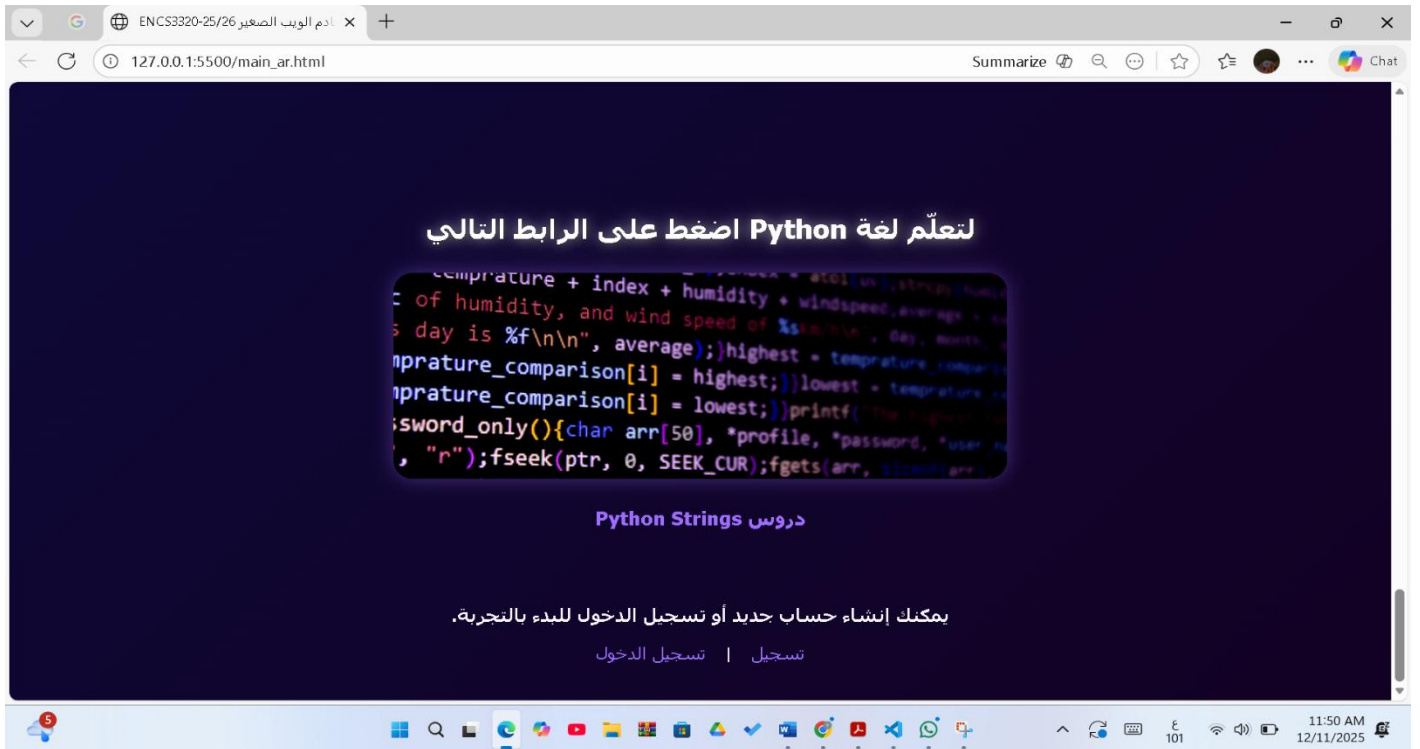


Figure 24:main_ar.html page

3. Registration Page (register.html):

- **Description of HTML User Interface:**

The registration page, contained within register.html, provides a user-friendly interface designed for new users to seamlessly create an account on the web server. The interface is anchored by an eye-catching icon circle at the top, a circular visual element featuring a user icon styled with a depth-creating radial gradient from purple to dark, which establishes immediate visual appeal. A clear and prominent "Register" title directly communicates the page's purpose to the user. Below this, the functional elements include a dedicated text input field for the user to enter their desired username and a secure password field where characters are masked (•••) for privacy. A prominently styled "Create Account" submit button, featuring a purple gradient, serves as the primary action, while a convenient navigation link directs users who already possess an account to the login page. Overall, the design adheres to modern UI principles, employing a consistent purple color scheme (#9f6eff), contemporary aesthetics through rounded input fields with a border-radius of 999px, a visually engaging dark gradient background, and responsive sizing to ensure suitability across various screen sizes.

- **HTML Code for Register Page:**

```

106 <body>
107   <div class="register-container">
108     <div class="icon-circle"></div>
109     <h2>Register</h2>
110
111     <form method="POST" action="/register">
112       <div class="input-box">
113         <label>Username</label>
114         <input type="text" name="username" placeholder="Enter username" required>
115       </div>
116
117       <div class="input-box">
118         <label>Password</label>
119         <input type="password" name="password" placeholder="....." required>
120       </div>
121
122       <button type="submit" class="register-btn">Create Account</button>
123     </form>
124
125     <p class="link-text">
126       Already have an account? <a href="/login.html">Login here</a>
127     </p>
128   </div>
129 </body>
130 </html>
131

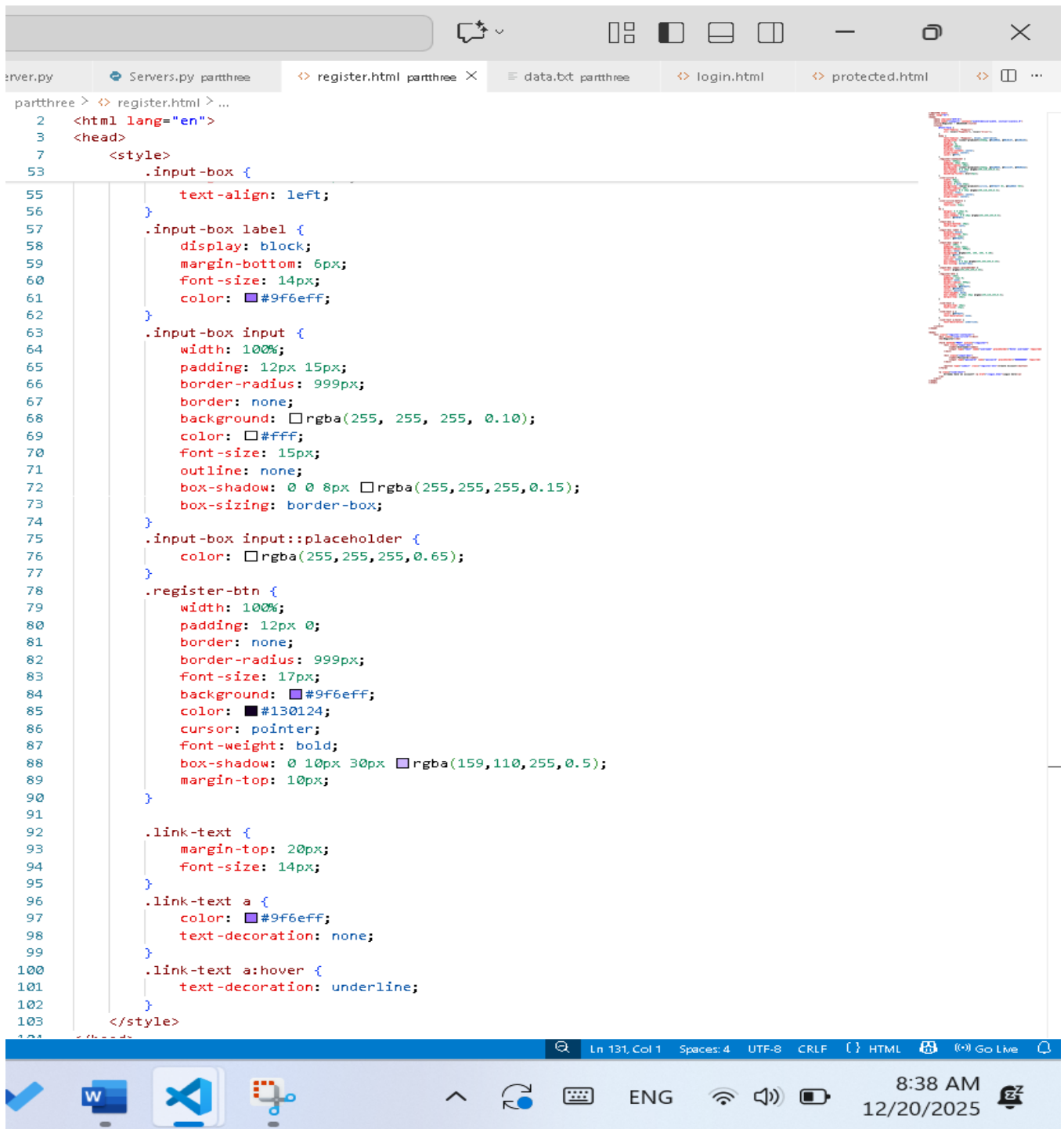
```

- **Styling Using CSS :**

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Register - ENCS3320</title>
7    <style>
8      @font-face {
9        font-family: "Poppins";
10       src: local("Poppins"), local("Arial");
11     }
12     body {
13       font-family: "Poppins", Arial, sans-serif;
14       background: linear-gradient(135deg, #1a002d, #0c0c3f, #130124);
15       margin: 0;
16       padding: 0;
17       height: 100vh;
18       display: flex;
19       justify-content: center;
20       align-items: center;
21       color: #fff;
22     }
23     .register-container {
24       width: 360px;
25       padding: 45px 40px;
26       border-radius: 30px;
27       background: linear-gradient(145deg, #2a0035, #1c1c3f, #0e0e2a);
28       box-shadow: 0 0 30px rgba(159,110,255,0.3);
29       text-align: center;
30       backdrop-filter: blur(4px);
31     }
32     .icon-circle {
33       width: 90px;
34       height: 90px;
35       margin: 0 auto 25px;
36       background: radial-gradient(circle, #9f6eff 0%, #2a0035 70%);
37       border-radius: 50%;
38       box-shadow: 0 0 20px rgba(159,110,255,0.5);
39       display: flex;
40       justify-content: center;
41       align-items: center;
42     }
43     .icon-circle:before {
44       content: "👤";
45       font-size: 42px;
46     }
47     h2 {
48       margin: 0 0 20px 0;
49       font-size: 28px;
50       text-shadow: 0 0 10px rgba(255,255,255,0.4);
51       color: #fff0ff;
52     }
53     .input-box {
54       margin-bottom: 20px;

```



- **Screenshot of register page :**

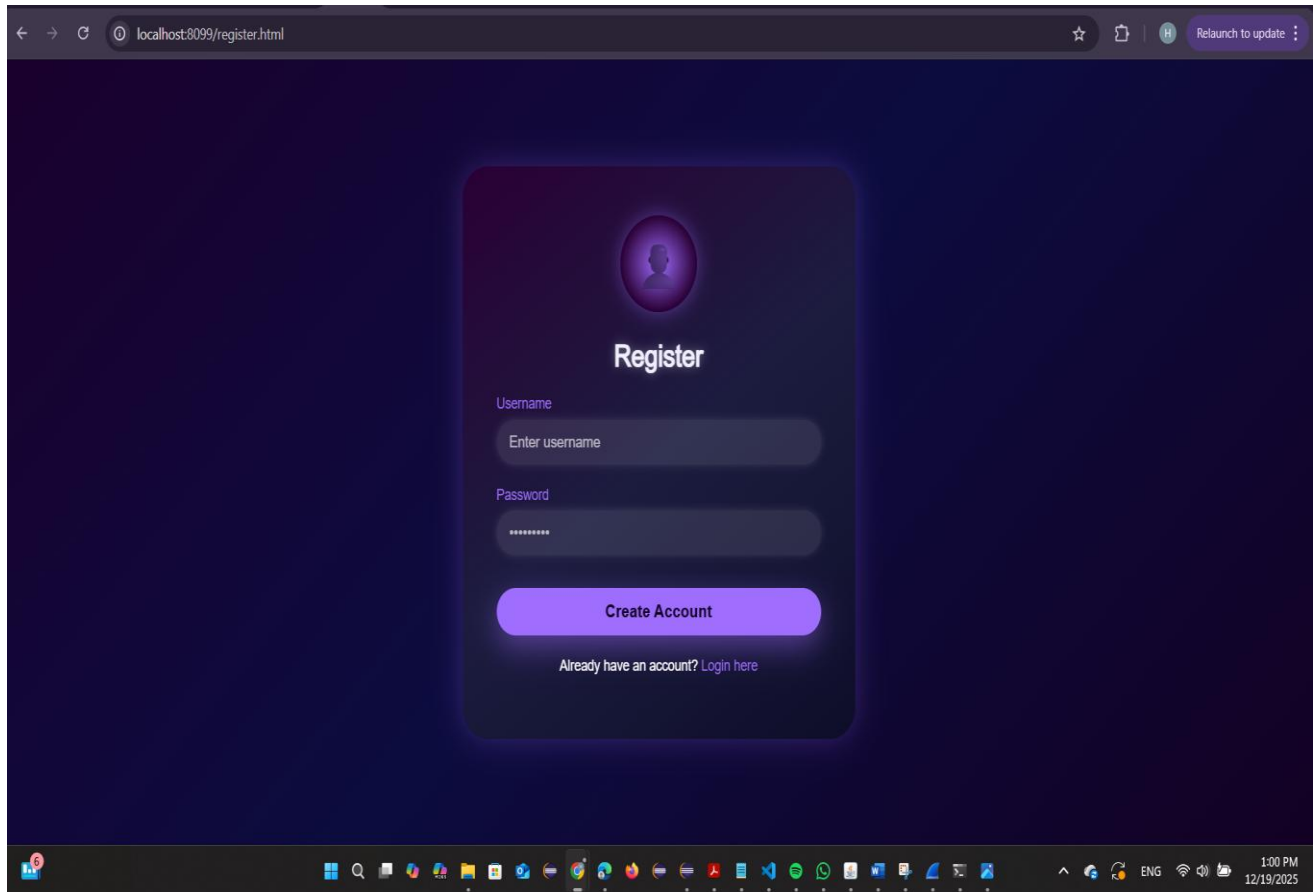


Figure 25:Registration page interface displaying the form for new user account creation

2. Login Page (login.html) :

- **Description of HTML User Interface :**

The login page provides secure authentication for existing users with a feature set designed for clarity and security, beginning with a prominent icon circle featuring a power symbol (⏻) enhanced by a glowing visual effect to intuitively suggest system access. A clearly defined "Login" heading, styled with a subtle text shadow for added depth, anchors the page's purpose. Users are presented with a dedicated input field for entering their registered username and a secure password field that masks characters for privacy protection. The primary action is facilitated by an eye-catching purple button labeled "Login," while a convenient navigation link offers a straightforward redirect to the registration page for new users needing to create an account.

- **HTML Code for Login Page :**

```

108
109 <body>
110     <div class="login-container">
111         <div class="icon-circle"></div>
112         <h2>Login</h2>
113
114         <form method="POST" action="/login">
115             <div class="input-box">
116                 <label>Username</label>
117                 <input type="text" name="username" placeholder="Enter username" required>
118             </div>
119
120             <div class="input-box">
121                 <label>Password</label>
122                 <input type="password" name="password" placeholder="....." required>
123             </div>
124
125             <button type="submit" class="login-btn">Login</button>
126         </form>
127
128         <p class="link-text">
129             Don't have an account? <a href="/register.html">Register here</a>
130         </p>
131     </div>
132 </body>
133 </html>
134

```

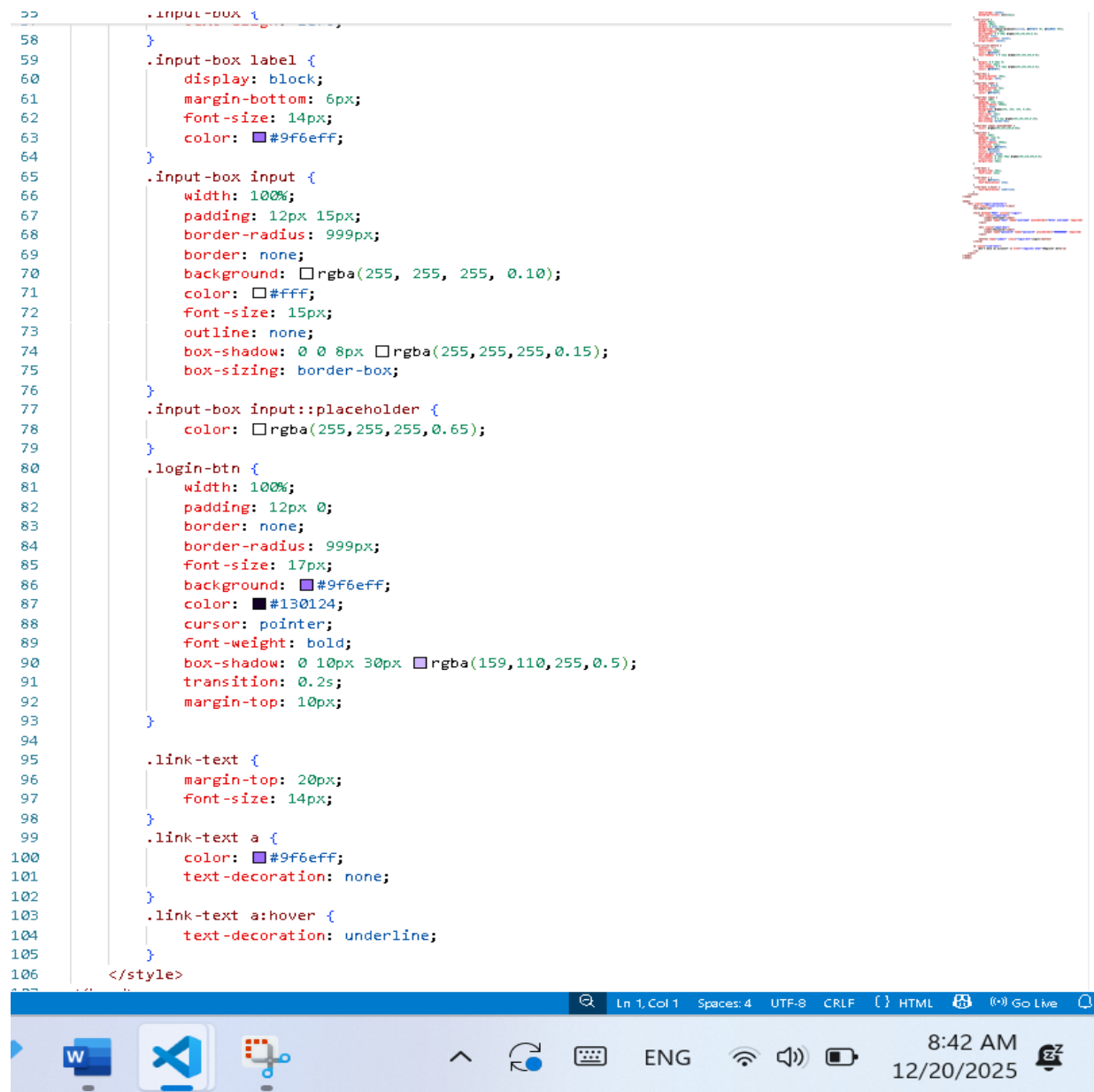
- **Styling Using CSS:**

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>Login - ENCS3320</title>
7     <style>
8         @font-face {
9             font-family: "Poppins";
10            src: local("Poppins"), local("Arial");
11        }
12        body {
13            font-family: "Poppins", Arial, sans-serif;
14            background: linear-gradient(135deg, #1a002d, #0c0c3f, #130124);
15            margin: 0;
16            padding: 0;
17            height: 100vh;
18            display: flex;
19            justify-content: center;
20            align-items: center;
21            color: #fff;
22        }
23        .login-container {
24            width: 380px;
25            padding: 45px 40px;
26            border-radius: 30px;
27            background: linear-gradient(145deg, #2a0035, #1c1c3f, #0e0e2a);
28            box-shadow: 0 0 30px rgba(159,110,255,0.3);
29            text-align: center;
30            backdrop-filter: blur(4px);
31        }
32        .icon-circle {
33            width: 90px;
34            height: 90px;
35            margin: 0 auto 25px;
36            background: radial-gradient(circle, #9f6eff 0%, #2a0035 70%);
37            border-radius: 50%;
38            box-shadow: 0 0 20px rgba(159,110,255,0.5);
39            display: flex;
40            justify-content: center;
41            align-items: center;
42        }
43        .icon-circle:before {
44            content: "👤";
45            font-size: 42px;
46            color: #e8e8ff;
47            text-shadow: 0 0 15px rgba(255,255,255,0.8);
48        }
49        h2 {
50            margin: 0 0 20px 0;
51            font-size: 28px;
52            text-shadow: 0 0 10px rgba(255,255,255,0.4);
53            color: #f0f0ff;
54        }

```

```
55 .input-box {
56
57 }
58
59 .input-box label {
60     display: block;
61     margin-bottom: 6px;
62     font-size: 14px;
63     color: #9f6eff;
64 }
65
66 .input-box input {
67     width: 100%;
68     padding: 12px 15px;
69     border-radius: 999px;
70     border: none;
71     background: rgba(255, 255, 255, 0.10);
72     color: #fff;
73     font-size: 15px;
74     outline: none;
75     box-shadow: 0 0 8px rgba(255, 255, 255, 0.15);
76     box-sizing: border-box;
77 }
78
79 .input-box input::placeholder {
80     color: rgba(255, 255, 255, 0.65);
81 }
82
83 .login-btn {
84     width: 100%;
85     padding: 12px 0;
86     border: none;
87     border-radius: 999px;
88     font-size: 17px;
89     background: #9f6eff;
90     color: #130124;
91     cursor: pointer;
92     font-weight: bold;
93     box-shadow: 0 10px 30px rgba(159, 110, 255, 0.5);
94     transition: 0.2s;
95     margin-top: 10px;
96 }
97
98 .link-text {
99     margin-top: 20px;
100     font-size: 14px;
101 }
102
103 .link-text a {
104     color: #9f6eff;
105     text-decoration: none;
106 }
107
108 .link-text a:hover {
109     text-decoration: underline;
110 }
111
112 </style>
```



- **Screenshot for login page :**

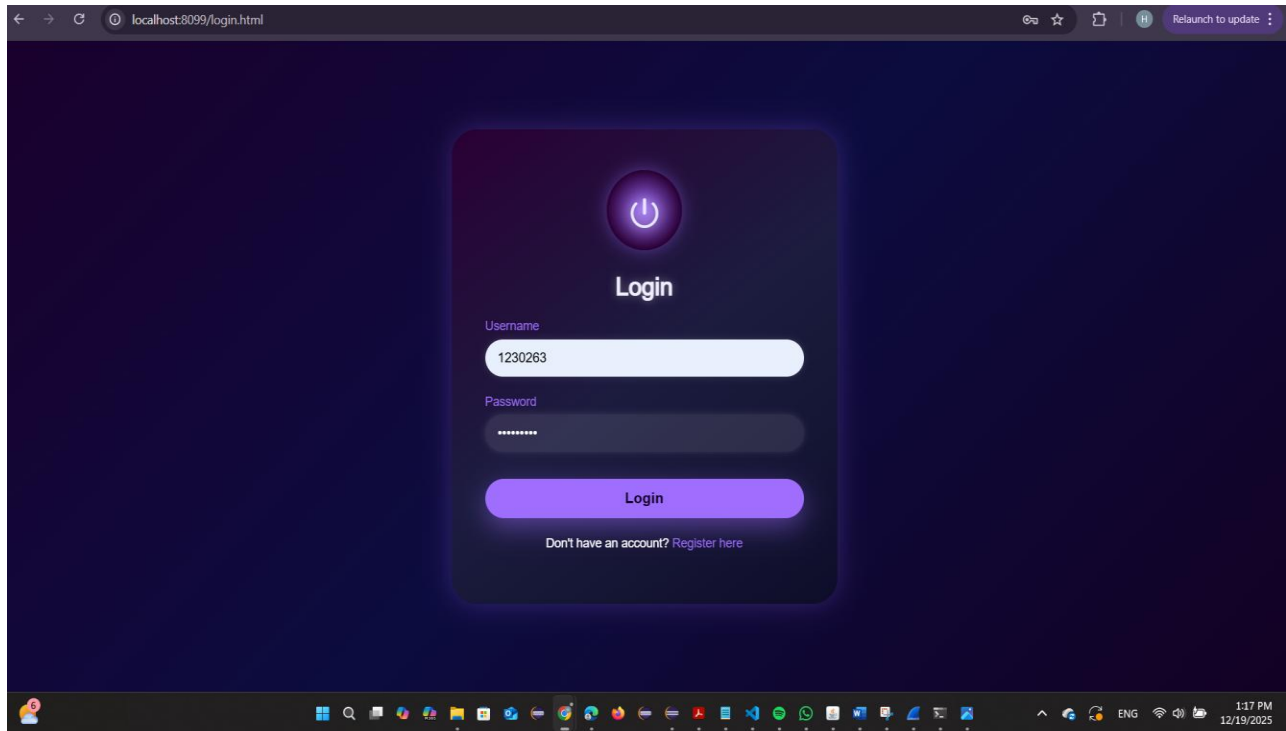


Figure 26:login page

3. Protected Page (protected.html) :

- **Description:**

The protected page serves as the secure, authenticated area accessible only to logged-in users, designed to clearly communicate a successful and safeguarded session. Its key features are led by a prominent green checkmark (✓) displayed within a circular badge, serving as an immediate success icon that visually confirms authentication. A personalized welcome message greets the user by displaying their logged-in username, fostering a sense of individual access. Critical session details are presented in a structured, semi-transparent information box that lists login confirmation, protected page access status, a session activity indicator, notes on multi-tab functionality, and information regarding cookie storage. For session termination, a clearly identifiable red-themed logout button is provided as the primary call-to-action. Throughout the interface, subtle security indicators and visual cues consistently reinforce the secure, authenticated state of the user. The design employs a green accent color (#4CAF50) to universally indicate success and authorized access, presents information in a professional and structured format, and maintains consistency with the site-wide design language, all while placing a clear emphasis on security and transparent session management.

- html code for protected page:

```

89
90 <body>
91   <div class="protected-container">
92     <div class="icon-circle"></div>
93     <h2>Welcome!</h2>
94
95
96     <div class="info-box">
97       <p>✓ You are successfully logged in</p>
98       <p>✓ This is a protected page</p>
99       <p>✓ Your session is active</p>
100      <p>✓ You can open this page in multiple browser tabs and remain logged in</p>
101      <p>✓ The session cookie is stored in your browser</p>
102    </div>
103
104    <form method="POST" action="/logout">
105      <button type="submit" class="logout-btn">Logout</button>
106    </form>
107  </div>
108 </body>
109 </html>
110

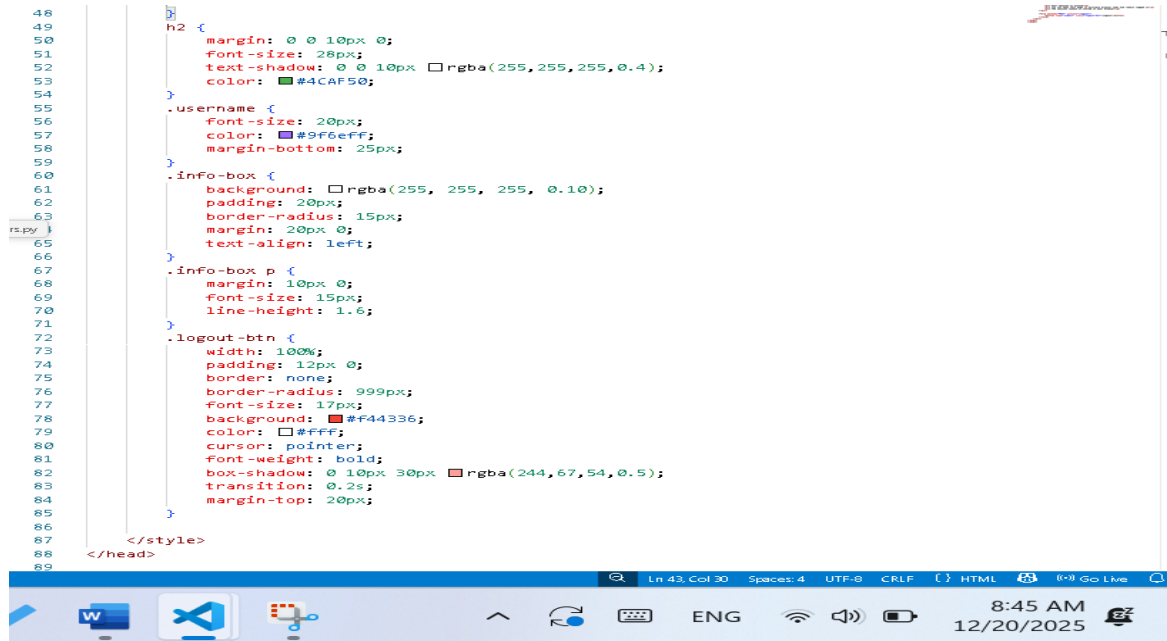
```

- Styling Using CSS :

```

er.py Servers.py parthree <> register.html parthree <> login.html parthree <> protected.html parthree x data.b
parthree > <> protected.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Protected Page - ENCS3320</title>
7   <style>
8     @font-face {
9       font-family: "Poppins";
10      src: local("Poppins"), local("Arial");
11    }
12    body {
13      font-family: "Poppins", Arial, sans-serif;
14      background: linear-gradient(135deg, #1a002d, #0c0c3f, #130124);
15      margin: 0;
16      padding: 0;
17      min-height: 100vh;
18      display: flex;
19      justify-content: center;
20      align-items: center;
21      color: #fff;
22    }
23    .protected-container {
24      width: 500px;
25      padding: 45px 40px;
26      border-radius: 30px;
27      background: linear-gradient(145deg, #2a0035, #1c1c3f, #0e0e2a);
28      box-shadow: 0 0 30px rgba(159,110,255,0.3);
29      text-align: center;
30      backdrop-filter: blur(4px);
31    }
32    .icon-circle {
33      width: 90px;
34      height: 90px;
35      margin: 0 auto 25px;
36      background: radial-gradient(circle, #4CAF50 0%, #2a0035 70%);
37      border-radius: 50%;
38      box-shadow: 0 0 20px rgba(76,175,80,0.5);
39      display: flex;
40      justify-content: center;
41      align-items: center;
42    }
43    .icon-circle:before {
44      content: "✓";
45      font-size: 50px;
46      color: #fff;
47      font-weight: bold;
48    }
49    h2 {
50      margin: 0 0 10px 0;
51      font-size: 28px;
52      text-shadow: 0 0 10px rgba(255,255,255,0.4);
53      color: #4CAF50;
54

```



• Screenshot of Protected Page

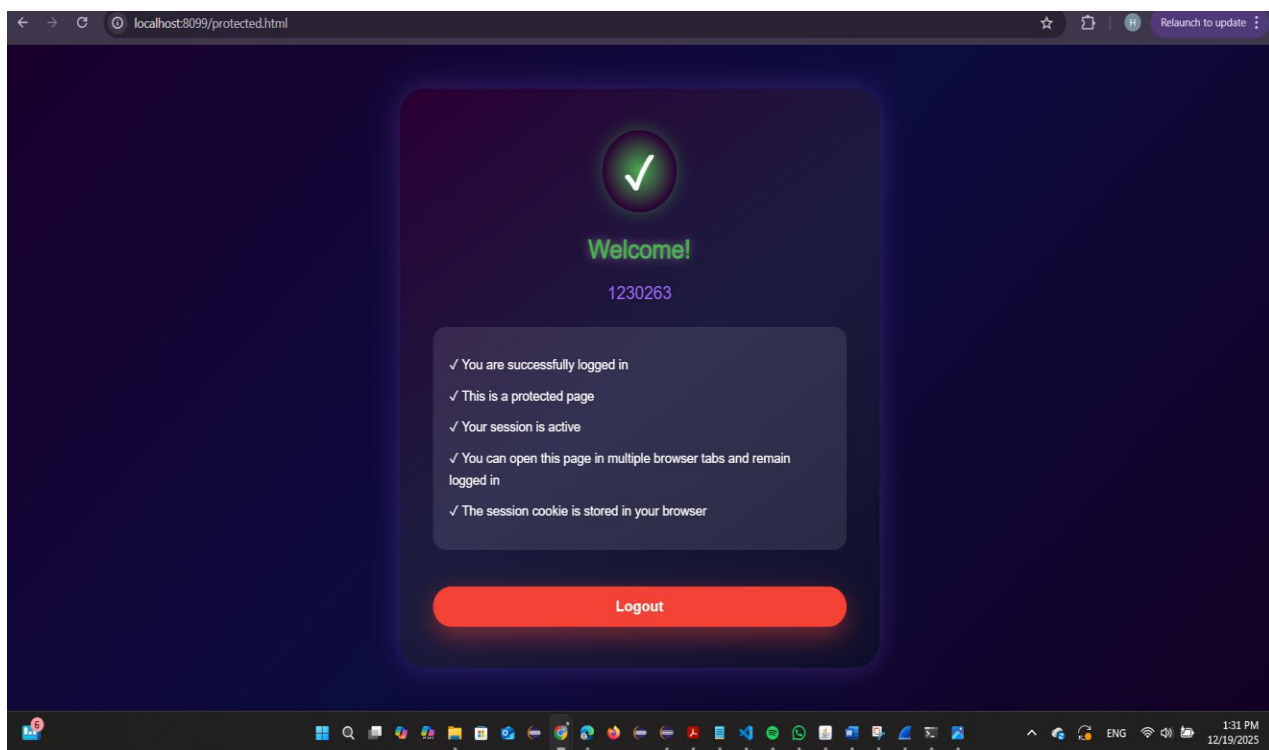
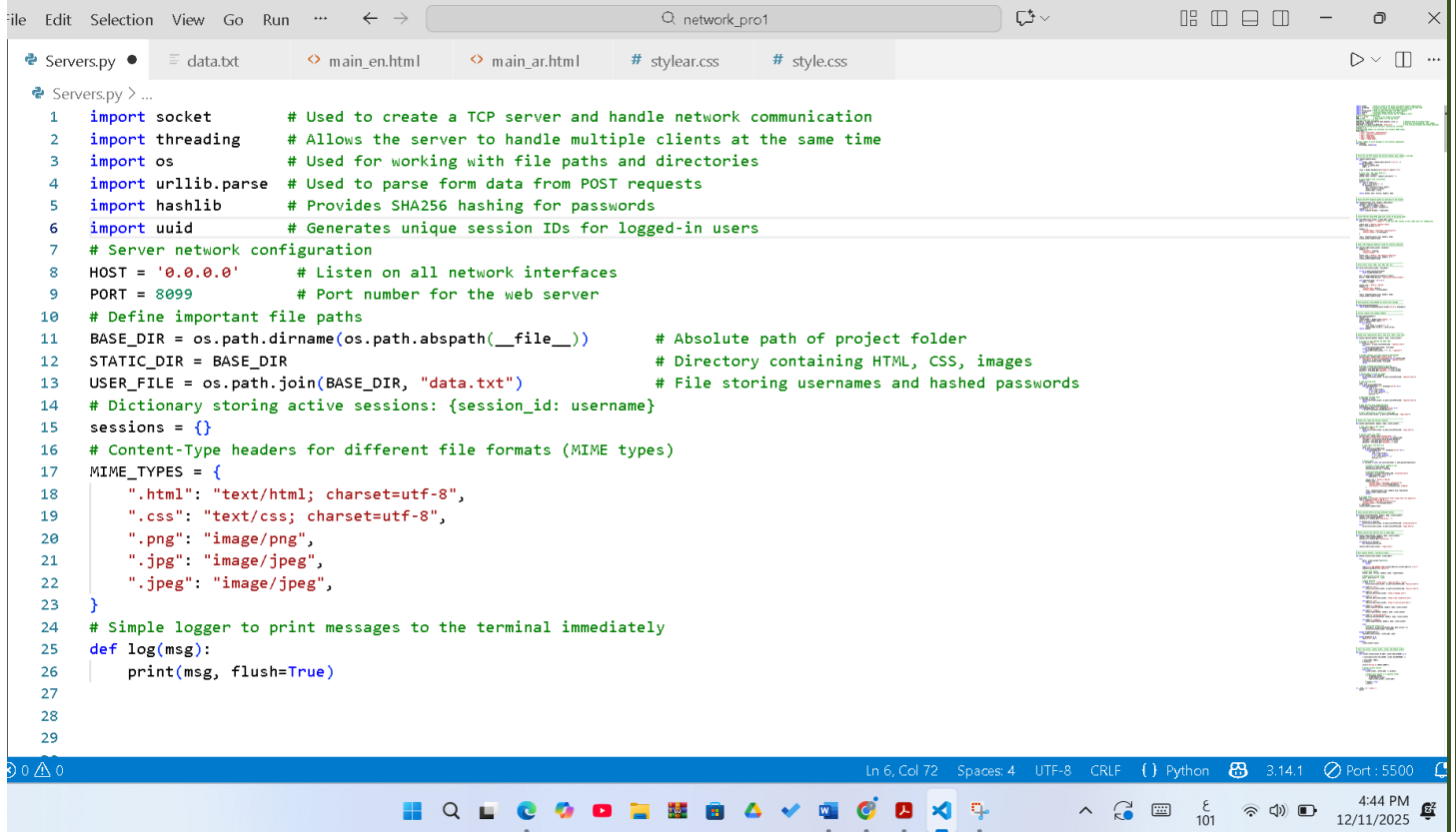


Figure 27:protected page

Web Server Implementation (server.py)

This section explains the logic and implementation of the Python web server. The server uses low-level socket programming to receive HTTP requests, parse them, generate responses, and manage user authentication.



```
1 import socket          # Used to create a TCP server and handle network communication
2 import threading        # Allows the server to handle multiple clients at the same time
3 import os               # Used for working with file paths and directories
4 import urllib.parse      # Used to parse form data from POST requests
5 import hashlib           # Provides SHA256 hashing for passwords
6 import uuid             # Generates unique session IDs for logged-in users
7
8 # Server network configuration
9 HOST = '0.0.0.0'         # Listen on all network interfaces
10 PORT = 8099             # Port number for the web server
11
12 # Define important file paths
13 BASE_DIR = os.path.dirname(os.path.abspath(__file__)) # Absolute path of project folder
14 STATIC_DIR = BASE_DIR # Directory containing HTML, CSS, images
15 USER_FILE = os.path.join(BASE_DIR, "data.txt") # File storing usernames and hashed passwords
16
17 # Dictionary storing active sessions: {session_id: username}
18 sessions = {}
19
20 # Content-Type headers for different file formats (MIME types)
21 MIME_TYPES = {
22     ".html": "text/html; charset=utf-8",
23     ".css": "text/css; charset=utf-8",
24     ".png": "image/png",
25     ".jpg": "image/jpeg",
26     ".jpeg": "image/jpeg",
27 }
28
29 # Simple logger to print messages to the terminal immediately
30 def log(msg):
31     print(msg, flush=True)
```

At the beginning of the file, we import several Python modules that the web server needs:

- **Socket:** Used to create the TCP server and handle low-level network communication using sockets. And It allows the server to listen on port 8099 and send/receive HTTP messages.
- **Threading:** Enables the server to handle multiple clients at the same time.
- **Os:** We use it to find the project directory and to build paths to HTML, CSS, image files, and data.txt.
- **urllib.parse:** Used to decode URL-encoded form data that comes from HTML forms
- **hashlib:** Provides cryptographic hash functions such as SHA256. We use it to hash passwords before saving them in data.txt so that passwords are not stored in plain text.
- **Uuid:** Used to generate unique session IDs for logged-in users. Each session ID is a random value that identifies a user's login session in the sessions dictionary.

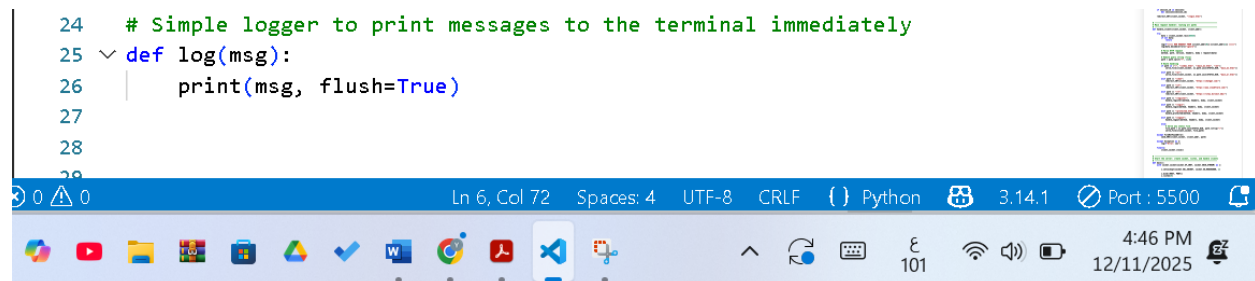
Then we define Global Configuration Variables:

- `HOST = '0.0.0.0'` :Tells the server to listen on all available network interfaces on the machine. This means any client that can reach the computer can connect to the server on port 8099.
- `PORT = 8099` : All browser requests go to `http://localhost:8099/...`
- `BASE_DIR`: This gives the absolute path of the folder where `server.py` is located. We use it as a base to safely build paths to other files in the project.
- `STATIC_DIR`: Points to the folder that contains static files (HTML, CSS, images). In this project, the static files are in the same directory as `server.py`, so `STATIC_DIR` equals `BASE_DIR`.
- `USER_FILE`:Full path to the file that stores user credentials.
- `sessions` :An empty dictionary that will store active user sessions.This allows the server to know which user is logged in when it receives a session cookie.
- `MIME_TYPES`:A dictionary that maps file extensions to the correct Content-Type header for HTTP

Then we defined some function:

log() function:

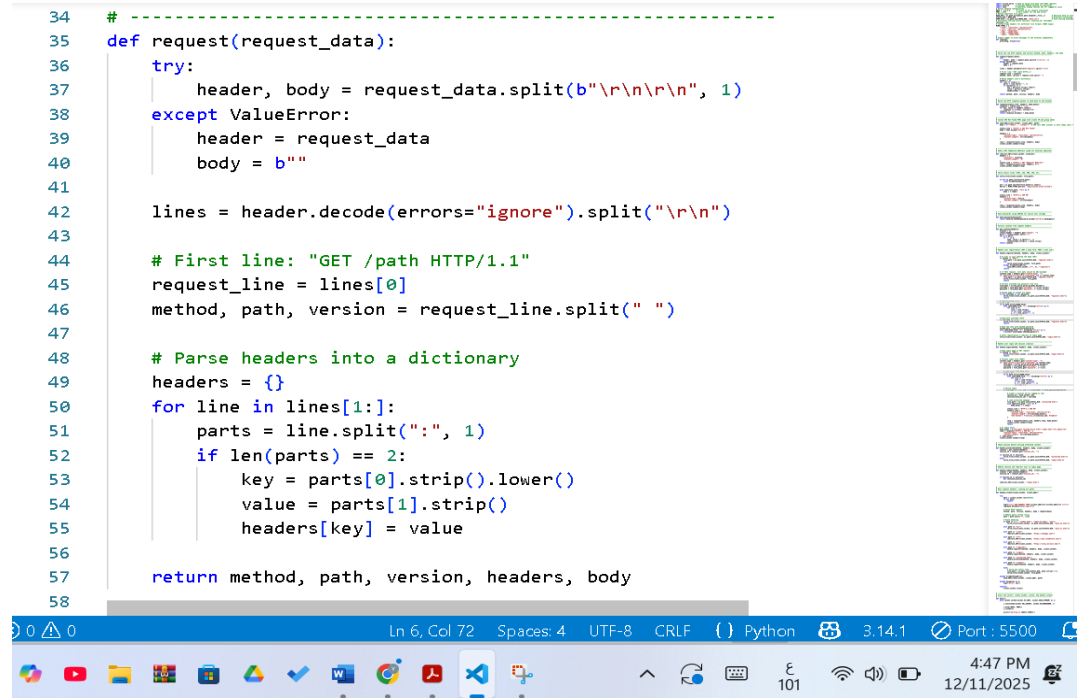
```
24 # Simple logger to print messages to the terminal immediately
25 def log(msg):
26     print(msg, flush=True)
27
28
29
```

The image shows a code editor window with a Python file. The code defines a function `log(msg)` that prints the message to the terminal with `flush=True`. The editor's status bar at the bottom indicates the file is at line 6, column 72, using UTF-8 encoding with CRLF line endings. The right side of the status bar shows the Python version as 3.14.1 and the port as 5500. Below the editor, a Windows taskbar is visible with various application icons. To the right of the code editor, a small terminal window is open, showing a log of HTTP requests and responses, including details like IP addresses, status codes, and response times.

We defined this function to provide a simple and consistent way to display server messages in the terminal. It is mainly used to print incoming HTTP requests, errors, and debugging information while the server is running. By using `flush=True`, the output appears immediately, which is especially important when handling multiple clients through threading.

Request() function:

```
34 # -----
35 def request(request_data):
36     try:
37         header, body = request_data.split(b"\r\n\r\n", 1)
38     except ValueError:
39         header = request_data
40         body = b""
41
42     lines = header.decode(errors="ignore").split("\r\n")
43
44     # First line: "GET /path HTTP/1.1"
45     request_line = lines[0]
46     method, path, version = request_line.split(" ")
47
48     # Parse headers into a dictionary
49     headers = {}
50     for line in lines[1:]:
51         parts = line.split(":", 1)
52         if len(parts) == 2:
53             key = parts[0].strip().lower()
54             value = parts[1].strip()
55             headers[key] = value
56
57     return method, path, version, headers, body
58
```

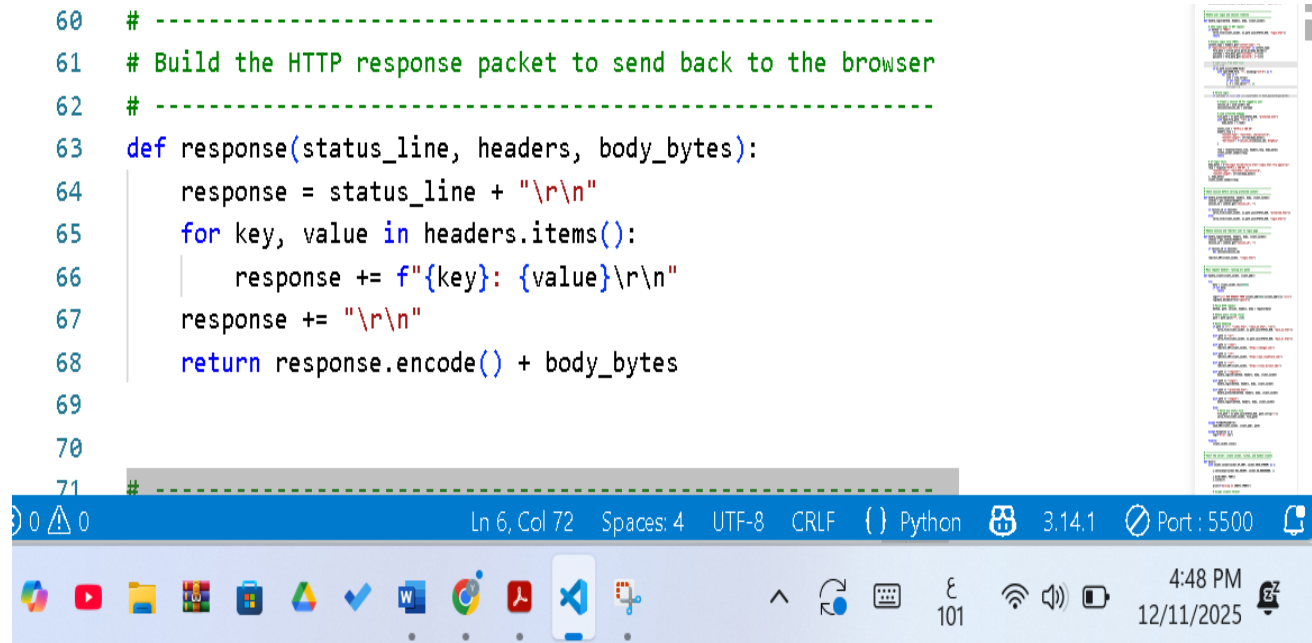
A screenshot of a code editor with a light blue theme. The main window displays a Python function named 'request' with line numbers 34 to 58. The function takes 'request_data' as an argument and returns a tuple of (method, path, version, headers, body). It uses try-except to handle missing body, decodes the header, and parses headers into a dictionary. To the right, a terminal window shows a series of log messages, likely from a web server, showing various HTTP requests and responses. The bottom status bar of the editor shows 'Ln 6, Col 72', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', '3.14.1', 'Port: 5500', and the system clock '4:47 PM 12/11/2025'.

The purpose of this function is to take the raw HTTP message received from the browser and convert it into a structured format that the server can easily understand and work with.

When the browser sends a request, it arrives as a long block of text and bytes that contains the method (GET/POST), the requested path, headers, and possibly a body. To achieve this, the function first tries to split the incoming data into header and body using the separator `\r\n\r\n`, which is the standard boundary between the two sections in HTTP messages. If the request has no body, the function defaults to an empty one. After separating these parts, the header is decoded into readable text and split into individual lines. The first line is the request line, which contains the method, the path, and the HTTP version—these three values tell the server what the client wants. Next, the function processes the remaining lines to extract HTTP headers. Each line is split into a key and value (such as Content-Length, Host, etc.), and stored inside a dictionary for easy access. Finally, the function returns all the extracted information—method, path, version, headers, and body—in a clean, organized format.

response() function

```
60 # -----
61 # Build the HTTP response packet to send back to the browser
62 # -----
63 def response(status_line, headers, body_bytes):
64     response = status_line + "\r\n"
65     for key, value in headers.items():
66         response += f"{key}: {value}\r\n"
67     response += "\r\n"
68     return response.encode() + body_bytes
69
70
71 # -----
```

A screenshot of a code editor window. The main area shows Python code for the response() function. The code is as follows:

```
60 # -----
61 # Build the HTTP response packet to send back to the browser
62 # -----
63 def response(status_line, headers, body_bytes):
64     response = status_line + "\r\n"
65     for key, value in headers.items():
66         response += f"{key}: {value}\r\n"
67     response += "\r\n"
68     return response.encode() + body_bytes
69
70
71 # -----
```

The editor's status bar at the bottom shows "Ln 6, Col 72", "Spaces: 4", "UTF-8", "CRLF", "Python", "3.14.1", and "Port: 5500". Below the status bar is a Windows taskbar with various application icons (File Explorer, Edge, Word, etc.) and system icons (network, volume, battery) on the right. The system clock shows "4:48 PM" and "12/11/2025".

this function takes three inputs—status line, headers, and body—and assembles them into a valid HTTP response. This makes it easy for the server to return any type of content (HTML, CSS, images, etc.) while ensuring the browser receives it in the correct format.

To implement this function, we start by creating a response string that contains the **status line**, followed by a newline sequence (`\r\n`). Then, the function loops through all headers in the dictionary and appends each one to the response in the format `key: value`. After all headers are added, an extra empty line (`\r\n`) is included to indicate the end of the header section, as required by the HTTP protocol. Finally, the function encodes the response header into bytes and appends the body bytes to form the complete HTTP message.

send_404 Function

```
74 def send_404(client_socket, client_addr, path):
75     html = f"""<html>
76     <head>
77         <meta charset="UTF-8">
78         <title>Error 404</title>
79     </head>
80     <body>
81         <h1 style="color:red;">The file is not found</h1>
82
83         <h2>Student Names:</h2>
84         <p><strong>Hanaa Jawad - ID: 1230263</strong></p>
85         <p><strong>Yasmeen Badwan - ID: 2345678</strong></p>
86         <p><strong>Sewar Masri - ID: 3456789</strong></p>
87
88         <h3>Client Information:</h3>
89         <p><strong>Client IP:</strong> {client_addr[0]}</p>
90         <p><strong>Client Port:</strong> {client_addr[1]}</p>
91     </body>
92     </html>"""
93
94     # The full HTML content is here (kept short for readability)
95
96     status_line = "HTTP/1.1 404 Not Found"
97     body = html.encode("utf-8")
98
99     headers = {
100         "Content-Type": "text/html; charset=utf-8",
101         "Content-Length": str(len(body))
102     }
103
104     resp = response(status_line, headers, body)
105     client_socket.sendall(resp)
106
107
```

The `send_404()` function is responsible for handling situations where the client requests a file that does not exist on the server. Instead of returning a blank screen or causing the server to fail, this function generates a complete custom 404 error page and sends it back to the browser. This ensures a user-friendly and informative response whenever an invalid path is requested.

Inside the function, an HTML template is prepared to display a styled error message along with details such as the students' names, their IDs, and the client's IP address and port. These details help identify the context of the request and satisfy the project requirements for the custom error page. After creating the HTML content, it is encoded into bytes so it can be transmitted over the network.

The function then constructs an appropriate HTTP response by setting the status line to "HTTP/1.1 404 Not Found" and generating the necessary headers, including the content type and content length. These components, along with the body of the error page, are passed to the `response()` function, which assembles them into a properly formatted HTTP response. Finally, the completed response is sent to the client through the socket connection using `client_socket.sendall()`.

serve_file() function

```
121 # -----
122 # Serve static files: HTML, CSS, PNG, JPG, etc.
123 # -----
124 def serve_file(client_socket, file_path):
125     if not os.path.isfile(file_path):
126         raise FileNotFoundError()
127
128     ext = os.path.splitext(file_path)[1].lower()
129     Myfile = MIME_TYPES.get(ext, "application/octet-stream")
130
131     with open(file_path, "rb") as f:
132         body = f.read()
133
134     status_line = "HTTP/1.1 200 OK"
135     headers = {
136         "Content-Type": Myfile,
137         "Content-Length": str(len(body))
138     }
139
140     resp = response(status_line, headers, body)
141     client_socket.sendall(resp)
142
143 # -----
144 # Hash passwords using SHA256 for secure user storage
```

its used to send files such as HTML, CSS, and images to the browser when they are requested. This allows the browser to display the website's pages, styles, and images correctly. When the server receives a request for a file, this function locates the file in the project directory and prepares it to be sent back as an HTTP response.

The function first checks whether the file exists. If it does not, a `FileNotFoundError` is raised so the server can return a 404 error page. If the file exists, the function reads its contents and identifies its type (HTML, CSS, PNG, JPG, etc.) using the file extension. This ensures the browser receives the correct content type and knows how to display the file.

After reading the file, the function builds a normal HTTP "200 OK" response and includes the necessary headers, such as the file's content type and size. It then sends the full response back to the browser through the socket connection using the `response()` function. This allows the requested file to appear properly on the user's screen.

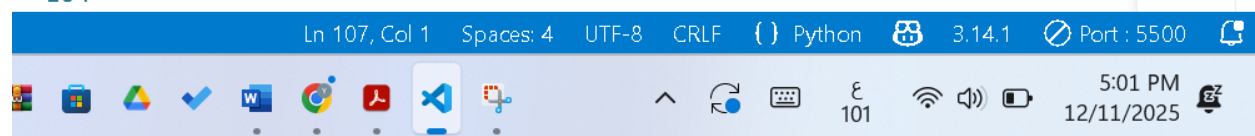
hash_password() function

```
145 # -----
146 # Hash passwords using SHA256 for secure user storage
147 # -----
148 def hash_password(password):
149     return hashlib.sha256(password.encode("utf-8")).hexdigest()
150
```

The function is used to securely transform a user's password into a SHA256 hash before storing it in the data.txt file. Instead of saving the password in plain text the function converts the password into a fixed, irreversible hexadecimal string.

get_cookies() function

```
152 # -----
153 # Extract cookies from request headers
154 # -----
155 def get_cookies(headers):
156     cookies = {}
157     cookie_header = headers.get("cookie", "")
158     parts = cookie_header.split(";")
159     for p in parts:
160         if "=" in p:
161             name, value = p.split("=", 1)
162             cookies[name.strip()] = value.strip()
163     return cookies
164
```



The get_cookies() function is used to read and extract cookies from the HTTP request sent by the browser. Cookies are small pieces of data that the browser automatically sends with every request, and in this project they are used to identify logged-in users through their session ID.

The function first looks for the "cookie". If it does not exist, it simply works with an empty string. Then it splits the cookie string into smaller parts using ; because multiple cookies can be sent together. Each part should have the format name=value, so the function separates the name from the value and removes any extra spaces.

Finally, the function stores all extracted cookies in a dictionary and returns it. This makes it easy for the server to check whether a session cookie was sent and whether the user is still logged in.

handle_register() function

```
167 # Handle user registration (GET → show form, POST → save user)
168 # -----
169 def handle_register(method, headers, body, client_socket):
170
171     # If user is just opening the page (GET)
172     if method != "POST":
173         file_path = os.path.join(STATIC_DIR, "register.html")
174         try:
175             serve_file(client_socket, file_path)
176         except FileNotFoundError:
177             send_404(client_socket, ("?", 0), "/register")
178         return
179
180     # If POST request, form data should be URL-encoded
181     content_type = headers.get("content-type", "")
182     if "application/x-www-form-urlencoded" not in content_type:
183         file_path = os.path.join(STATIC_DIR, "register.html")
184         serve_file(client_socket, file_path)
185         return
186
187     # Extract username and password from form
188     form_data = urllib.parse.parse_qs(body.decode())
189     username = form_data.get("username", [""])[0].strip()
190     password = form_data.get("password", [""])[0].strip()
191
192     # Reload page if fields are empty
193     if not username or not password:
194         serve_file(client_socket, os.path.join(STATIC_DIR, "register.html"))
195         return
196
197     # Load existing users
198     users = {}
199     if os.path.exists(USER_FILE):
200         with open(USER_FILE, "r", encoding="utf-8") as f:
201             for line in f:
202                 line = line.strip()
203                 if not line: continue
204                 u, h = line.split(":", 1)
205                 users[u] = h
206
207     # Duplicate username check
208     if username in users:
209         serve_file(client_socket, os.path.join(STATIC_DIR, "register.html"))
210         return
211
212     # Save new user with hashed password
213     hashed_pass = hash_password(password)
214     with open(USER_FILE, "a", encoding="utf-8") as f:
215         f.write(f"{username}:{hashed_pass}\n")
216
217     # After registration → redirect to login page
218     serve_file(client_socket, os.path.join(STATIC_DIR, "login.html"))
219
```

this function is responsible for processing the user registration process. It handles both displaying the registration page and creating a new user account when the form is submitted.

The function first checks whether the request method is POST. If it is not, this means the user is simply trying to open the registration page, so the server responds by sending register.html. If the file cannot be found, a 404 error page is returned instead. If the method is POST, the function

ensures that the form data was submitted in the correct format (application/x-www-form-urlencoded). If the content type is incorrect, the server reloads the registration page. After verifying the format, the function extracts the submitted username and password from the request body using `urllib.parse.parse_qs()`. Next, the function checks whether the username or password fields are empty. If either one is missing, the registration page is sent again. If both fields are valid, the server loads existing users from `data.txt` to check whether the username already exists. If the username is already taken, the registration page is shown again. If the username is new, the password is hashed using the `hash_password()` function, and the pair `username:hashed_password` is saved to the `data.txt` file. A message confirming successful registration is printed on the server side. Finally, after creating the account, the server sends the `login.html` page to guide the user to the login process

handle_login Function

```

221 # -----
222 # Handle user login and session creation
223 # -----
224 def handle_login(method, headers, body, client_socket):
225     # Show login page if GET request
226     if method != "POST":
227         serve_file(client_socket, os.path.join(STATIC_DIR, "login.html"))
228         return
229     # Process login form (POST)
230     content_type = headers.get("content-type", "")
231     if "application/x-www-form-urlencoded" in content_type:
232         form_data = urllib.parse.parse_qs(body.decode())
233         username = form_data.get("username", [""])[0]
234         password = form_data.get("password", [""])[0]
235         # Load users from data file
236         users = {}
237         if os.path.exists(USER_FILE):
238             with open(USER_FILE, "r", encoding="utf-8") as f:
239                 for line in f:
240                     line = line.strip()
241                     if not line: continue
242                     u, h = line.split(":", 1)
243                     users[u] = h
244
245         # Verify login
246         if username in users and users[username] == hash_password(password):
247
248             # Create a session ID for logged-in user
249             session_id = uuid.uuid4().hex
250             sessions[session_id] = username
251
252             # Load protected webpage
253             file_path = os.path.join(STATIC_DIR, "protected.html")
254             with open(file_path, "rb") as f:
255                 body_bytes = f.read()
256             status_line = "HTTP/1.1 200 OK"
257             headers_resp = {
258                 "Content-Type": "text/html; charset=utf-8",
259                 "Content-Length": str(len(body_bytes)),
260                 "Set-Cookie": f"session_id={session_id}; HttpOnly"
261             }
262             resp = response(status_line, headers_resp, body_bytes)
263             client_socket.sendall(resp)
264             return
265
266         # If login fails
267         body_bytes = b"<h1>login failed</h1><a href='/login.html'>Try again</a>"
268         resp = response("HTTP/1.1 200 OK", {
269             "Content-Type": "text/html; charset=utf-8",
270             "Content-Length": str(len(body_bytes))
271         }, body_bytes)
272         client_socket.sendall(resp)
273

```

The function is responsible for managing the entire login process. It determines whether the user is simply requesting the login page or attempting to submit their username and password for authentication.

The function begins by checking the HTTP method. If the method is not POST, this means the user just wants to open the login form, so the server responds by sending login.html. If the file is missing, the server displays the 404 error page. This ensures that the login page is shown correctly whenever users access /login.

When the method is POST, the function processes the submitted login form. It first checks whether the content type is application/x-www-form-urlencoded, which confirms that the form data is in the correct format. The function then extracts the submitted username and password from the request body.

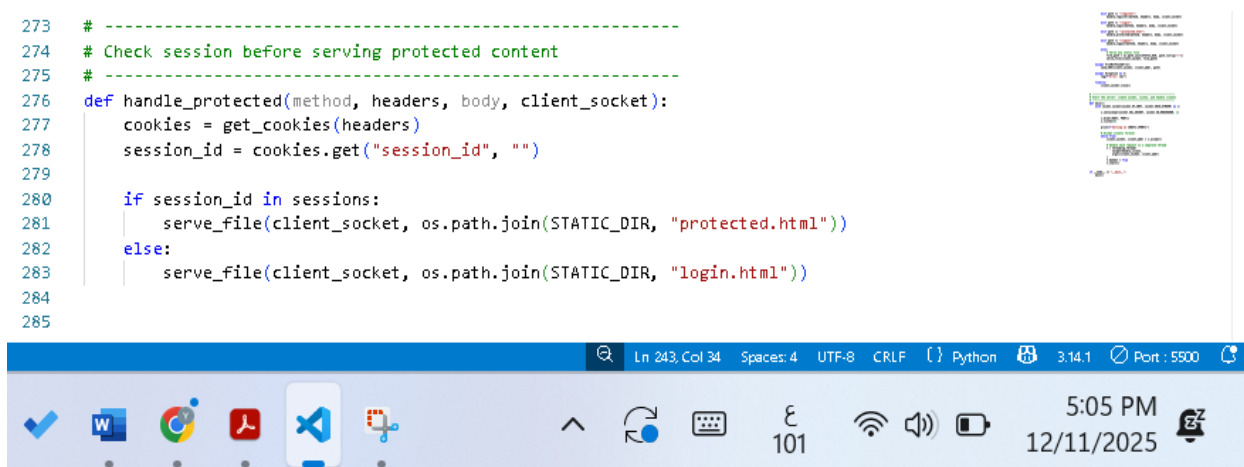
Next, the server loads all registered users from the data.txt file. Each line in the file contains a username and a hashed password. If the submitted username exists in the file, the server hashes the entered password and compares it with the stored hash. If they match, the login is successful.

When authentication succeeds, the server creates a new session by generating a unique session_id using uuid.uuid4().hex. This session ID is stored in the sessions dictionary and sent back to the browser through a Set-Cookie header. This allows the browser to remain logged in while navigating through different pages. After creating the session, the server loads and sends protected.html to the client, indicating successful login.

If the username does not exist or the password is incorrect, the login attempt fails. In this case, the server sends a simple HTML message indicating “Login failed” along with a link to try again.

handle_protected() function

```
273 # -----
274 # Check session before serving protected content
275 # -----
276 def handle_protected(method, headers, body, client_socket):
277     cookies = get_cookies(headers)
278     session_id = cookies.get("session_id", "")
279
280     if session_id in sessions:
281         serve_file(client_socket, os.path.join(STATIC_DIR, "protected.html"))
282     else:
283         serve_file(client_socket, os.path.join(STATIC_DIR, "login.html"))
284
285
```



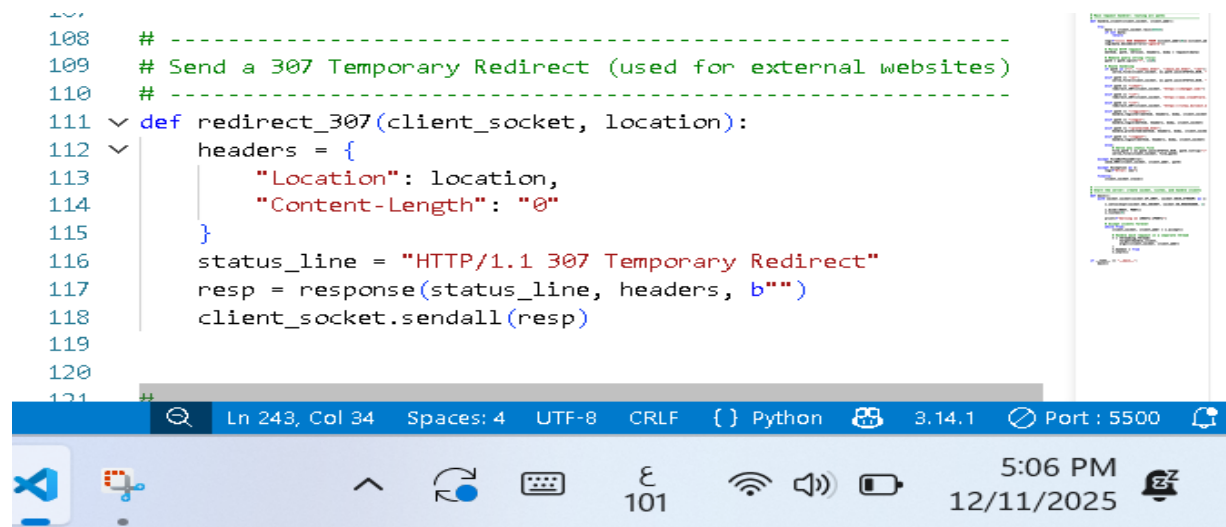
This function ensures that only authenticated users can view protected content by checking session cookies and forwarding unauthorized users to the login page.

The function begins by extracting cookies from the request headers using `get_cookies()`. It then looks specifically for the `session_id` cookie, which is assigned to the user during login. If this session ID exists and is found in the sessions dictionary, the server knows that the user is authenticated. In this case, the function loads and sends `protected.html` to the browser.

If the session ID is missing, invalid, or not found in the session database, the user does not have permission to access the protected page. Instead of showing the restricted content, the function redirects the user back to `login.html` so they can log in again. If any of these required files are missing, the server displays the custom 404 error page.

Redirect_307 Function

```
108 # -----
109 # Send a 307 Temporary Redirect (used for external websites)
110 # -----
111 def redirect_307(client_socket, location):
112     headers = {
113         "Location": location,
114         "Content-Length": "0"
115     }
116     status_line = "HTTP/1.1 307 Temporary Redirect"
117     resp = response(status_line, headers, b"")
118     client_socket.sendall(resp)
119
120
121
```

A screenshot of a code editor window. The main area shows a Python function named `redirect_307` with its implementation. The function takes `client_socket` and `location` as arguments. It creates a dictionary for headers with 'Location' and 'Content-Length' (set to '0'). It then constructs an HTTP status line 'HTTP/1.1 307 Temporary Redirect' and uses the `response` helper function to create a response object. Finally, it sends the response to the client socket using `sendall`. The editor's status bar at the bottom shows 'Ln 243, Col 34', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', '3.14.1', and 'Port: 5500'. A system tray at the very bottom displays icons for a microphone, a blue circular icon, a keyboard, a battery level of 101%, a Wi-Fi signal, a speaker, and the time '5:06 PM' on '12/11/2025'.

is used to send an **HTTP 307 Temporary Redirect** response to the browser. This type of redirect tells the browser to immediately load a different URL while keeping the same HTTP method (such as GET or POST). In this project, the redirect function is used when the user visits paths like `/chat`, `/cf`, or `/rt`, which should automatically redirect the user to external websites such as ChatGPT or Cloudflare.

The function works by creating an HTTP response with the status line "HTTP/1.1 307 Temporary Redirect". It then includes a Location header that specifies the URL the browser should navigate to. The Content-Length is set to 0 because no body content is required for this type of response. After constructing the redirect message using the `response()` helper function, the server sends the complete response to the client through the socket.

handle_logout() function

```
288 # -----
289 # Remove session |
290 # -----
291 def handle_logout(method, headers, body, client_socket):
292     cookies = get_cookies(headers)
293     session_id = cookies.get("session_id", "")
294
295     # Remove session from server memory
296     if session_id in sessions:
297         del sessions[session_id]
298
299     # Delete cookie in the browser + redirect to login
300     status_line = "HTTP/1.1 307 Temporary Redirect"
301     headers_resp = {
302         "Location": "/login.html",
303         "Content-Length": "0",
304         "Set-Cookie": "session_id=; Max-Age=0; Expires=Thu, 01 Jan 1970 00:00:00 GMT; Path=/; HttpOnly"
305     }
306
307     resp = response(status_line, headers_resp, b"")
308     client_socket.sendall(resp)
309
310 # -----
311 # Make request handlers receive all requests
```

The `handle_logout()` function is responsible for properly terminating a user's login session on both the server and the client sides. When a logout request is received, the function first extracts the cookies sent by the browser and checks for the presence of a `session_id`. If a valid session ID exists in the server's sessions dictionary, it is removed immediately, which invalidates the user's session on the server side.

After clearing the session from memory, the function also ensures that the session cookie is deleted from the browser. This is done by sending a `Set-Cookie` header with an empty value and an expired timestamp, forcing the browser to remove the cookie. Finally, the server responds with a 307 Temporary Redirect that redirects the user to the login page (`/login.html`). This approach guarantees a clean logout process, prevents reuse of old session cookies, and ensures that access to protected pages is no longer possible without logging in again.

handle_client() function

```
299 # -----
300 # Main request handler: routing all paths
301 # -----
302 def handle_client(client_socket, client_addr):
303     try:
304         data = client_socket.recv(65535)
305         if not data:
306             return
307         log(f"===== NEW REQUEST FROM {client_addr[0]}:{client_addr[1]} =====")
308         log(data.decode(errors="ignore"))
309         # Parse HTTP request
310         method, path, version, headers, body = request(data)
311         # Remove query string (?x=y)
312         path = path.split("?", 1)[0]
313         # Route handling
314         if path in ("/", "/index.html", "/main_en.html", "/en"):
315             serve_file(client_socket, os.path.join(STATIC_DIR, "main_en.html"))
316         elif path == "/ar":
317             serve_file(client_socket, os.path.join(STATIC_DIR, "main_ar.html"))
318         elif path == "/chat":
319             redirect_307(client_socket, "https://chatgpt.com/")
320         elif path == "/cf":
321             redirect_307(client_socket, "https://www.cloudflare.com/")
322         elif path == "/rt":
323             redirect_307(client_socket, "https://ritaj.birzeit.edu/")
324         elif path == "/register":
325             handle_register(method, headers, body, client_socket)
326         elif path == "/login":
327             handle_login(method, headers, body, client_socket)
328         elif path == "/protected.html":
329             handle_protected(method, headers, body, client_socket)
330         elif path == "/logout":
331             handle_logout(method, headers, body, client_socket)
332         else:
333             # Serve any static file
334             file_path = os.path.join(STATIC_DIR, path.lstrip("/"))
335             serve_file(client_socket, file_path)
336     except FileNotFoundError:
337         send_404(client_socket, client_addr, path)
338     except Exception as e:
339         log(f"Error: {e}")
340     finally:
341         client_socket.close()
342
```

serves as the main request handler of the server, managing every interaction between the client browser and the web application. When a client connects, the server first attempts to read the incoming HTTP request from the socket. If the request is empty, the function returns immediately. Otherwise, the raw request is printed to the terminal using the logging function so the developer can monitor all incoming traffic. The request is then passed to the request() helper function, which extracts the HTTP method, requested path, headers, and body. Before processing the request, any query parameters are removed so that only the actual path is used for routing.

Once the path is cleaned, the server begins determining the correct response. Requests for the English homepage—such as `/`, `/index.html`, `/main_en.html`, or `/en`—result in sending `main_en.html`. Similarly, a request to `/ar` loads the Arabic version of the main page. The function also handles several special paths that trigger HTTP redirection. Visiting `/chat`, `/cf`, or `/rt` produces a 307 Temporary Redirect to external websites such as ChatGPT, Cloudflare, or Ritaj. If the request relates to user registration or login, the function forwards the data to `handle_register()` or `handle_login()`, which manage account creation and authentication. Requests to `/protected.html` use session validation to determine whether the user is logged in and therefore allowed to access the protected page. A request to `/logout` triggers the logout handler, which removes the session and redirects the user back to the login page.

If the requested path does not match any of the predefined routes, the server treats it as a static file request. It attempts to locate the file inside the project directory and sends it back to the browser. If the file does not exist, the server generates a custom 404 Not Found page. Throughout the entire process, any unexpected errors are caught and logged so that the server continues running without interruption. After finishing the response, the server closes the client socket, completing the request–response cycle. This structured flow allows the web server to reliably handle different types of requests while maintaining clear organization and predictable behavior.

```

344 # -----
345 # Start the server: create socket, listen, and handle clients
346 # -----
347 def main():
348     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
349
350         s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
351
352         s.bind((HOST, PORT))
353         s.listen(5)
354
355         print(f"Serving on {HOST}:{PORT}")
356
357         # Accept clients forever
358         while True:
359             client_socket, client_addr = s.accept()
360
361             # Handle each request in a separate thread
362             t = threading.Thread(
363                 target=handle_client,
364                 args=(client_socket, client_addr)
365             )
366             t.daemon = True
367             t.start()
368
369
370 if __name__ == "__main__":
371     main()
372

```



When the program starts, the `main()` function is called and is responsible for launching the entire web server. Inside `main()`, a TCP socket is created and configured to listen on the specified HOST and PORT (0.0.0.0:8099). After binding and starting to listen, the server enters an infinite loop where it waits for clients to connect. Each time a browser sends a request to `http://localhost:8099`, the `accept()` call returns a new `client_socket` and `client_addr`. Instead of handling this request directly in the main thread, the server creates a new thread and assigns it to the `handle_client()` function. This design allows multiple users to be served at the same time without blocking the server.

From this point, the flow moves from `main()` into `handle_client()`, which becomes the central controller for each individual request. Inside `handle_client()`, the raw HTTP data is read from `client_socket` and passed to the `request()` function. This function parses the method, path, headers, and body so that the server can understand what the browser is asking for. Based on the resulting path, `handle_client()` decides which part of the system should respond. If the request is for a normal page such as `/`, `/en`, or `/ar`, it calls `serve_file()` to send `main_en.html` or `main_ar.html`. If the path corresponds to an image, CSS file, or another static resource, `serve_file()` is also used, and it relies on `MIME_TYPES` to choose the correct content type. When the requested file does not exist, `serve_file()` raises a `FileNotFoundError`, and `handle_client()` responds by calling `send_404()` to generate the custom 404 page

Testing and Results

Server Startup

This test verifies that the web server starts correctly and listens on port 8099.

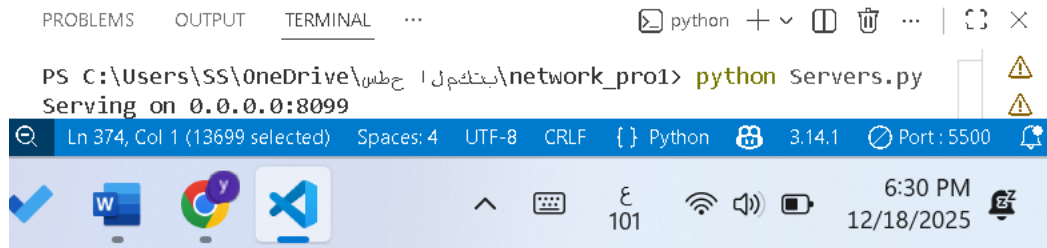


Figure 28:Server running successfully on port 8099

English Main Page Tests

The following tests ensure that the English main page is correctly served for different valid URLs.

- **Accessing English Main Page:**

URL Tested:

<http://localhost:8099/>

Screenshot:

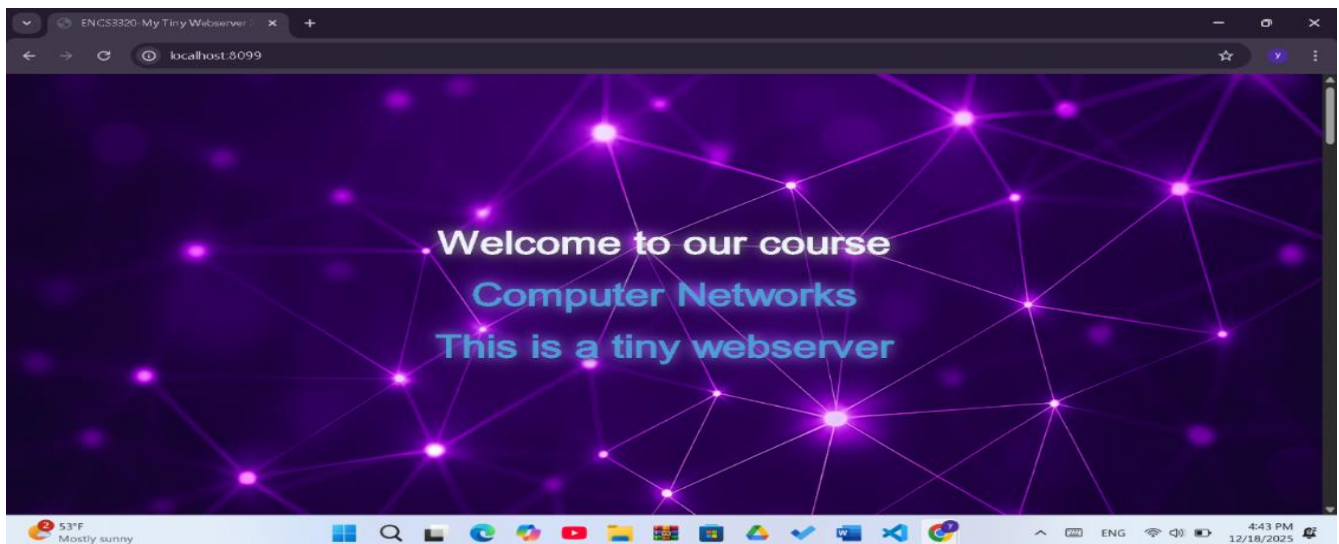


Figure 29:English main page loaded using root path.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

python

Python

Python

```
PS C:\Users\SS\OneDrive\مكتبتي\network_pro1> python Servers.py
Serving on 0.0.0.0:8099
===== NEW REQUEST FROM 127.0.0.1:49645 =====
GET / HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

===== NEW REQUEST FROM 127.0.0.1:59329 =====
GET /style.css HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: text/css,*/*;q=0.1
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: style
Referer: http://localhost:8099/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

===== NEW REQUEST FROM 127.0.0.1:60516 =====
GET /2.jpeg HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
```

Ln 372, Col 1 (13661 selected) Spaces: 4 UTF-8 CRLF Python 3.14.1 Port: 5500

4:45 PM
12/18/2025

Figure 30:Terminal output showing GET of en.page .

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Sec-Fetch-Dest: image
Referer: http://localhost:8099/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

===== NEW REQUEST FROM 127.0.0.1:55971 =====
===== NEW REQUEST FROM 127.0.0.1:56254 =====
GET /3.jpeg HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8099/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

GET /1.jpg HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8099/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

===== NEW REQUEST FROM 127.0.0.1:51096 =====
GET /p.jpg HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
```

Ln 372, Col 1 (13661 selected) Spaces: 4 UTF-8 CRLF {} Python 3.14.1 Port: 5500

4:46 PM 12/18/2025

Figure 31: Terminal output showing GET of en.page .

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8099/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

===== NEW REQUEST FROM 127.0.0.1:61896 =====
GET /0.png HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8099/style.css
Accept-Encoding: gzip, deflate, br, zstd

===== NEW REQUEST FROM 127.0.0.1:62292 =====
GET /favicon.ico HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8099/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

|
```

Ln 372, Col 1 (13661 selected) Spaces: 4 UTF-8 CRLF () Python 3.14.1 Port : 5500

4:46 PM 12/18/2025

Figure 32: Terminal output showing GET en.page .

Accessing English Page Using /en

URL Tested:

<http://localhost:8099/en>

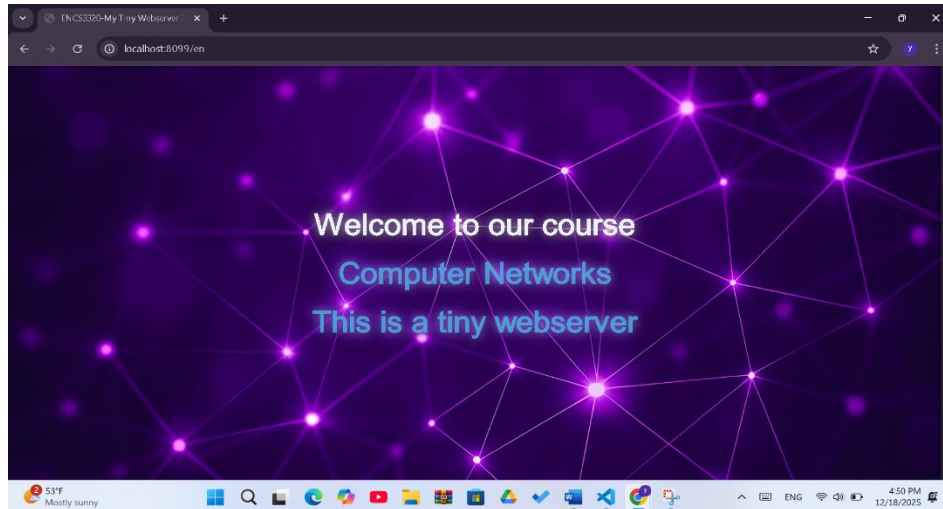
Screenshot:

Figure 33: English page accessed using /en.

Accessing English Page Using main_en.html**URL Tested:**

http://localhost:8099/main_en.html

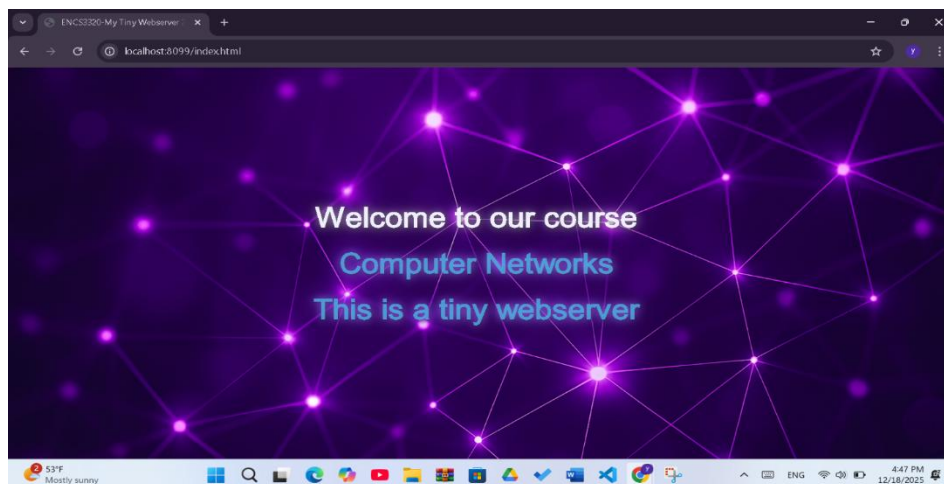
Screenshot:

Figure 34: English page accessed using main_en.html.

Arabic Main Page Test

URL Tested:

<http://localhost:8099/ar>

Screenshot:

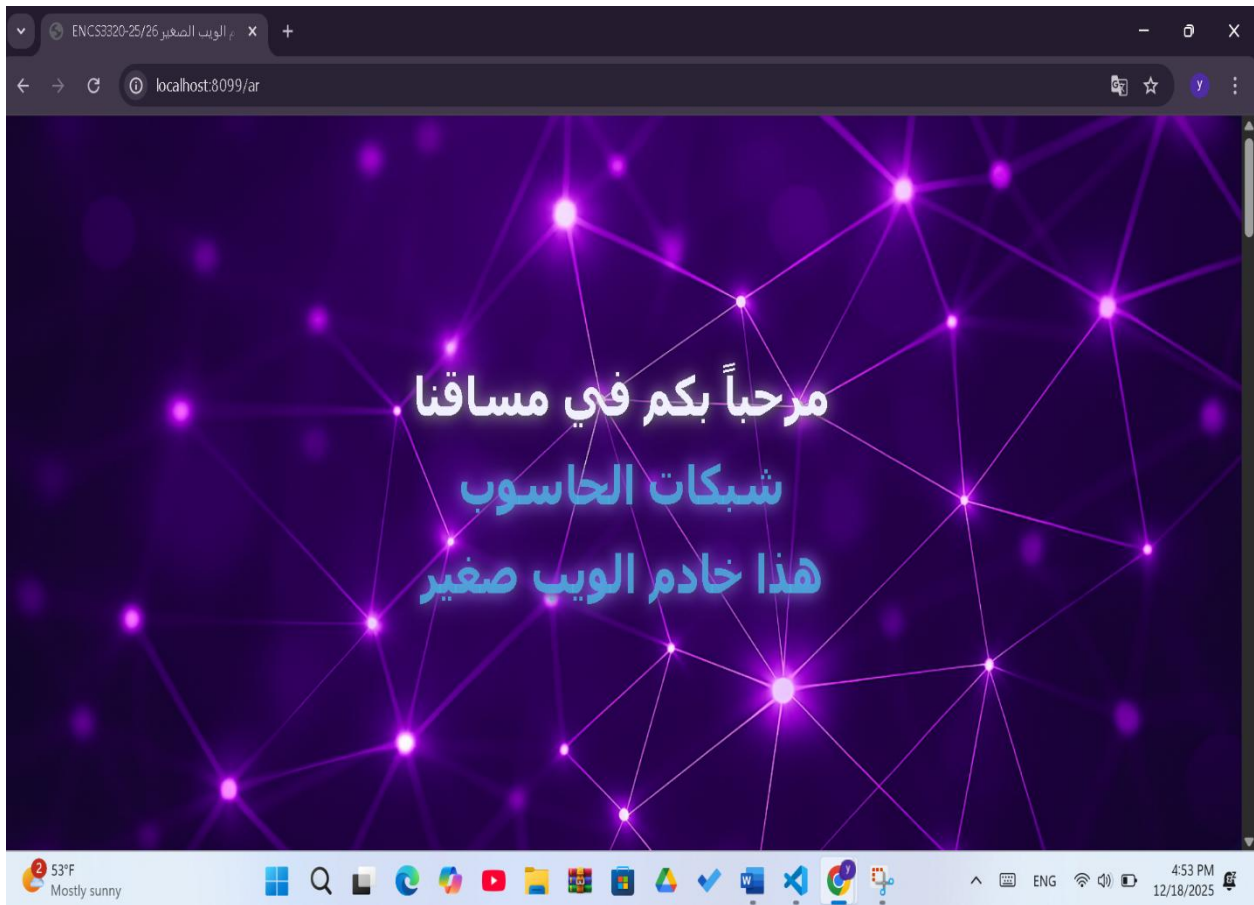


Figure 35:Arabic main page.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

===== NEW REQUEST FROM 127.0.0.1:49704 =====
GET /ar HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

===== NEW REQUEST FROM 127.0.0.1:54593 =====
GET /stylear.css HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: text/css,*/*;q=0.1
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: style
Referer: http://localhost:8099/ar
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

===== NEW REQUEST FROM 127.0.0.1:62754 =====
GET /1.jpg HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8099/ar

Ln 372, Col 1 (13661 selected) Spaces: 4 UTF-8 CRLF {} Python 3.14.1 Port: 5500
```

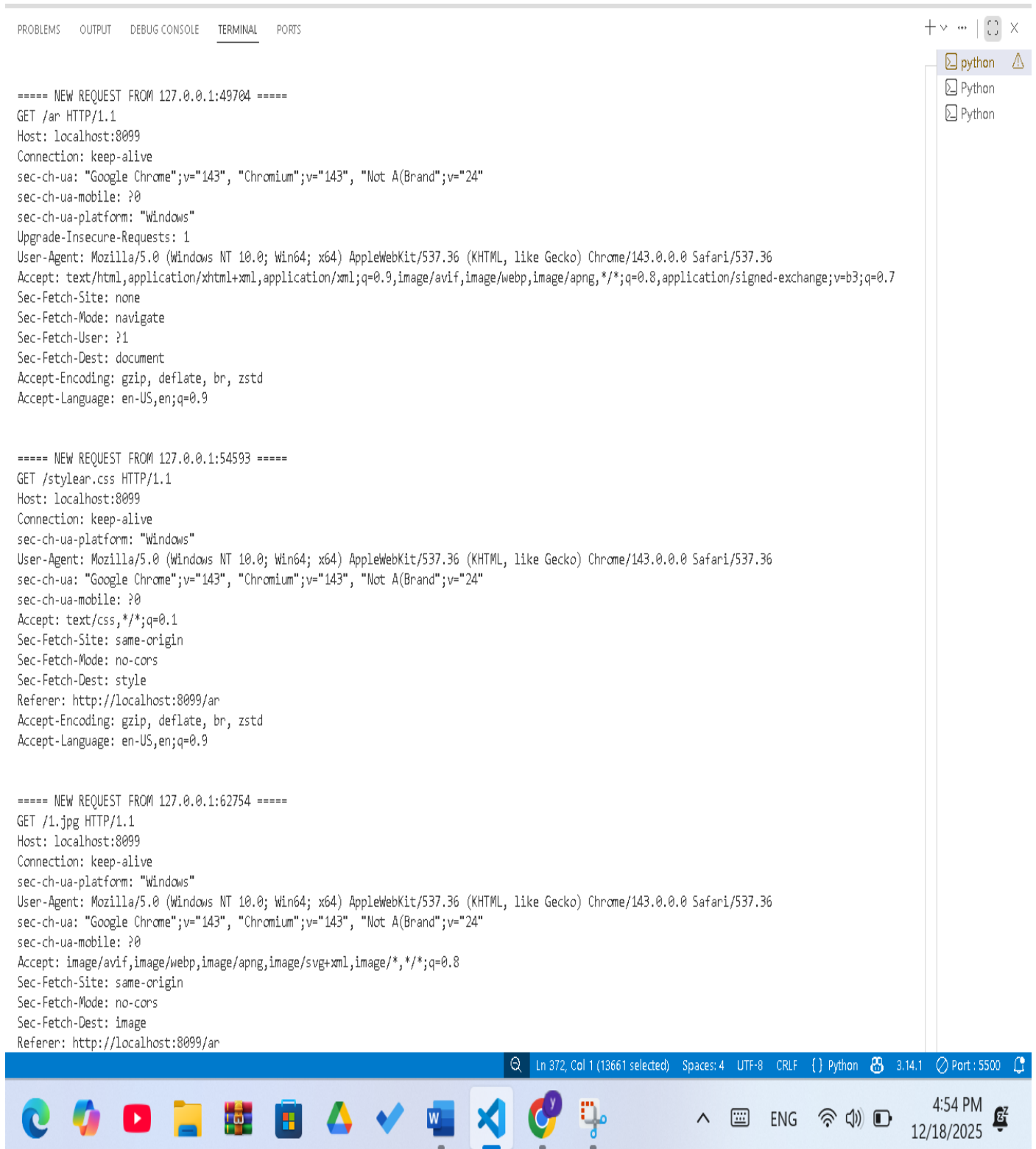


Figure 36: Terminal output showing GET of Arabic page



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

===== NEW REQUEST FROM 127.0.0.1:62666 =====
===== NEW REQUEST FROM 127.0.0.1:57125 =====
GET /2.jpeg HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8099/ar
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

===== NEW REQUEST FROM 127.0.0.1:59026 =====

GET /p.jpg HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8099/ar
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

GET /3.jpeg HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
```

Ln 372, Col 1 (13661 selected) Spaces: 4 UTF-8 CRLF {} Python 3.14.1 Port: 5500

4:55 PM
12/18/2025

Figure 37: Terminal output showing GET of Arabic page

```

Sec-Fetch-Dest: image
Referer: http://localhost:8099/ar
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

===== NEW REQUEST FROM 127.0.0.1:63391 =====
GET /0.png HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8099/stylear.css
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

```



Figure 38: Terminal output showing GET of Arabic page

CSS File Test

URL Tested:

<http://localhost:8099/style.css>

Screenshot:

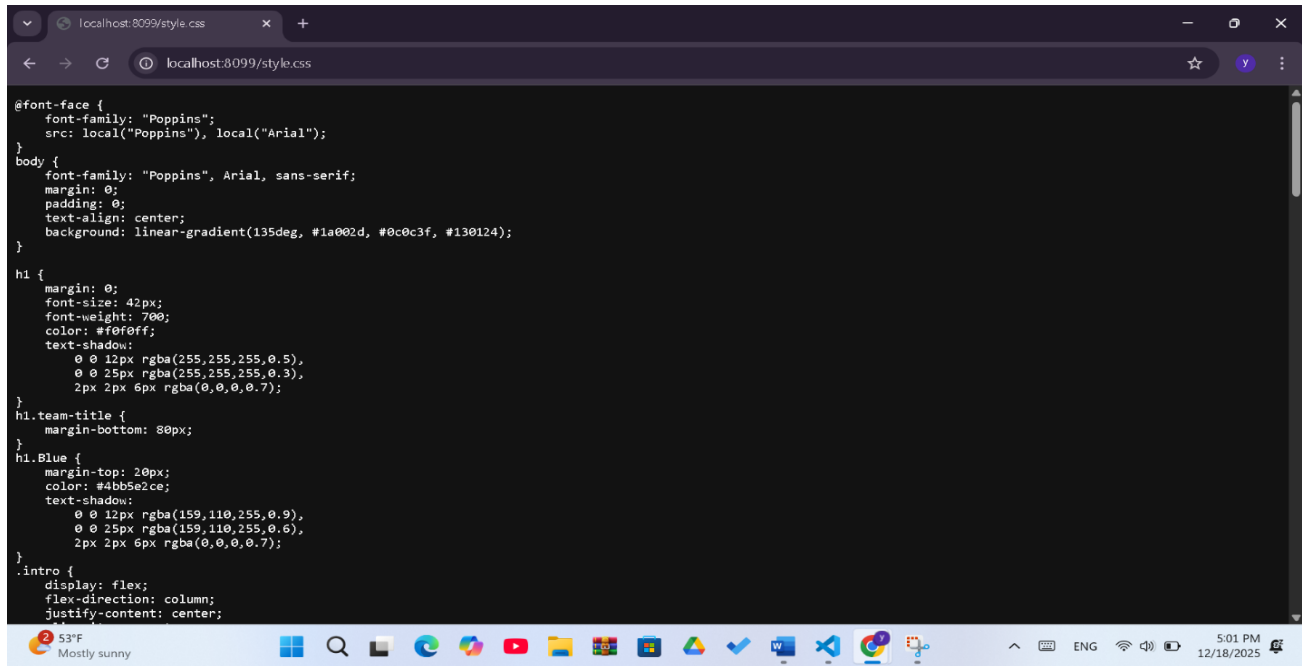


Figure 39: Webpage styled using CSS.

```
==== NEW REQUEST FROM 127.0.0.1:55148 ====
GET /style.css HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9
```



Figure 40: Terminal output showing GET css file

Image Files Test (png/jpg)

URL Tested:

<http://localhost:8099/0.png>

Screenshot:

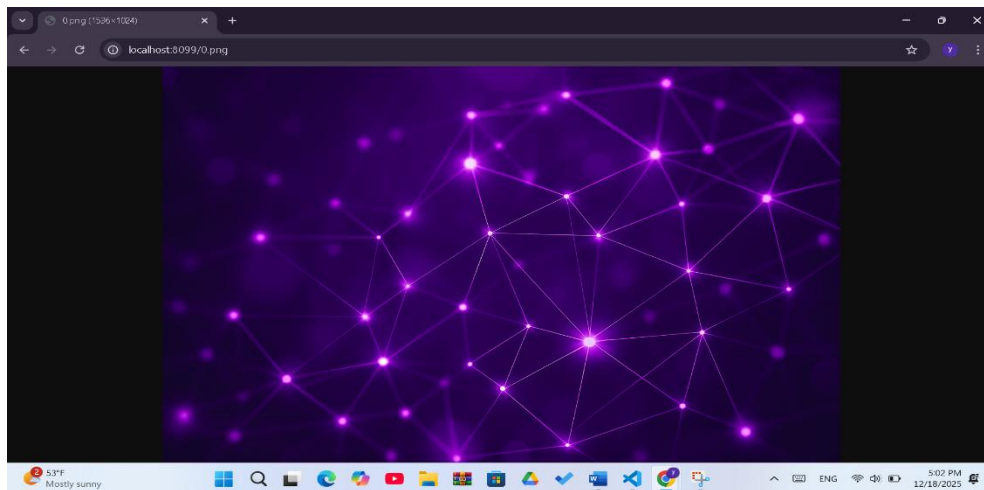


Figure 41: Image file(png) served by the web server.

```

===== NEW REQUEST FROM 127.0.0.1:54077 =====
GET /0.png HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

```



Figure 42: Terminal output showing GET of image png

URL Tested:

<http://localhost:8099/jpgphoto.jpg>

Screenshot:

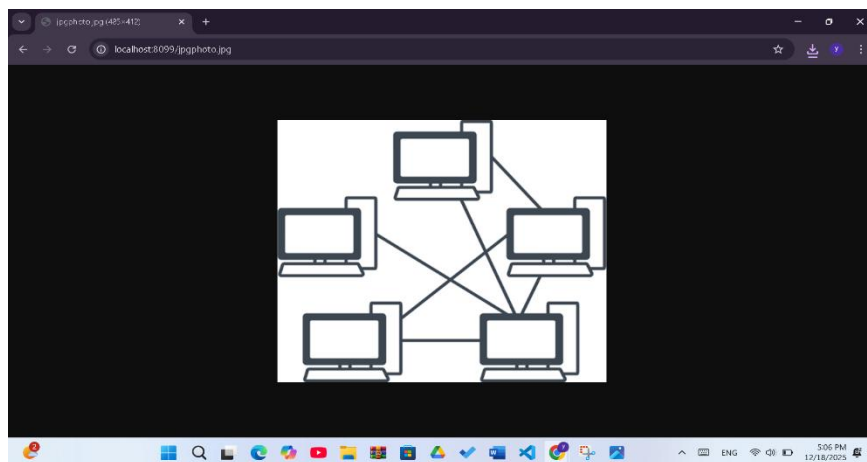


Figure 43: Image file(jpg) served by the web server.

```

===== NEW REQUEST FROM 127.0.0.1:57597 =====
GET /jpgphoto.jpg HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

```



Figure 44: Terminal output showing GET of image jpg.

Page Not Found (404) Test

URL Tested:

http://localhost:8099/invalid

Screenshot:s



The file is not found

Student Names and ID:

Hanaa Jawad - ID: 1230263

Yasmeen Badwan - ID: 1230271

Sewar Masri - ID: 1231240

Client Address:127.0.0.1 : 50414



Figure 45:Custom 404 error page.

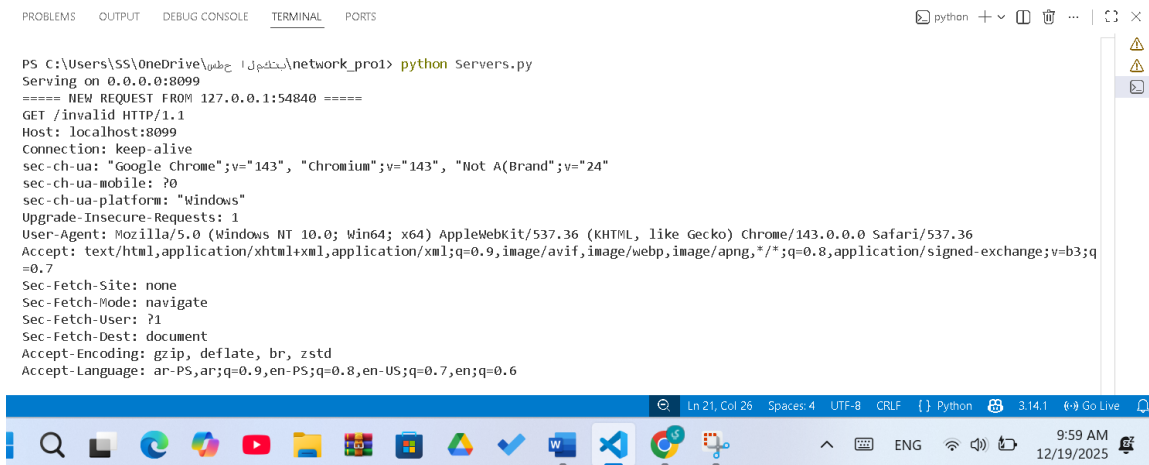


Figure 46:Terminal output showing GET of not found page

307 Redirect test :

URL Tested:

http://localhost:8099/chat

Screenshot:

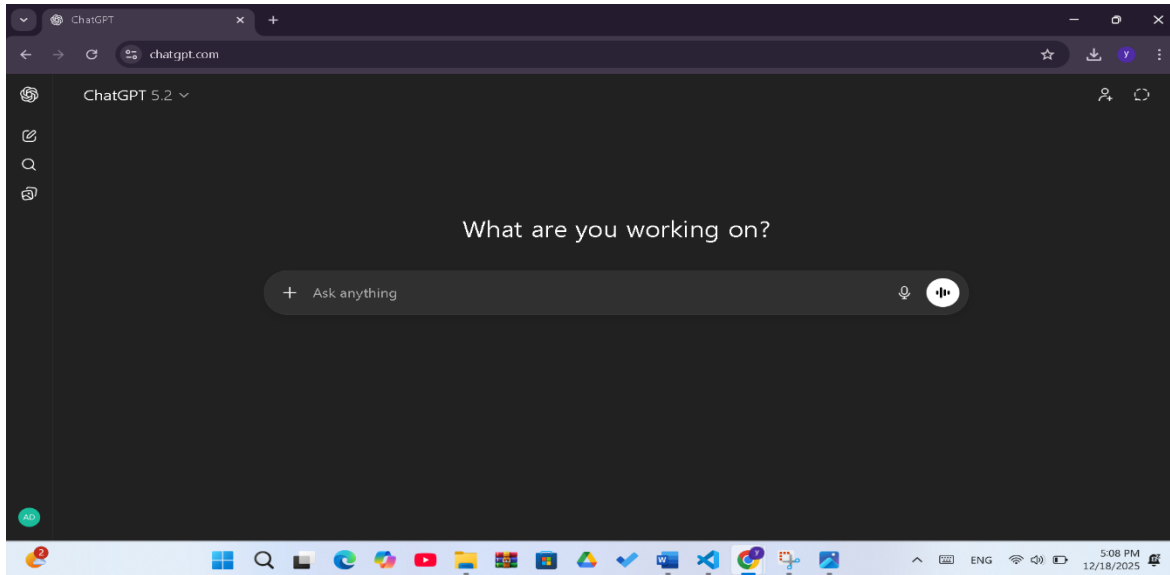


Figure 47: chatgpt page.

```
===== NEW REQUEST FROM 127.0.0.1:62157 =====
GET /chat HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9
```



Figure 48: Terminal output showing GET of chat page

URL Tested:

http://localhost:8099/cf

Screenshot:

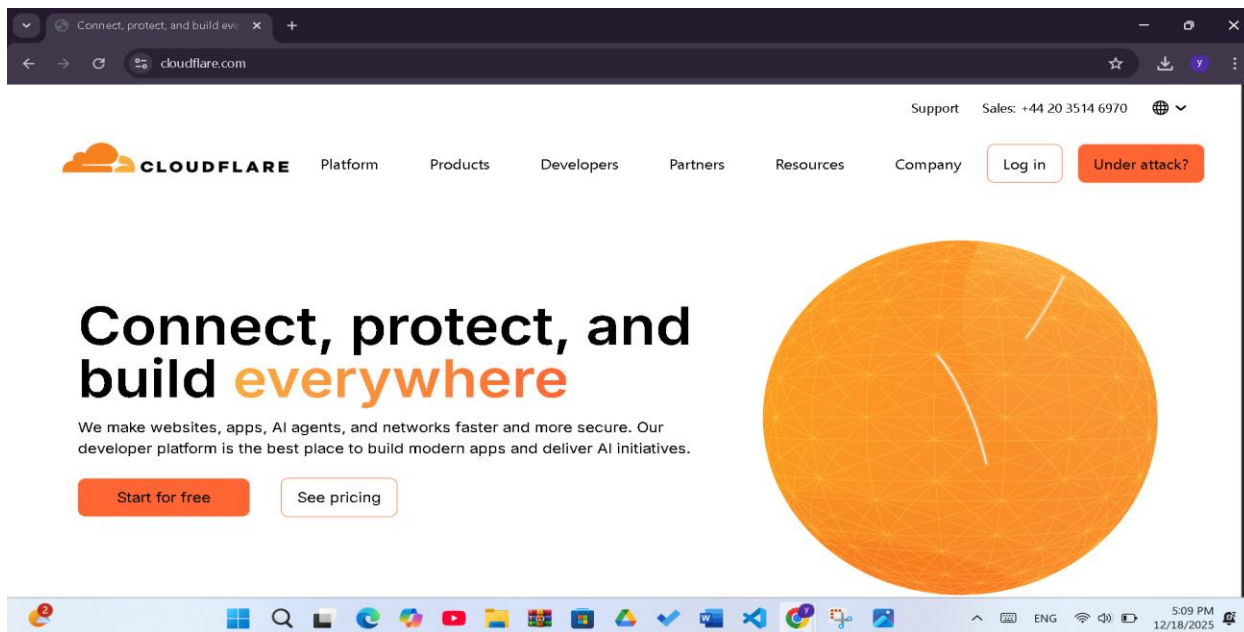


Figure 49: cloudeflare page

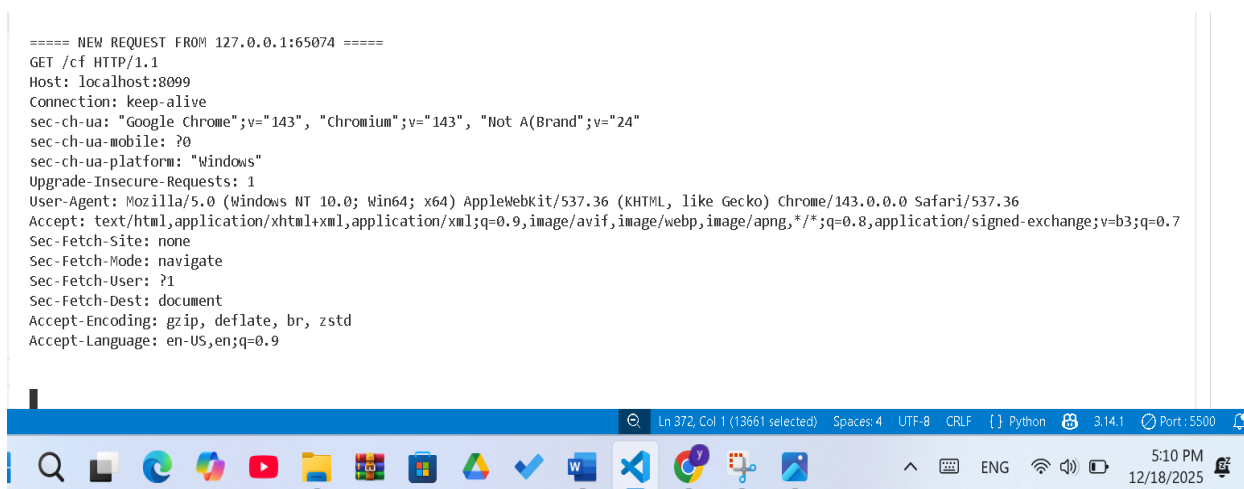


Figure 50: Terminal output showing GET of cloudeflare page

URL Tested:

http://localhost:8099/rt

Screenshot:

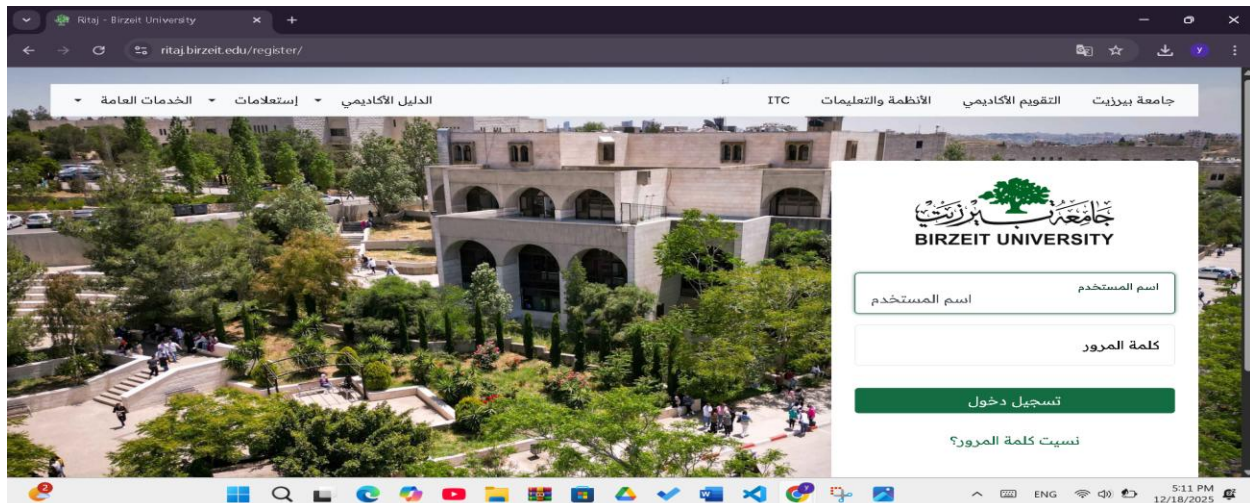


Figure 51:Ritaj page

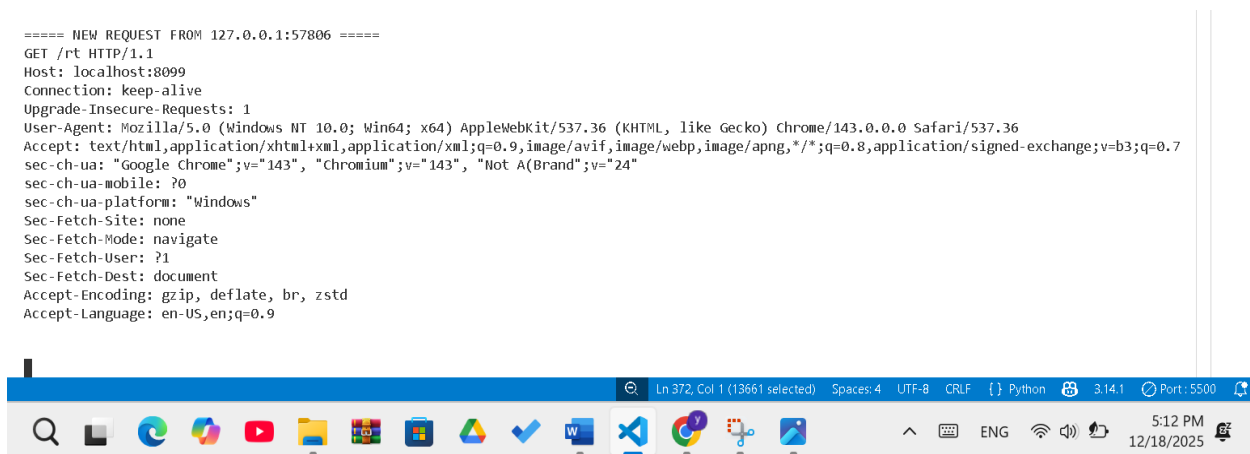


Figure 52: Terminal output showing GET of Ritaj page

When accessing /chat, the server responded with a 307 Temporary Redirect, sending the user to the ChatGPT website. Similarly, requests to /cf and /rt correctly redirected the browser to the Cloudflare and Ritaj websites, respectively. These tests confirm that HTTP redirection is implemented correctly using status code 307.

User Registration Tests

- **Load register.html:**
URL Tested:
`http://localhost:8099/register.html`

Screenshot:

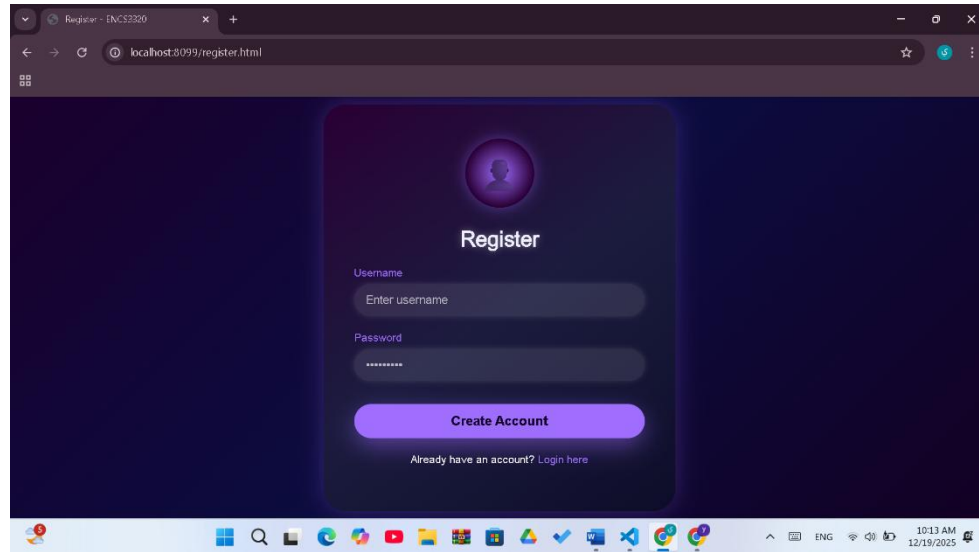


Figure 53:register page

```
===== NEW REQUEST FROM 127.0.0.1:63112 =====
GET /register.html HTTP/1.1
Host: localhost:8099
Connection: keep-alive
sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: ar-PS;ar;q=0.9,en-PS;q=0.8,en-US;q=0.7,en;q=0.6
```



Figure 54:Terminal output showing GET of register page

- **Valid Registration**

A valid registration test was performed by entering a new username and password. The server hashed the password using SHA256 and stored it in the data.txt file. The terminal output confirmed successful registration, and the stored password appeared in hashed form.

Screenshot:

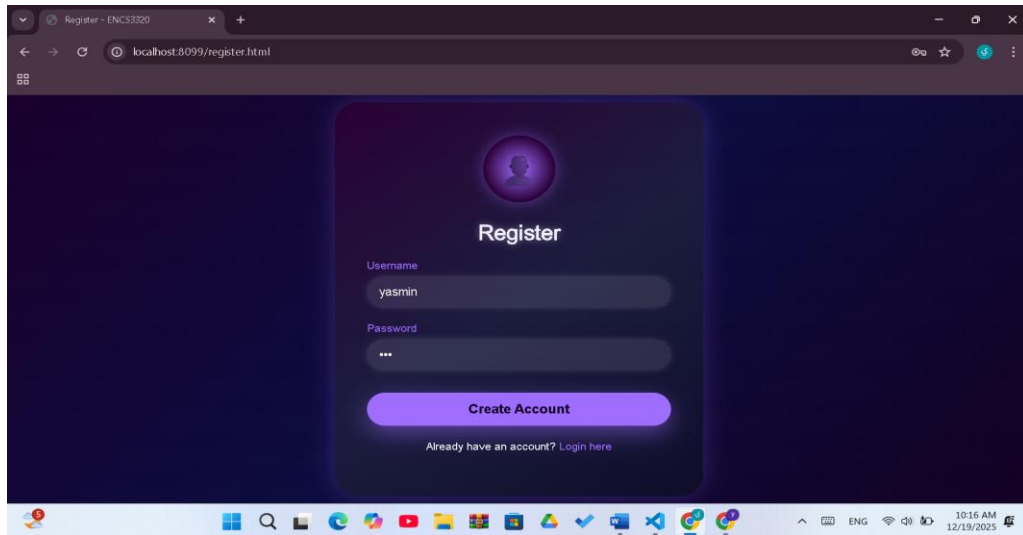


Figure 55:register page with name yasmin and pasword 123



Figure 56:Terminal output showing post of register page



Figure 57:data.txt with hashed password.

- **Invalid Registration**

An invalid registration attempt was tested by using incorrect or duplicate input. The server rejected the registration and reloaded the registration page, demonstrating proper input validation.

Screenshot:

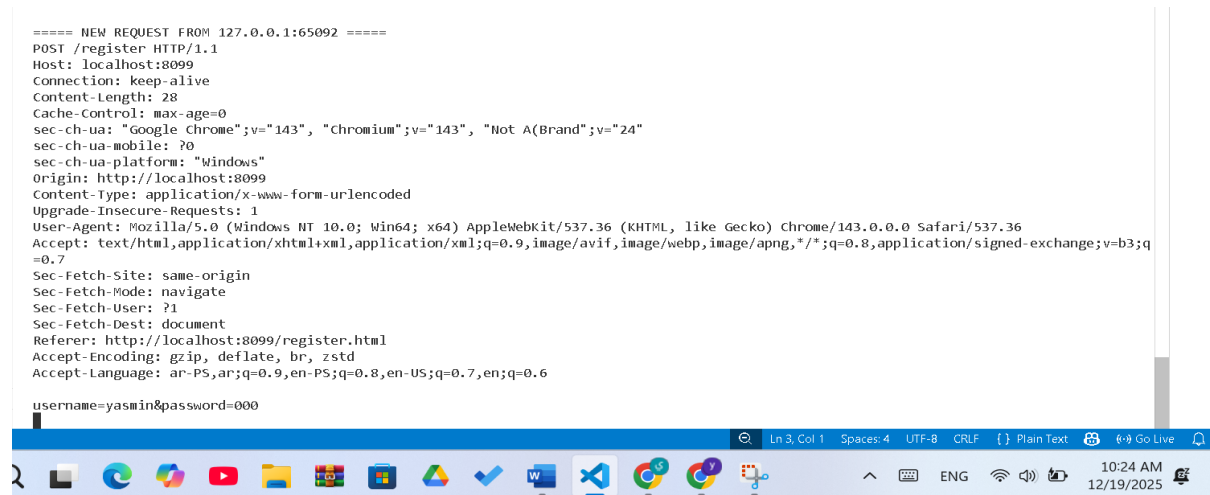


Figure 58:Registration rejected.

User Login Tests

- **Load Login.html:**
URL Tested:
<http://localhost:8099/login.html>

- **Screenshot:**

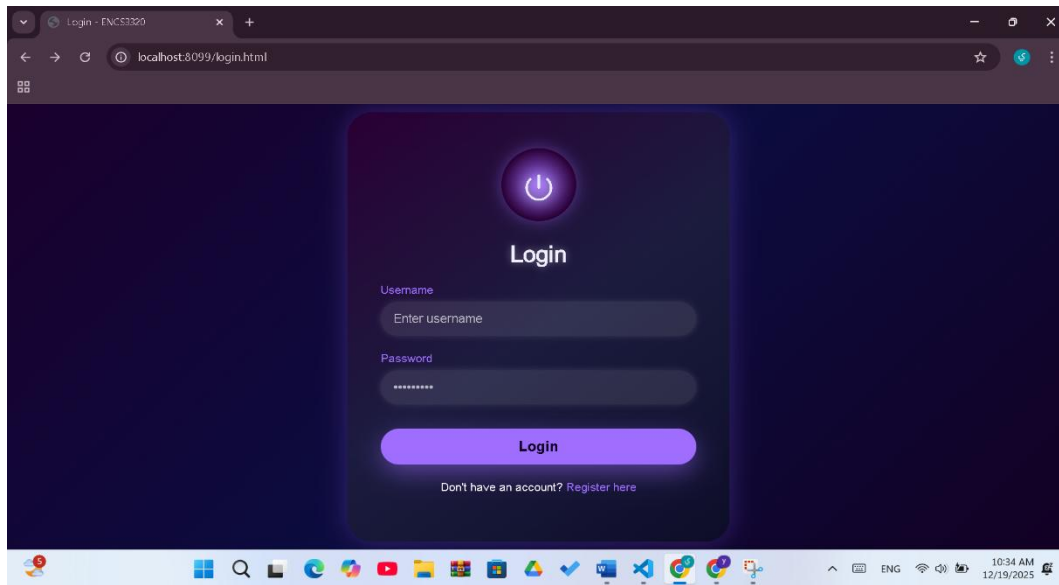


Figure 59:login page

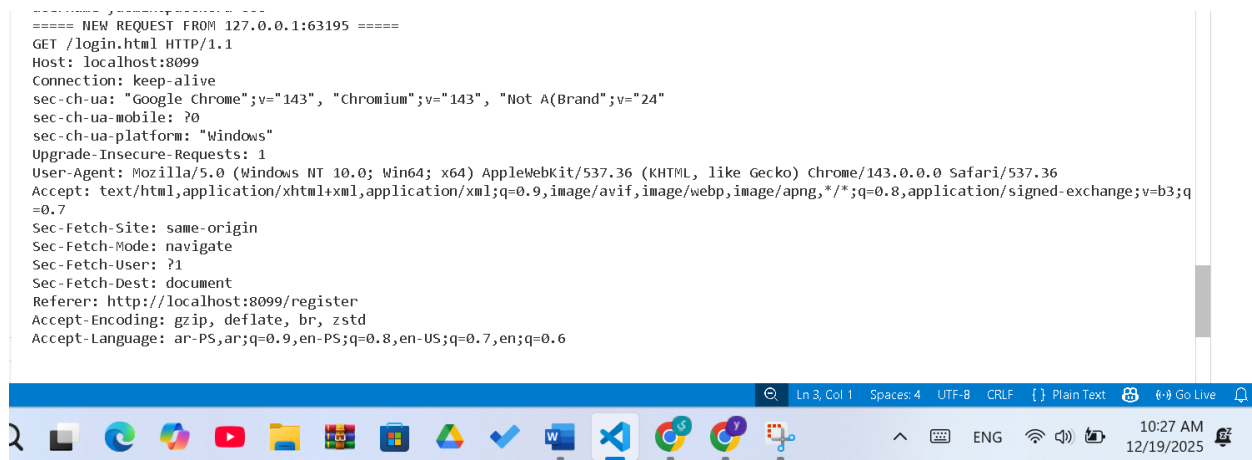


Figure 60:Terminal output showing get of login page

- **Invalid Login**

An invalid login attempt was performed using an incorrect password. The server rejected the login and displayed a failure message, while the terminal showed the corresponding POST request.

Screenshot:

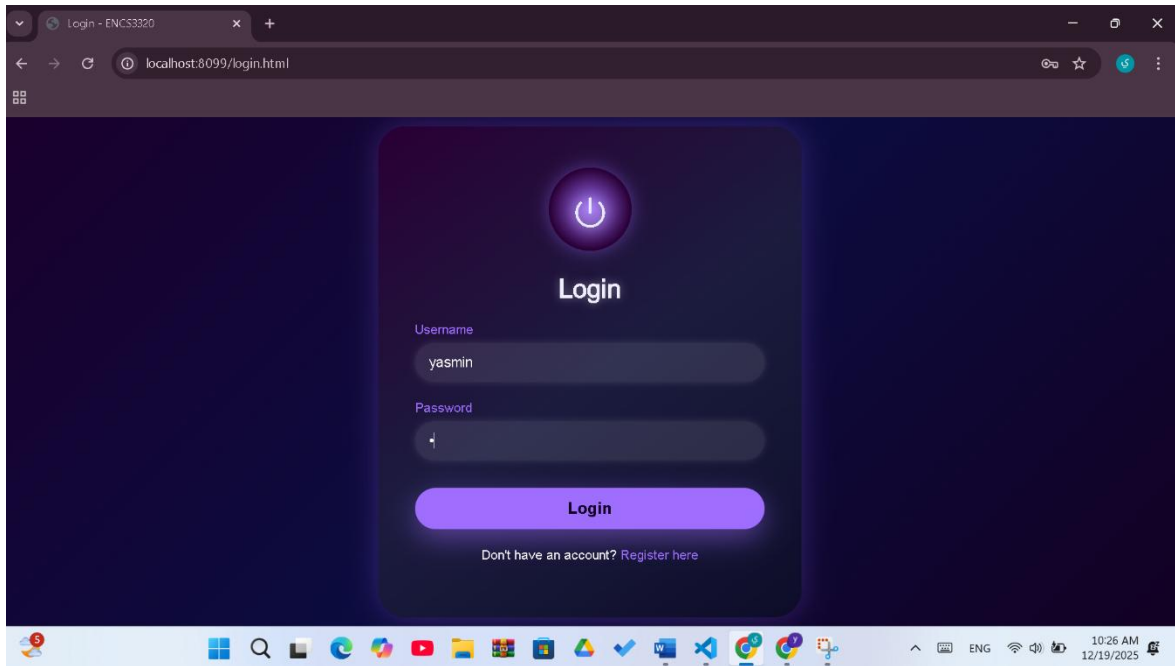
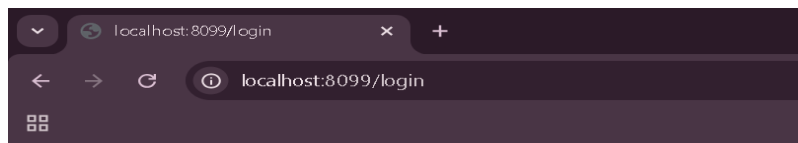


Figure 61: login page with wrong password



Login failed

[Try again](#)

Figure 62: login fail page

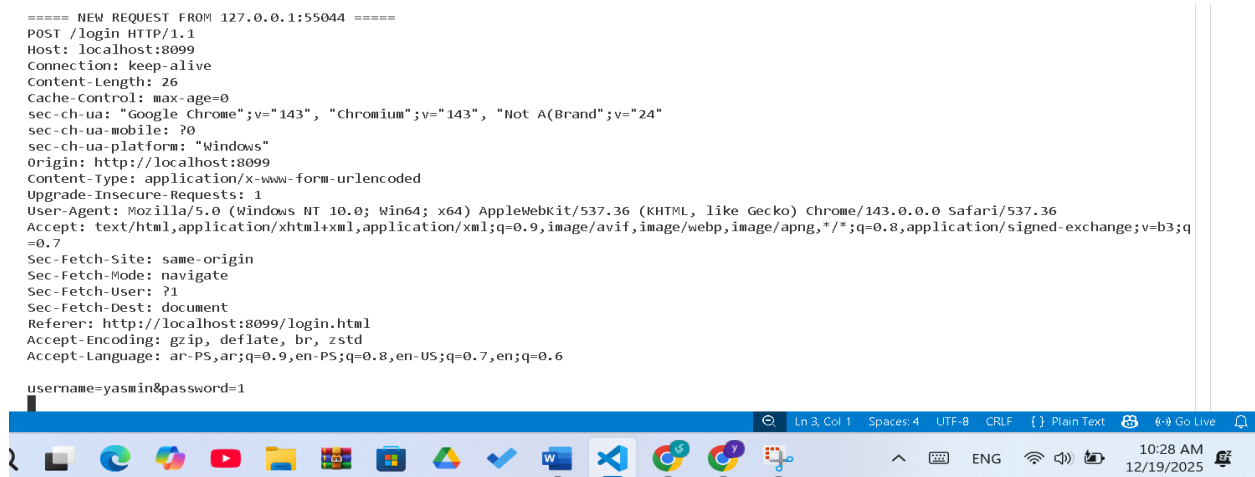


Figure 63: Terminal output showing post of login page

- **Valid Login**

A valid login was tested using correct credentials. The server verified the hashed password, created a unique session ID, and sent it to the browser as a cookie. The protected page was displayed successfully, confirming correct session management.

Screenshot:

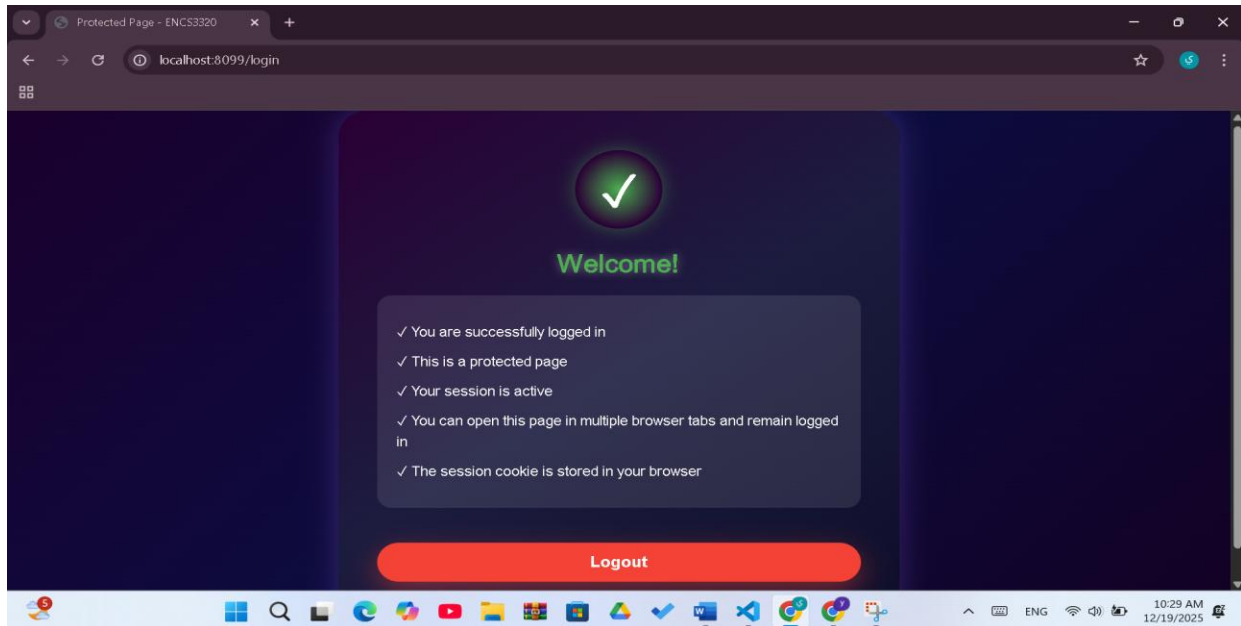


Figure 64: Protected page after login.

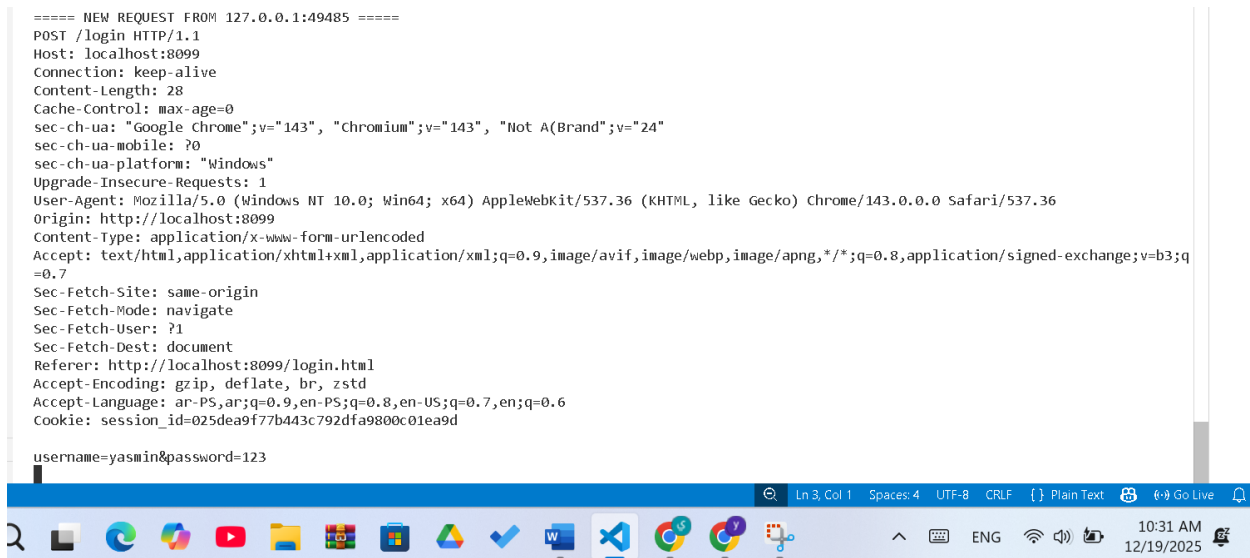


Figure 65: Terminal output showing post of login page &&Session cookie created.

Cookie Validation Test

In this test, the handling of session cookies was verified. After the user logged in successfully, a session cookie was stored in the browser. When the user accessed another page or opened a new browser tab, this cookie was automatically included in the HTTP request headers. The server extracted the session cookie, matched it with an existing entry in the session table, and identified the user as authenticated. Consequently, access to protected pages such as **protected.html** was allowed without requiring the user to log in again. This confirms that cookie-based session validation functions correctly.

Screenshot:

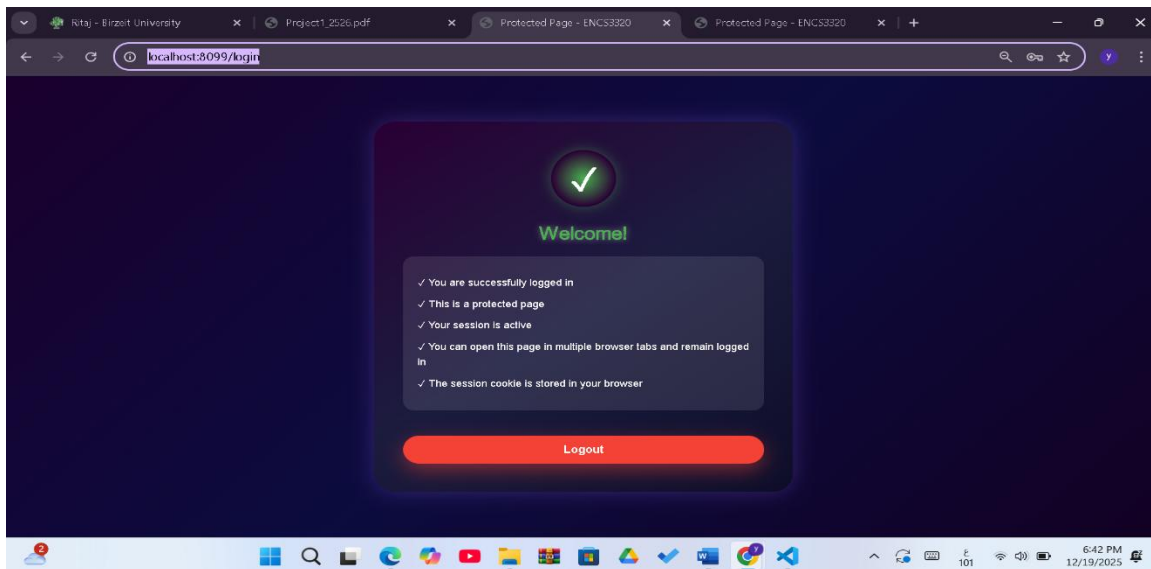


Figure 66: Protected Page Access (Authorization Check)

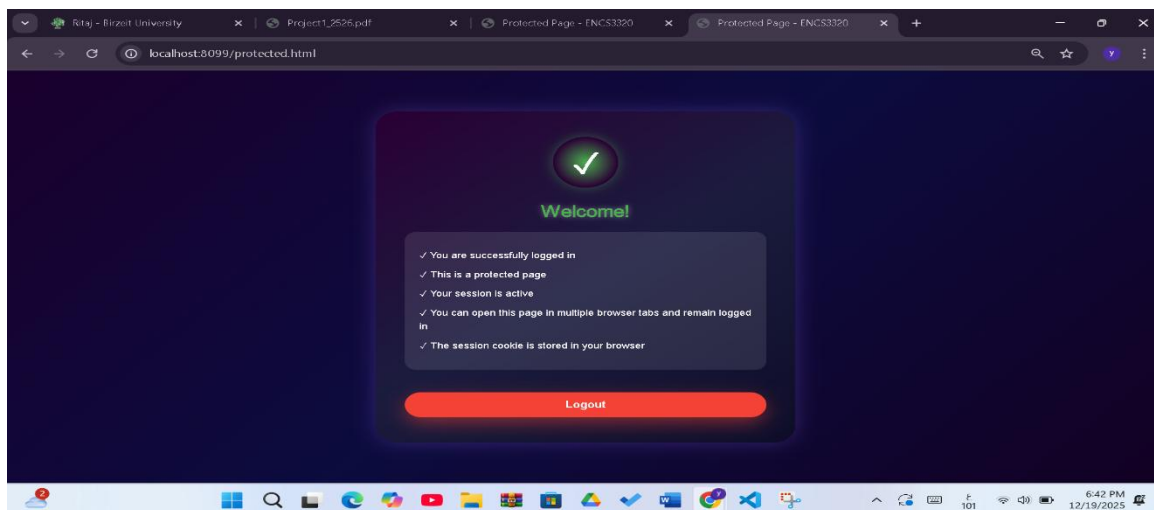


Figure 67: Session Cookie Persistence in New Browser Tab

Logout Test

Finally, the logout functionality was tested. When the user clicked logout, the server removed the session from memory, deleted the session cookie, and redirected the user to the login page. This confirms that session termination and logout redirection work correctly.

Screenshot:

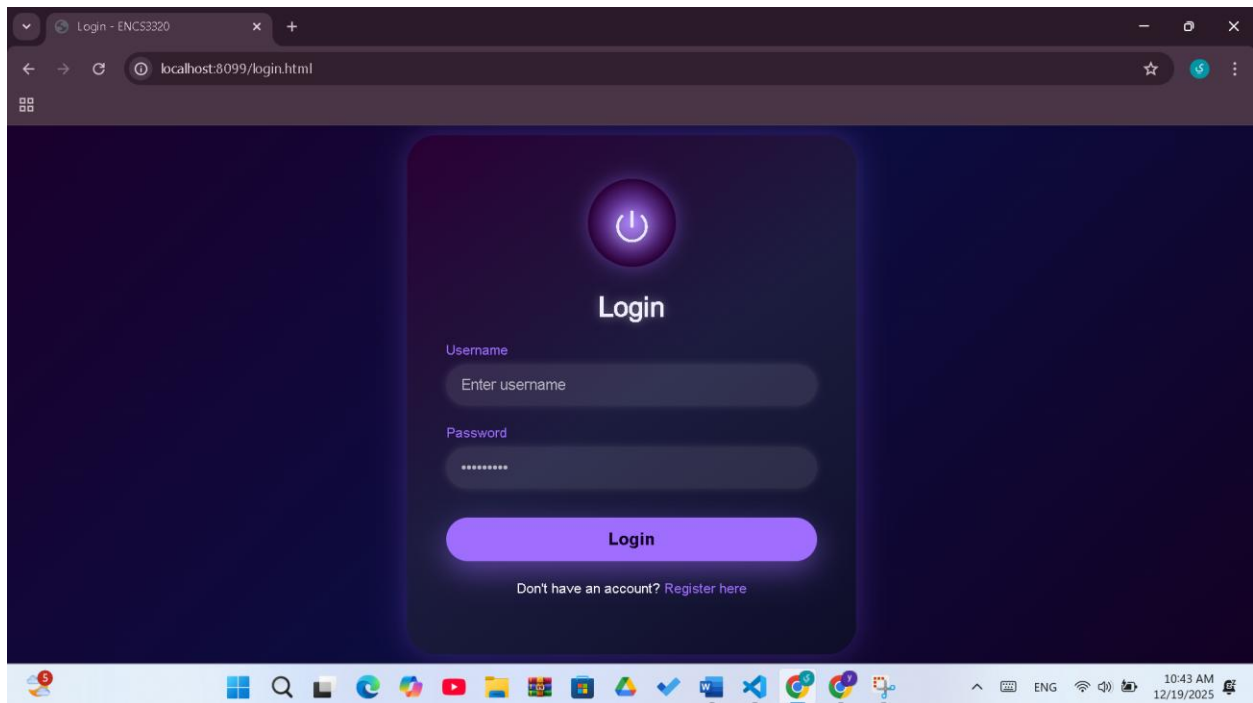


Figure 69:Logout redirection.

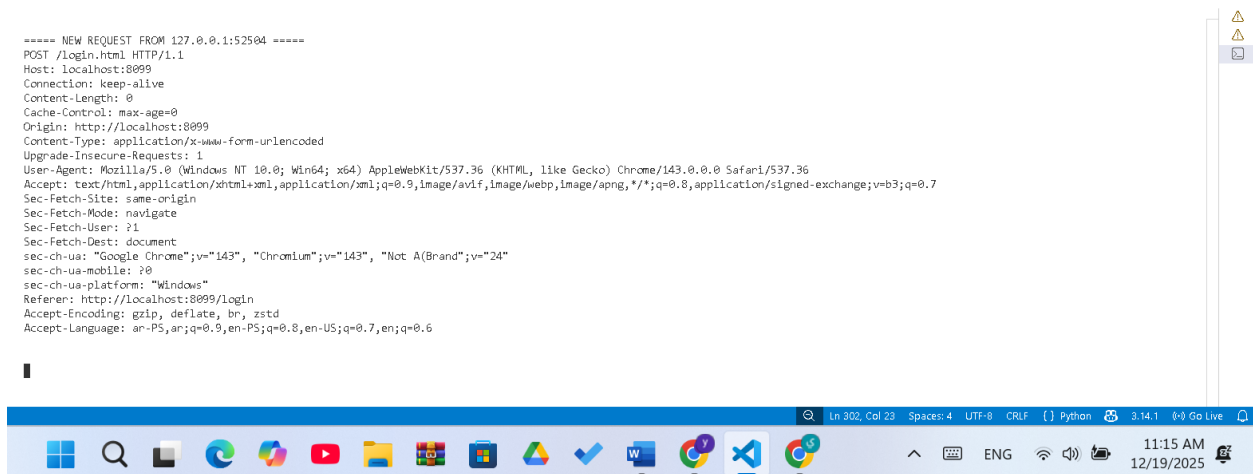


Figure 70:Session cookie removed.

Team work

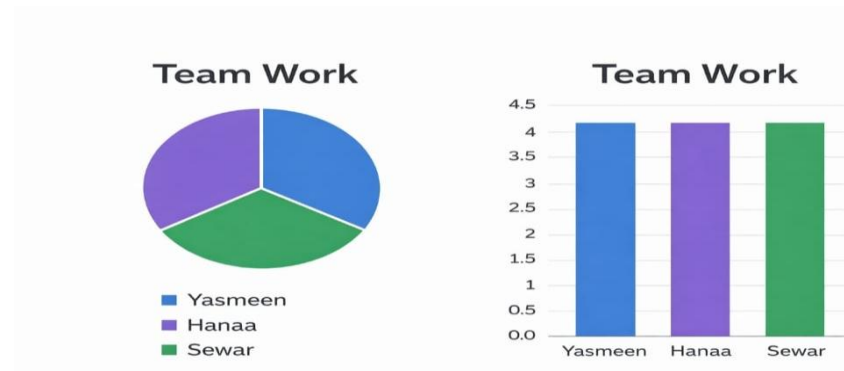


Figure 71: Contribution distribution among team members (Yasmeen, Hanaa, Sewar).

Throughout this project, our team of three students — Yasmeen, Hanaa, and Sewar — maintained strong collaboration and effective communication. We worked together closely during all stages of the project, including planning, designing, implementing, testing, and documenting the web server. Regular discussions were held to divide tasks clearly and ensure that all requirements of the project were met.

Each team member contributed equally to the development of the system. Responsibilities were shared among implementing the server using socket programming, handling HTTP requests and responses, managing user registration and login, implementing session management with cookies, testing different scenarios, and preparing the final project report. This balanced distribution of work ensured consistency and reliability across all components of the project.

Our workflow was highly interactive. We frequently reviewed each other's code, debugged issues together, and tested different cases such as page routing, static file handling, redirections, authentication, and error handling. This collaborative approach helped us identify problems early and improve the overall quality and stability of the web server.

Beyond the technical outcomes, this project significantly strengthened our teamwork and communication skills. Working as a group taught us how to coordinate tasks effectively, solve problems collaboratively, and support one another under time constraints. Overall, this experience enhanced our ability to work efficiently in team-based software engineering projects and improved our confidence in building complete network-based applications.

Conclusion

This project successfully demonstrated fundamental computer networking concepts through practical implementation and experimentation. The results obtained from networking tools such as ping and traceroute illustrated network latency and hop count, while DNS packet captures using Wireshark confirmed the process of resolving domain names into IP addresses.

The project was completed through three integrated parts. In Part One, essential networking tools and concepts were explored, providing a solid understanding of client–server communication. In Part Two, this knowledge was applied by implementing TCP and UDP client–server applications using socket programming, which verified reliable data transmission and packet behavior. In Part Three, the accumulated experience was utilized to build a functional web server using low-level Python sockets, supporting static content delivery, redirection, error handling, user authentication, and session management.

Testing results confirmed that the system operated correctly and that protected resources were accessible only to authenticated users. Overall, this project successfully connected theoretical concepts with real-world applications and enhanced practical skills in networking, web server development, and teamwork.

References:

- [1] [What is Ping? - GeeksforGeeks](#) (Accessed on 30/11/2025)
- [2] [How to Use Tracert/Traceroute Command in Windows - GeeksforGeeks](#)(Accessed on 30/11/2025)
- [3] [What Exactly Is nslookup, and How Does It Work?](#) (Accessed on 1/12/2025)
- [4] [How to Use the telnet Command in Windows for Network Diagnostics? - GeeksforGeeks](#)(Accessed on 1/12/2025)
- [5] [Internet Control Message Protocol \(ICMP\) - GeeksforGeeks](#)(Accessed on 1/12/2025)
- [6] [What is RTT\(Round Trip Time\)? - GeeksforGeeks](#)(Accessed on 2/12/2025)
- [7] [Types of ICMP \(Internet Control Message Protocol\) Messages - GeeksforGeeks](#)(Accessed on 2/12/2025)
- [8] [ICMP Protocol \(Internet Control Message Protocol\): A Guide | Okta](#)(Accessed on 2/12/2025)
- [9] [Introduction to Wireshark - GeeksforGeeks](#)(Accessed on 3/12/2025)
- [10] [Socket in Computer Network - GeeksforGeeks](#)(Accessed on 3/12/2025)
- [11] [Networking and Socket Programming](#)(Accessed on 3/12/2025)
- [12] [Differences between TCP and UDP - GeeksforGeeks](#)(Accessed on 3/12/2025)
- [13] [Hypertext Transfer Protocol - HTTP - GeeksforGeeks](#)(Accessed on 3/12/2025)
- [14] [HTTP response status codes - HTTP | MDN](#)(Accessed on 10/12/2025)
- [15] [Static Content Hosting Pattern - System Design - GeeksforGeeks](#)(Accessed on 10/12/2025)
- [16] [How to work with Session and Cookies in CodeIgniter | TechBriefers](#)(Accessed on 10/12/2025)